

New Proof for Plain OAEP: Post-Quantum Security without Parameter Restrictions or Collision-Resistance

Heming Liao^{1,2} , Jiangxia Ge³ , Shujiao Cao^{1,2}  , and Rui Xue^{1,2} 

¹ State Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100085, China

² School of Cyber Security, University of Chinese Academy of Sciences, Beijing 100049, China

³ China Telecom Quantum Group Ltd., Hefei, China

{liaoheming, caoshujiao, xuerui}@iie.ac.cn, gejiangxia@chinatelecom.cn

Abstract. During NIST’s post-quantum cryptography standardization process, two generic transforms, Fujisaki-Okamoto (FO) and OAEP, are widely used to achieve the IND-CCA security. For instance, the final winner Kyber has utilized FO, and a variant of the 3rd-round finalist NTRU has utilized OAEP. The FO and OAEP are both constructed in the random oracle model (ROM), so to evaluate their post-quantum security, a security proof in the quantum random oracle model (QROM) is required. So far, the QROM security proof of FO has been given and improved by a sequence of works, however, the QROM security proof of OAEP has not been fully explored: current proofs either introduced an extra plaintext-confirming hash to the ciphertext (TCC 2016), or required parameter restrictions and a quantum collision-resistant term (PKC 2022).

In this paper, by reorganizing the proof route, we give a new QROM security proof of plain OAEP. The key techniques used in our proof are the compressed oracle technique proposed by Zhandry (CRYPTO 2019) and the Fixed Permutation One-Way to Hiding recently proposed by Jaeger (Eprint 2024/797), and our proof has the following three advantages:

- Similar to Ebrahimi’s proof (PKC 2022), our proof also achieves the stronger IND-qCCA security, where the decryption oracle can be accessed in superposition.
- The parameter restrictions " $n + k_1 \geq k_0$ " and " $k_0 - n = \mathcal{O}(n)$ " introduced in Ebrahimi’s proof (PKC 2022) are removed.
- The reliance on the collision resistance of quantum random oracles required in Ebrahimi’s proof (PKC 2022) is avoided, and hence our security bound does not introduce the quantum collision-resistant term " $\mathcal{O}(q^3/2^{n+k_1})$ ".

Keywords: quantum random oracle model · security proof · OAEP transform · IND-qCCA security.

1 Introduction

In [35], Shor designed an efficient quantum algorithm to solve the factoring and discrete logarithm problem. This sparked the cryptographic community to

study a new field named post-quantum cryptography (PQC), which aims at constructing classical systems secure against quantum adversaries. In December 2016 [29], National Institute of Standards and Technology (NIST) initiated a project to standardize public-key cryptographic algorithms which are secure against quantum adversaries, including public key encryption (PKE), digital signatures, and key encapsulation mechanisms (KEM). Among the PKE/KEM submissions, the indistinguishability against chosen-ciphertext attacks (IND-CCA) is a widely adopted security goal. To achieve this goal, the Fujisaki-Okamoto (FO) transform [14,15] and Optimal Asymmetric Encryption Padding (OAEP) transform [3], along with their variants, have been frequently used. In particular, the final winner Kyber [30] uses (a tweaked version of) $\text{FO}_m^\perp$, which is a KEM variant of FO, and a variant of the 3rd-round finalist NTRU [31] uses Q-OAEP, which is a variant of OAEP by adding a plaintext-confirming hash.

Compared with FO, OAEP has a more concise structure, since its decryption does not require a re-encryption check. Meanwhile, different from FO, the security proof of OAEP does not require the underlying scheme to be γ -spread. However, a drawback of OAEP is that, unlike FO which only requires the underlying scheme to be one-way secure against chosen-plaintext attacks (OW-CPA), OAEP instead requires a stronger partial domain one-wayness (PD-OW) security. Nevertheless, we are not pessimistic about this situation. Firstly, one may construct PD-OW secure cryptographic primitives based on lattice or code theory. Moreover, OAEP indeed offers scheme designers another option to achieve IND-CCA security besides FO, particularly in scenarios where re-encryption check leads to side-channel vulnerabilities [38]. Overall, FO and OAEP have their own advantages and disadvantages, and their post-quantum security needs to be considered when assessing the post-quantum security of NIST’s submissions.

Notice that FO and OAEP both are transforms in the random oracle model (ROM). Hence by [6], when analyzing their post-quantum security, we should prove their IND-CCA security in the quantum random oracle model (QROM). Up to now, a long sequence of works [37,24,4,27,20,34,16] have studied the IND-CCA security proofs of FO (or its variants) in the QROM. These works gradually optimized the security proof to get tighter security bounds, and [34,16] also considered a stronger security notion named indistinguishability against quantum chosen-ciphertext attacks (IND-qCCA), where the adversary can make decryption/decapsulation oracle queries in superposition.

On the other hand, however, research on the QROM security proof of OAEP is not yet in-depth. Ebrahimi and Unruh [37] provided the IND-CCA security proof of Q-OAEP in the QROM, where Q-OAEP is a variant of OAEP obtained by adding an extra plaintext-confirming hash to the ciphertext. Later, Ebrahimi [13] presented the first QROM security proof of plain OAEP, which also achieved the stronger IND-qCCA security. However, Ebrahimi’s proof additionally requires the parameters n, k_0, k_1 used by OAEP to satisfy $n + k_1 \geq k_0$ and $k_0 - n = \mathcal{O}(n)$, and its security bound has a quantum collision-resistant term $\mathcal{O}(q^3/2^{n+k_1})$ since his proof relies on the collision resistance of quantum random oracles. This makes his proof looser than the existing proof of FO [24,20,34]. Based on this observa-

tion, in this paper we focus on the following natural question:

In the QROM, can we prove the IND-CCA security of plain OAEP, without parameter restrictions or the quantum collision-resistant term?

1.1 Our Contribution

We give an affirmative answer to the above question. In this paper, we present a new QROM security proof of plain OAEP, and similar to [13], we also achieve a stronger IND-qCCA security. The detailed security bound is shown in Table 1.

Table 1. Security bounds in the QROM. Here q is the total number of queries to random oracles, and q_D is the total number of queries to the decryption oracle. The "Requirement" shows the property needed to be satisfied by the underlying scheme. ϵ denotes the security bound of the underlying scheme. n, k_0, k_1 and γ denote the parameters for the corresponding transforms, and WLOG we assume $n = k_0 = k_1 = \gamma$.

Transform	Underlying security	Achieved security	Requirement	Security bound ($\approx$)
FO [34]	OW-CPA	IND-CCA	γ -spread	$(q_D + q) \cdot \sqrt{\epsilon} + q \cdot \sqrt{2^{-\gamma^a}}$
OAEP [13]	PD-OW	IND-qCCA	$n + k_1 \geq k_0$ $k_0 - n = \mathcal{O}(n)$	$q \cdot \sqrt{\epsilon} + q \cdot \sqrt{2^{-k_0}} + q_D \cdot \sqrt{2^{-k_1}}$ $+ q^3 / 2^{n+k_1}$
OAEP Thm.4	PD-OW	IND-qCCA	$\setminus$	$q_D \cdot q \cdot \sqrt{\epsilon} + q_D \cdot q \cdot \sqrt{2^{-k_0}} + q_D \cdot \sqrt{2^{-k_1}}$

^a In [34], the security bound of FO also contains another parameter, ϵ^{sy} , which denotes the security bound of the underlying secret key scheme. Here we omit it WLOG.

As shown in Table 1, our bound avoids the quantum collision-resistant term $q^3 / 2^{n+k_1}$. So at least in the case where q_D is relatively small (e.g. a small constant⁴), our bound is comparable with that of FO in [34] and is tighter than that in [13]. However, when q_D is large, our bound is looser than that of FO in [34], and in this case it seems hard to compare our bound with that in [13].

Nevertheless, regardless of whether q_D is small or large, our proof does not impose additional parameter restrictions (i.e. $n + k_1 \geq k_0$ and $k_0 - n = \mathcal{O}(n)$) compared with [13]. Roughly speaking, setting $n = 256$, the parameter restrictions in [13] will make $k_0 = 3 * 256$, $k_1 = 2 * 256$. Consequently, to encrypt 256-bit plaintexts using OAEP, the input length of the underlying injective function f_{pk} should be $n + k_1 + k_0 = 6 * 256$, and hence the length of the ciphertext c will not be shorter than 6×256 since c is the output of f_{pk} . Fortunately, in our proof

⁴ There do exist scenarios where the number of decryption/decapsulation queries q_D is small, see [11, 25, 40, 18, 21, 23] for instance. These works focus on certain protocols in which IND-t-CCA secure PKEs/KEMs are sufficient, where IND-t-CCA is the same security notion as IND-CCA except that the adversary can only query the decryption/decapsulation oracles at most t times. Here t is a small constant independent of the security parameter.

$n = k_1 = k_0 = 256$ is enough to encrypt 256-bit plaintexts, which will result in a scheme with higher computational efficiency and smaller ciphertext size.

Remark 1. We would like to point out that the corresponding proof in [13], which leads to the highlighted term $q \cdot \sqrt{\epsilon} + q \cdot \sqrt{2^{-k_0}}$, may contain a potential gap. A detailed discussion is provided in Supplementary Material A.

1.2 Technical Overview

We begin by briefly reviewing the structure of OAEP. Let $\text{TDI} = (\text{Gen}, f, f^{-1})$ be a trapdoor injective function (Definition 6). Let $G : \{0, 1\}^{k_0} \rightarrow \{0, 1\}^{n+k_1}$ and $H : \{0, 1\}^{n+k_1} \rightarrow \{0, 1\}^{k_0}$ be two random oracles. The structure of the encryption and decryption process of OAEP is shown in Fig. 1.

The encryption process of OAEP first uses a 2-round Feistel network to transform the plaintext m into two components (s, t) , and then computes the ciphertext $c = f_{\text{pk}}(s, t)$. The structure of the decryption process of OAEP is symmetric to that of the encryption: after receiving c , first derive s and t by computing $f_{\text{sk}}^{-1}(c)$, and then invert the 2-round Feistel network to recover M . Finally, check the redundancy 0^{k_1} added to m during encryption. If $[M]_{k_1} = 0^{k_1}$, return $[M]^n$; otherwise, return $\perp$.

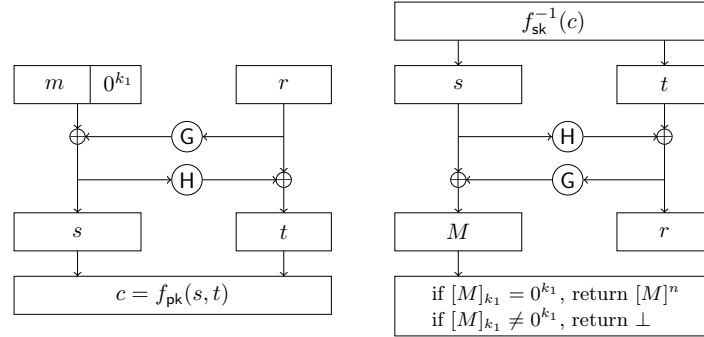


Fig. 1. The encryption and decryption structure of OAEP. Here (pk, sk) is generated by Gen , m denotes the plaintext, and r is the randomness of encrypting m .

We now briefly outline our new proof route. In the QROM, we complete the IND-qCCA security proof of OAEP through 17 games⁵, among which $\mathbf{G}_5$, $\mathbf{G}_{14}$, and $\mathbf{G}_{17}$, shown in Fig. 2, serve as three core games. Based on these core games, our proof can be divided into the following three steps:

1. **Games $\mathbf{G}_0$ - $\mathbf{G}_5$:** Game $\mathbf{G}_0$ is the IND-qCCA game of OAEP, where the simulation of decryption oracle qDec requires the secret key sk . In this step, by

⁵ The number of games may seem a bit large, but this is reasonable, as some game hops are conceptual steps designed to improve the clarity of the proof.

- simulating G and H as compressed oracles, we gradually modify qDec and finally obtain qDec_{sk} in game $\mathbf{G}_5$, where qDec_{sk} no longer depends on sk .
2. **Games $\mathbf{G}_5$ - $\mathbf{G}_{14}$:** In this step, we reprogram the oracles G and H queried before $\mathcal{A}$ outputs (m_0^*, m_1^*) . After this reprogramming, we get game $\mathbf{G}_{14}$, where $G(r^*)$ and $H(s^*)$ used to generate c^* and b' are replaced by fresh uniformly random values g^* and h^* . Moreover, since G and H have been simulated as compressed oracles, the unitary operations $U_{g^*}^*$ (Eq. (15)) and $U_{h^*}^*$ (Eq. (19)) must be applied in game $\mathbf{G}_{14}$ to maintain consistency.
 3. **Games $\mathbf{G}_{14}$ - $\mathbf{G}_{17}$:** In this step, based on game $\mathbf{G}_{14}$, we reprogram the oracles $G[r^* \mapsto g^*]$ and $H[s^* \mapsto h^*]$ queried during the generation of b' . After this reprogramming, we obtain game $\mathbf{G}_{17}$, where $G[r^* \mapsto g^*]$ and $H[s^* \mapsto h^*]$ are replaced by the original G and H , respectively, and the implementation of $U_{g^*}^*$ and $U_{h^*}^*$ is canceled. Besides, s^* (resp. t^*) in game $\mathbf{G}_{17}$ is no longer computed based on g^* (resp. h^*), but is instead uniformly sampled.

At this point, we have $\Pr[1 \leftarrow \mathbf{G}_{17}] = 1/2$, and by combining the bounds derived in each step, we obtain the IND-qCCA security bound of OAEP and complete the proof. In the following, we will provide a high-level explanation of our techniques for each step.

GAME $\mathbf{G}_5$	GAME $\mathbf{G}_{14}$	GAME $\mathbf{G}_{17}$
1. $(\text{pk}, \text{sk}) \leftarrow \text{Gen}$ uniformly sample r^*	1. $(\text{pk}, \text{sk}) \leftarrow \text{Gen}$ uniformly sample r^*, g^*, h^*	1. $(\text{pk}, \text{sk}) \leftarrow \text{Gen}$ uniformly sample r^*, s^*, t^*
2. $(m_0^*, m_1^*) \leftarrow \mathcal{A}^{G, H, \text{qDec}_{\text{sk}}}$	2. $(m_0^*, m_1^*) \leftarrow \mathcal{A}^{G, H, \text{qDec}_{\text{sk}}}$	2. $(m_0^*, m_1^*) \leftarrow \mathcal{A}^{G, H, \text{qDec}_{\text{sk}}}$
3. $b \leftarrow_{\$} \{0, 1\}$ $s^* = (m_b^* 0^{k_1}) \oplus G(r^*)$ $t^* = r^* \oplus H(s^*)$ $c^* = f_{\text{pk}}(s^*, t^*)$	3. $b \leftarrow_{\$} \{0, 1\}$ $s^* = (m_b^* 0^{k_1}) \oplus g^*$ $t^* = r^* \oplus h^*$ $c^* = f_{\text{pk}}(s^*, t^*)$, perform $U_{h^*}^* \circ U_{g^*}^*$	3. $b \leftarrow_{\$} \{0, 1\}$ $c^* = f_{\text{pk}}(s^*, t^*)$
4. $b' \leftarrow \mathcal{A}^{G, H, \text{qDec}_{\text{sk}}}(c^*)$	4. $b' \leftarrow \mathcal{A}^{G[r^* \mapsto g^*], H[s^* \mapsto h^*], \text{qDec}_{\text{sk}}}(c^*)$	4. $b' \leftarrow \mathcal{A}^{G, H, \text{qDec}_{\text{sk}}}(c^*)$
5. return $\llbracket b' = b \rrbracket$	5. return $\llbracket b' = b \rrbracket$	5. return $\llbracket b' = b \rrbracket$

Fig. 2. Three core games $\mathbf{G}_5$, $\mathbf{G}_{14}$ and $\mathbf{G}_{17}$. Here for simplicity and clarity we only present informal descriptions of the corresponding games defined in Section 4. Moreover, the $G[r^* \mapsto g^*]$ (resp. $H[s^* \mapsto h^*]$) used in game $\mathbf{G}_{14}$ is the same as G (resp. H) except that it directly replies g^* (resp. h^*) for the query r^* (resp. s^*).

Techniques in the game hop from $\mathbf{G}_0$ to $\mathbf{G}_5$: In this game hop, we introduce two variants of qDec : qDec_1 and qDec_2 , as the bridge transforming qDec to qDec_{sk} ⁶. The construction of qDec_1 and qDec_2 is shown in Fig. 3.

⁶ In fact, the notations " qDec_1 ", " qDec_2 ", and " qDec_{sk} " were not used in our proof. They are introduced here only for the convenience of explaining the techniques.

- Compared with qDec , qDec_1 simulates G as a compressed oracle and adds a judgment for the query state $|c, y, D_G\rangle$: if $D_G(r) = \perp$, qDec_1 returns $\perp$, otherwise the next " $M = s \oplus G(r)$ " is replaced by " $M = s \oplus D_G(r)$ ". Intuitively speaking, if $D_G(r) = \perp$, the compressed oracle simulation of G on r will purify $G(r)$ into a uniform superposition $\sqrt{2^{-n-k_1}} \sum_{l \in \{0,1\}^{n+k_1}} |l\rangle$. Obviously, for a fixed s , only 2^n out of 2^{n+k_1} l in this superposition satisfy $[s \oplus l]_{k_1} = 0^{k_1}$. Hence, even if qDec_1 returns $\perp$ when $D_G(r) = \perp$, the probability that such change is detected cannot exceed $2^n / 2^{n+k_1} = 2^{-k_1}$. Moreover, if $D_G(r) \neq \perp$, we can indeed replace $G(r)$ by $D_G(r)$. The reason is that, roughly speaking, for a state $|c, y, D_G\rangle$ with $D_G(r) \neq \perp$, the probability that $G(r)$ computed by the compressed oracle does not equal to $D_G(r)$ is negligible. Thus with a negligible error we can replace qDec with qDec_1 . Indeed, we compute the difference between qDec and qDec_1 in the proof of Lemma 1, and find that it can be bounded by $8 \cdot \sqrt{2^{-k_1}}$. The square root arises because both qDec and qDec_1 can be accessed in superposition.

$\text{qDec}_1(c, y, D_G\rangle)$	$\text{qDec}_2(c, y, D_G, D_H\rangle)$
1. if $f_{\text{sk}}^{-1}(c) = \perp$, return $ c, y \oplus \perp, D_G\rangle$	1. if $f_{\text{sk}}^{-1}(c) = \perp$, return $ c, y \oplus \perp, D_G, D_H\rangle$
2. else phase $f_{\text{sk}}^{-1}(c)$ into $s t$ $r = t \oplus H(s)$ if $D_G(r) = \perp$, return $ c, y \oplus \perp, D_G\rangle$ else $M = s \oplus D_G(r)$	2. else phase $f_{\text{sk}}^{-1}(c)$ into $s t$ if $D_H(s) = \perp$, return $ c, y \oplus \perp, D_G, D_G\rangle$ else $r = t \oplus D_H(s)$ if $D_G(r) = \perp$, return $ c, y \oplus \perp, D_G, D_H\rangle$ else $M = s \oplus D_G(r)$
3. if $[M]_{k_1} = 0^{k_1}$, return $ c, y \oplus [M]^n, D_G\rangle$ else return $ c, y \oplus \perp, D_G\rangle$	3. if $[M]_{k_1} = 0^{k_1}$, return $ c, y \oplus [M]^n, D_G, D_H\rangle$ else return $ c, y \oplus \perp, D_G, D_H\rangle$

Fig. 3. Decryption oracles qDec_1 and qDec_2 . Since they are quantum accessible, we write their query state as $|c, y\rangle$. When G or H is simulated as a compressed oracle, we attach the corresponding database state $|D_G\rangle$ or $|D_H\rangle$ to $|c, y\rangle$. We also stress that we omit the judgment whether c equals to the challenge ciphertext c^* for simplicity.

- Based on qDec_1 , we make a similar change in qDec_2 , which simulates H as a compressed oracle and adds a judgment for the query state $|c, y, D_G, D_H\rangle$: if $D_H(s) = \perp$, qDec_2 returns $\perp$, otherwise the next " $r = t \oplus H(s)$ " is replaced by " $r = t \oplus D_H(s)$ ". The difference between qDec_1 and qDec_2 is analogous to that between qDec and qDec_1 . Roughly speaking, if $D_H(s) = \perp$, $H(s)$ computed by the compressed oracle is uniformly random, making $D_G(r) \neq \perp$ ($r = t \oplus H(s)$) hard to achieve since there are at most q_G different x satisfying $D_G(x) \neq \perp$. So under a negligible error we can directly return $\perp$ when $D_H(s) = \perp$ in qDec_2 . For the case that $D_H(s) \neq \perp$, similar to the change from $G(r)$ to $D_G(r)$ in qDec_1 , we can change $H(s)$ to $D_H(s)$ in qDec_2 with a negligible error. In fact,

we compute the difference between qDec_1 and qDec_2 in the proof of Lemma 2, and find that it is bounded by $8\sqrt{q_G} \cdot \sqrt{2^{-k_0}}$.

- If qDec_2 does not return $\perp$ for the query state $|c, y, D_G, D_H\rangle$, we know that there must exist (r, s) s.t. $D_G(r) \neq \perp$, $D_H(s) \neq \perp$ and $f_{\text{sk}}^{-1}(c) = (s, r \oplus D_H(s))$. That is, the inverse $(s, t) (= f_{\text{sk}}^{-1}(c))$ of c must already be stored in the databases D_G and D_H as it is satisfied that $D_H(s) \neq \perp$ and $D_G(t \oplus D_H(s)) \neq \perp$. Based on this observation, by using the injectivity of f_{pk} and ignoring the decryption error of the underlying TDI⁷, in Supplementary Material D.4 we show that we can equivalently transform qDec_2 into qDec_{sk} , which does not use the secret key sk , to compute $f_{\text{sk}}^{-1}(c)$ and instead directly searches $(r, D_G(r) \neq \perp)$, $(s, D_H(s) \neq \perp)$ from D_G, D_H such that $f_{\text{pk}}(s, r \oplus D_H(s)) = c$ and $[s \oplus D_G(r)]_{k_1} = 0^{k_1}$, and then returns $[s \oplus D_G(r)]^n$ (resp. $\perp$) if this search succeeds (resp. fails).

At this point, the decryption oracle qDec_{sk} used in game $\mathbf{G}_5$ no longer uses sk .

Remark 2 (How we avoid the parameter restrictions and the quantum collision-resistant term). In [13], the author noticed that the search over the databases (like qDec_1 , qDec_2 and qDec_{sk} we described above) can disturb the simulation of the compressed oracle, which may be detected by the adversary. Hence [13] used the online extraction technique in [12] to bound the potential disturbances, which introduces a special parameter $\Gamma_{R_H^c}$. In order to bound $\Gamma_{R_H^c}$, [13] transformed $G : \{0, 1\}^{k_0} \rightarrow \{0, 1\}^{n+k_1}$ into an injective function, which inevitably introduced the restriction $n + k_1 \geq k_0$ and the quantum collision-resistant term. Further, in order to make $\Gamma_{R_H^c}$ negligible, the restriction $k_0 - n = \mathcal{O}(n)$ is also introduced. In our game hop $\mathbf{G}_0$ to $\mathbf{G}_5$ explained above, we simulate G and H both as compressed oracles, and then similar to the proof of [8, 16], we compute the differences between qDec and qDec_1 , qDec_1 and qDec_2 , qDec_2 and qDec_{sk} directly by the basic rules of compressed oracle. Essentially, the computation of these differences has already covered the estimation of the disturbances mentioned above, therefore the online extraction technique is no longer needed, thus avoiding the parameter restrictions and the quantum collision-resistant term needed in [13].

Techniques in the game hop from $\mathbf{G}_5$ to $\mathbf{G}_{14}$: In this game hop, we use three middle games $\mathbf{G}_5^a$ to $\mathbf{G}_5^c$ as shown in Fig. 4 to reprogram the oracles G and H queried before $\mathcal{A}$ outputs (m_0^*, m_1^*) .

- Compared with game $\mathbf{G}_5$ shown in Fig. 2, game $\mathbf{G}_5^a$ introduces the semi-classical oracle $\mathcal{O}_{\{r^*\}}^{SC}$ to perform a binary projective measurement on $\mathcal{A}$'s quantum query to G , and adds a flag that if Find_{r^*} occurs, game finally returns $\perp$. Here Find_{r^*} is the event that $\mathcal{O}_{\{r^*\}}^{SC}$ ever has measurement result 1. Now by directly using the Semi-Classical One-way to Hiding (SC-O2H) theorem [1], we can bound the difference between game $\mathbf{G}_5$ and $\mathbf{G}_5^a$.

⁷ That is, in our paper we assume that $c = f_{\text{pk}}(s, t)$ must imply $f_{\text{sk}}^{-1}(c) = (s, t)$.

- The flag added in game $\mathbf{G}_5^a$ actually indicates that if game $\mathbf{G}_5^a$ returns 1, then the event Find_{r^*} must do not occur. Hence, since we only focus on the probability $\Pr[1 \leftarrow \mathbf{G}_5^a]$, we can default to the assumption that Find_{r^*} does not occur. Under this assumption, and by the definition of $\mathcal{O}_{\{r^*\}}^{SC}$ (Definition 1), we can conclude that $\mathcal{A}$ only queries $\mathbf{G}$ with a query state in the form $\sum_{x \neq r^*, y} \alpha_{x,y} |x, y\rangle$ before $\mathcal{A}$ outputs (m_0^*, m_1^*) . This means that the $\mathbf{G}(r^*)$ used to compute s^* in the next step is actually the first r^* query to $\mathbf{G}$. Recall that $\mathbf{G}$ is simulated as a compressed oracle, so by its basic rules, we can equivalently change the compressed oracle simulation of $\mathbf{G}(r^*)$ in game $\mathbf{G}_5^a$ to the particular simulation of $\mathbf{G}(r^*)$ in game $\mathbf{G}_5^b$. This simulation first chooses g^* uniformly and views it as $\mathbf{G}(r^*)$, then inserts (r^*, g^*) into the database state by sequentially applying the unitary operation $U_{g^*}^{r^*}$ (Eq. (15)) and the local decompression procedure StdDecomp_{r^*} (Section 2.4).

Moreover, in order to maintain consistency, during the generation of b' , game $\mathbf{G}_5^b$ replaces oracle $\mathbf{G}$ as $\mathbf{G}[r^* \mapsto g^*]$, which is simulated the same as $\mathbf{G}$ except that it directly replies g^* to the query r^* . In fact, by basic rules of compressed oracle, such replacement cannot be detected by the adversary if StdDecomp_{r^*} commutes with qDec_{sk} . To address this, we prove our Lemma 4, which shows that StdDecomp_{r^*} almost commutes with qDec_{sk} , and hence we can transform game $\mathbf{G}_5^a$ into game $\mathbf{G}_5^b$ under a negligible error.

GAME $\mathbf{G}_5^a$	GAME $\mathbf{G}_5^b$	GAME $\mathbf{G}_5^c$
1. $(pk, sk) \leftarrow \text{Gen}$ r^* uniformly sample	1. $(pk, sk) \leftarrow \text{Gen}$ r^*, g^* uniformly sample	1. $(pk, sk) \leftarrow \text{Gen}$ r^*, s^*, h^* uniformly sample
2. $(m_0^*, m_1^*) \leftarrow \mathcal{A}^{\mathbf{G} \circ \mathcal{O}_{\{r^*\}}^{SC}, H, \text{qDec}_{sk}}$ if Find_{r^*} occurs, return $\perp$	2. $(m_0^*, m_1^*) \leftarrow \mathcal{A}^{\mathbf{G} \circ \mathcal{O}_{\{r^*\}}^{SC}, H, \text{qDec}_{sk}}$ if Find_{r^*} occurs, return $\perp$	2. $(m_0^*, m_1^*) \leftarrow \mathcal{A}^{\mathbf{G} \circ \mathcal{O}_{\{r^*\}}^{SC}, H \circ \mathcal{O}_{\{s^*\}}^{SC}, \text{qDec}_{sk}}$ if $\text{Find}_{r^*} \vee \text{Find}_{s^*}$ occurs, return $\perp$
3. $b \leftarrow_{\mathcal{S}} \{0, 1\}$ $s^* = (m_b^* 0^{k_1}) \oplus \mathbf{G}(r^*)$ $t^* = r^* \oplus H(s^*)$ $c^* = f_{pk}(s^*, t^*)$	3. $b \leftarrow_{\mathcal{S}} \{0, 1\}$ $s^* = (m_b^* 0^{k_1}) \oplus g^*$ $t^* = r^* \oplus H(s^*)$ $c^* = f_{pk}(s^*, t^*)$, perform $U_{g^*}^{r^*}$	3. $b \leftarrow_{\mathcal{S}} \{0, 1\}$ $g^* = (m_b^* 0^{k_1}) \oplus s^*$ $t^* = r^* \oplus h^*$ $c^* = f_{pk}(s^*, t^*)$, perform $U_{h^*}^{s^*} \circ U_{g^*}^{r^*}$
4. $b' \leftarrow \mathcal{A}^{\mathbf{G}, H, \text{qDec}_{sk}}(c^*)$	perform StdDecomp_{r^*} 4. $b' \leftarrow \mathcal{A}^{\mathbf{G}[r^* \mapsto g^*], H, \text{qDec}_{sk}}(c^*)$	perform $\text{StdDecomp}_{s^*} \circ \text{StdDecomp}_{r^*}$ 4. $b' \leftarrow \mathcal{A}^{\mathbf{G}[r^* \mapsto g^*], H[s^* \mapsto h^*], \text{qDec}_{sk}}(c^*)$
5. return $[b' = b]$	5. return $[b' = b]$	5. return $[b' = b]$

Fig. 4. Games $\mathbf{G}_5^a$ to $\mathbf{G}_5^c$. We stress that, these three games were not used in our proof, and they are introduced here for the convenience of explaining our techniques.

- The s^* in game $\mathbf{G}_5^b$ is computed as $(m_b^* || 0^{k_1}) \oplus g^*$, where g^* is uniformly sampled. In game $\mathbf{G}_5^c$, based on the basic property of XOR, we make a conceptual change, i.e., we first uniformly sample s^* at the beginning of the game and then compute $g^* = (m_b^* || 0^{k_1}) \oplus s^*$. Therefore the generation of s^* in game $\mathbf{G}_5^c$

is independent of the adversary. Based on this fact, we make another three changes in game $\mathbf{G}_5^c$ as follows:

- We introduce the semi-classical oracle $\mathcal{O}_{\{s^*\}}^{SC}$ to perform a binary projective measurement on $\mathcal{A}$'s quantum query to $\mathbf{H}$, and add a flag that if Find_{s^*} occurs, game directly returns $\perp$.
- The compressed oracle simulation of $\mathbf{H}(s^*)$ is changed to a particular simulation, which first chooses h^* uniformly and views it as $\mathbf{H}(s^*)$, then inserts (s^*, h^*) into the database state by sequentially applying the unitary operation $\mathbf{U}_{h^*}^{s^*}$ (Eq. (19)) and the local decompression procedure StdDecomp_{s^*} .
- The oracle $\mathbf{H}$ queried during the generation of b' is replaced by $\mathbf{H}[s^* \mapsto h^*]$, which is the same as $\mathbf{H}$ except that it directly replies h^* to the query s^* .

Obviously the above changes are the same as those made in game $\mathbf{G}_5^a$ and $\mathbf{G}_5^b$, except that the perspective is shifted from oracle $\mathbf{G}$ to oracle $\mathbf{H}$. Hence similar to the explanation of game hop $\mathbf{G}_5$ to $\mathbf{G}_5^a$ and game hop $\mathbf{G}_5^a$ to $\mathbf{G}_5^b$, the difference between game hop $\mathbf{G}_5^c$ to $\mathbf{G}_5^b$ is also negligible.

- Based on game $\mathbf{G}_5^c$, we make three changes to get game $\mathbf{G}_{14}$ shown in Fig. 2. First, we remove $\mathcal{O}_{\{r^*\}}^{SC}$, $\mathcal{O}_{\{s^*\}}^{SC}$, and the flags related to Find_{r^*} and Find_{s^*} . The difference caused by this change can be bounded by using the SC-O2H theorem in reverse. Second, instead of uniformly sampling s^* and then computing $g^* = (m_b^* || 0^{k_1}) \oplus s^*$, game $\mathbf{G}_{14}$ instead first uniformly samples g^* and then computes $s^* = (m_b^* || 0^{k_1}) \oplus g^*$. Obviously, this is just a conceptual change. Third, we remove the performing of StdDecomp_{r^*} and StdDecomp_{s^*} . In fact, StdDecomp_{r^*} and StdDecomp_{s^*} must commute with $\mathbf{G}[r^* \mapsto g^*]$ and $\mathbf{H}[s^* \mapsto h^*]$, and the reason is that, roughly speaking, the query to $\mathbf{G}[r^* \mapsto g^*]$ and $\mathbf{H}[s^* \mapsto h^*]$ at point r^* and s^* , respectively, is no longer simulated by the compressed oracle. Furthermore, as proved in our Lemma 4, StdDecomp_{r^*} and StdDecomp_{s^*} almost commute with qDec_{sk} . According to these two facts, under a negligible error, we move the performing of StdDecomp_{r^*} and StdDecomp_{s^*} to the end of the game and then delete them.

At this point, we complete the game hop from $\mathbf{G}_5$ to $\mathbf{G}_{14}$, during which we successfully reprogram the oracles $\mathbf{G}$ and $\mathbf{H}$ queried before $\mathcal{A}$ outputs (m_0^*, m_1^*) .

Techniques in the game hop $\mathbf{G}_{14}$ to $\mathbf{G}_{17}$: In this game hop, we use two middle games $\mathbf{G}_{14}^a$ and $\mathbf{G}_{14}^b$ as shown in Fig. 5 to reprogram the oracles $\mathbf{G}[r^* \mapsto g^*]$ and $\mathbf{H}[s^* \mapsto h^*]$ queried during the generation of b' .

- Compared with game $\mathbf{G}_{14}$ shown in Fig. 2 which first uniformly samples g^*/h^* and then computes $s^* = (m_b^* || 0^{k_1}) \oplus g^*/t^* = r^* \oplus h^*$, game $\mathbf{G}_{14}^a$ first uniformly samples s^*/t^* and then computes $g^* = (m_b^* || 0^{k_1}) \oplus s^*/h^* = r^* \oplus t^*$. Obviously this is just a conceptual change.
- Compared with game $\mathbf{G}_{14}^a$, game $\mathbf{G}_{14}^b$ removes the performing of $\mathbf{U}_{h^*}^{s^*} \circ \mathbf{U}_{g^*}^{r^*}$. In fact, by the definition of $\mathbf{U}_{g^*}^{r^*}$ and $\mathbf{U}_{h^*}^{s^*}$ given in Eq. (15) and Eq. (19), they both quantumly compute a permutation on the corresponding database state. Now a key observation is that, roughly speaking, the (composite) permutation

GAME $\mathbf{G}_{14}^a$	GAME $\mathbf{G}_{14}^b$	GAME $\mathbf{G}_{17}$
1. $(pk, sk) \leftarrow \text{Gen}$	1. $(pk, sk) \leftarrow \text{Gen}$	1. $(pk, sk) \leftarrow \text{Gen}$
r^*, s^*, t^* uniformly sample	r^*, s^*, t^* uniformly sample	r^*, s^*, t^* uniformly sample
2. $(m_0^*, m_1^*) \leftarrow \mathcal{A}^{G, H, q\text{Dec}_{sk}}$	2. $(m_0^*, m_1^*) \leftarrow \mathcal{A}^{G, H, q\text{Dec}_{sk}}$	2. $(m_0^*, m_1^*) \leftarrow \mathcal{A}^{G, H, q\text{Dec}_{sk}}$
3. $b \leftarrow_{\$} \{0, 1\}$	3. $b \leftarrow_{\$} \{0, 1\}$	3. $b \leftarrow_{\$} \{0, 1\}$
$g^* = (m_b^* 0^{k_1}) \oplus s^*$	$g^* = (m_b^* 0^{k_1}) \oplus s^*$	$g^* = (m_b^* 0^{k_1}) \oplus s^*$
$h^* = r^* \oplus t^*$	$h^* = r^* \oplus t^*$	$h^* = r^* \oplus t^*$
$c^* = f_{pk}(s^*, t^*)$, perform $U_{h^*}^{s^*} \circ U_{g^*}^{r^*}$	$c^* = f_{pk}(s^*, t^*)$, perform $U_{h^*}^{s^*} \circ U_{g^*}^{r^*}$	$c^* = f_{pk}(s^*, t^*)$
4. $b' \leftarrow \mathcal{A}^{G[r^* \mapsto g^*], H[s^* \mapsto h^*], q\text{Dec}_{sk}}(c^*)$	4. $b' \leftarrow \mathcal{A}^{G[r^* \mapsto g^*], H[s^* \mapsto h^*], q\text{Dec}_{sk}}(c^*)$	4. $b' \leftarrow \mathcal{A}^{G, H, q\text{Dec}_{sk}}(c^*)$
5. return $\llbracket b' = b \rrbracket$	5. return $\llbracket b' = b \rrbracket$	5. return $\llbracket b' = b \rrbracket$

Fig. 5. Games $\mathbf{G}_{14}^a$ and $\mathbf{G}_{14}^b$. We stress that, these two games were not used in our proof, and they are introduced here for the convenience of explaining our techniques.

computed by $U_{h^*}^{s^*} \circ U_{g^*}^{r^*}$ will disturb the database state and hence affect the search result of $q\text{Dec}_{sk}$ performed during the generation of b' . This means that, if we remove the performing of $U_{h^*}^{s^*} \circ U_{g^*}^{r^*}$, the simulation of $q\text{Dec}_{sk}$ during the generation of b' will return affected search results. Based on this observation we design a quantum oracle algorithm $\mathcal{A}_1$ and abstract two fixed and different permutations P_0 and P_1 such that

$$\Pr[1 \leftarrow \mathbf{G}_{14}^a] = \Pr[1 \leftarrow \mathcal{A}_1^{P_0}], \Pr[1 \leftarrow \mathbf{G}_{14}^b] = \Pr[1 \leftarrow \mathcal{A}_1^{P_1}].$$

Here oracle P_0 (resp. P_1) maps the query state $|c, y, D_G, D_H\rangle$ of $q\text{Dec}_{sk}$ into $|P_0(c, y, D_G, D_H)\rangle$ (resp. $|P_1(c, y, D_G, D_H)\rangle$), and permutations P_0 and P_1 satisfy that $P_0(c, y, D_G, D_H) \neq P_1(c, y, D_G, D_H)$ if and only if ⁸ $(c, y, D_G, D_H) \in R_1 \cup R_2$, where

$$\begin{aligned} R_1 &:= \{(c, y, D_G, D_H) : \exists(s, D_H(s) \neq \perp) \text{ s.t. } c = f_{pk}(s, r^* \oplus D_H(s))\}, \\ R_2 &:= \{(c, y, D_G, D_H) : \exists(r, D_G(r) \neq \perp) \text{ s.t. } c = f_{pk}(s^*, r \oplus r^* \oplus t^*)\}. \end{aligned}$$

Thanks to the elegant structure of the Fixed Permutation One-Way to Hiding theorem [22], we are now able to bound the difference between game $\mathbf{G}_{14}^a$ and $\mathbf{G}_{14}^b$ by the success probability of another adversary aiming to guess one tuple $(c, y, D_G, D_H) \in R_1 \cup R_2$. In fact, the success probability of such guessing adversary must be negligible. The reason is that, roughly speaking, if $(c, y, D_G, D_H) \in R_1$ we can easily extract r^* from D_H and the inverse of c , even if r^* is uniformly sampled and independent of the guessing adversary. Else if $(c, y, D_G, D_H) \in R_2$, we can also extract r^* from D_G and the inverse of c with a noticeable probability since there are at most q_G different r satisfying $D_G(r) \neq \perp$. So with a negligible error we can transform game $\mathbf{G}_{14}^a$ into $\mathbf{G}_{14}^b$.

⁸ In fact the "if and only if" here is not a rigorously accurate expression, since several preprocessing is needed to derive this equivalence relation. Here we just use this expression for convenience, and more details can be seen in the proof of Lemma 6.

- Based on game $\mathbf{G}_{14}^b$, we get game $\mathbf{G}_{17}$ shown in Fig. 2 by modifying the oracles $G[r^* \mapsto g^*]$ and $H[s^* \mapsto h^*]$ queried during the generation of b' back to G and H , and removing the computation of g^* and h^* . First, we note that it is hard for the adversary $\mathcal{A}$ to detect the change of the oracles. The intuition is that, $\mathcal{A}$ can only detect such change by querying G on r^* or querying H on s^* . However, in game $\mathbf{G}_{17}$, r^* is independent in the view of $\mathcal{A}$, and $\mathcal{A}$ can only "touch" s^* through $c^*(=f_{pk}(s^*, t^*))$. As a result, the probability that $\mathcal{A}$ queries G on r^* must be negligible, meanwhile it is also hard for $\mathcal{A}$ to query H on s^* , otherwise we can use $\mathcal{A}$ to attack the PD-OW security of the underlying TDI. Naturally, after the modification of the oracles, g^* and h^* are no longer used by the game, hence we can directly remove their computations.

Now we complete the game hop from $\mathbf{G}_{14}$ to $\mathbf{G}_{17}$, meaning that we successfully reprogram $G[r^* \mapsto g^*]$ and $H[s^* \mapsto h^*]$ queried during the generation of b' .

1.3 Related Work

As a widely accepted generic transform in the ROM, OAEP has been extensively explored from various perspectives in the classical setting. Examples are [5] for designing more efficient variants of OAEP, [36,33] for weakening the underlying security assumptions used by the proof, and [7,2] for proving particular security goals going beyond IND-CCA. However, to the best of our knowledge, there are fewer works exploring OAEP in the quantum setting: in the QROM, except for the previously mentioned [37,13] which both focused on the fundamental IND-CCA/IND-qCCA security of OAEP, only one work [26] studied a particular "qIND-qCPA" security of OAEP. Therefore, there is still significant research potential for OAEP in the quantum setting.

In the QROM, different from [24,20,16], which proved the IND-CCA/IND-qCCA security of FO transform (or its variants) with the consideration of decryption error, all IND-CCA/IND-qCCA security proofs of OAEP in [37,13] and ours in this paper have not considered the decryption error. Roughly speaking, there are two main technical obstacles: first, the underlying scheme of OAEP is a trapdoor injective function, but as far as we know, there is currently no formal definition for the decryption error of trapdoor injective function; second, if the underlying trapdoor injective function is not perfectly correct, i.e., $c = f_{pk}(s, t)$ may not imply $f_{sk}^{-1}(c) = (s, t)$ for some (pk, sk, s, t) tuples, the current simulation of the decryption oracle is no longer valid and it seems tricky to fix this up. Hence in this paper, we do not consider the decryption error and we leave it as an open question.

In addition, the Fixed Permutation One-Way to Hiding theorem applied in our proof is actually a very powerful and concise tool to handle the adaptive quantum reprogramming. In fact, [22] proved that this theorem not only implies the Pan-Zeng [32] and GHHM [17] adaptive reprogramming framework, but is also able to provide a comparable or essentially the same upper bound.

2 Preliminaries

2.1 Notation

Let n be a positive integer and then denote $[n]$ as the set $\{1, 2, \dots, n\}$. Let s be a bitstring, denote $[s]_k$ as the k -least significant bits of s while $[s]^k$ denotes the k -most significant bits of s . $\llbracket P \rrbracket$ denotes a bit which is 1 if the predicate P is true and otherwise 0. $x \leftarrow_{\S} S$ denotes uniformly sampling an element x from a finite set S . $x \leftarrow D$ means that x is chosen according to distribution D . $\mathcal{T}_{\mathcal{A}}$ denotes the running time of algorithm or (computing) function $\mathcal{A}$. $\Pr[A : B]$ is the probability that predicate A keeps true where all variables in A are conditioned according to predicates B . During the security proof, we use $\Pr[1 \leftarrow \mathbf{G}]$ to denote the probability that the game $\mathbf{G}$ finally returns 1.

2.2 Quantum Background

In Supplementary Material B we present some fundamental quantum notions and principles utilized in this paper. For more detailed basics of quantum computation, we refer to a textbook [28].

2.3 Quantum Random Oracle Model

In the random oracle model (ROM), we assume that all parties have classical access to a random function. In practice, the random function is usually implemented by a suitable hash function. In the quantum setting, since the hash function can be locally computed in superposition, the ROM should be lifted to the quantum random oracle model (QROM) [6] where all parties have quantum access to the random oracle. In the QROM, the quantum query to a random oracle H is simulated by the unitary operation $O_H : |x, y\rangle \mapsto |x, y \oplus H(x)\rangle$.

In the following, we present some definitions and techniques employed in the QROM focusing on security proof methodologies.

Definition 1 (Semi-Classical Oracle [1]). *For a subset $S \subseteq \mathcal{X}$, the indicator function f_S from $\mathcal{X}$ to $\{0, 1\}$ equals 1 iff its input $x \in S$. The semi-classical oracle, $\mathcal{O}_S^{SC}$, is a quantum accessible oracle that performs the following two operations on its query state $\sum_{x \in \mathcal{X}} \alpha_x |x\rangle$:*

1. *Initialize a single qubit register L with the state $|0\rangle_L$.*
2. *Transform $\sum_{x \in \mathcal{X}} \alpha_x |x\rangle |0\rangle_L$ into $\sum_{x \in \mathcal{X}} \alpha_x |x\rangle |f_S(x)\rangle_L$ and then measure L .*

During the execution of a quantum oracle algorithm $\mathcal{A}^{\mathcal{O}_S^{SC}}$, let Find be the classical event that the measurement of $\mathcal{O}_S^{SC}$ (on L) ever has outcome 1.

Theorem 1 (Semi-Classical One-Way to Hiding (O2H) [1, Theorem 1]). *Let $S \subseteq \mathcal{X}$ be random. Let $G, H : \mathcal{X} \rightarrow \mathcal{Y}$ be random functions satisfying $\forall x \notin S, G(x) = H(x)$. Let z be a random bitstring. (G, H, S, z may have arbitrary joint distribution D .) Let $H \setminus S$ be an oracle that first queries $\mathcal{O}_S^{SC}$ on the input register*

of H and then queries H . Let $\mathcal{A}$ be a quantum oracle algorithm with query times q ⁹ (not necessarily unitary). Let

$$\begin{aligned} P_{\text{left}} &:= \Pr[b = 1 : b \leftarrow \mathcal{A}^H(z), (G, H, S, z) \leftarrow D], \\ P_{\text{right}} &:= \Pr[b = 1 \wedge \neg \text{Find} : b \leftarrow \mathcal{A}^{H \setminus S}(z), (G, H, S, z) \leftarrow D]. \end{aligned}$$

Let $P_{\text{find}} := \Pr[\text{Find} : \mathcal{A}^{H \setminus S}(z), (G, H, S, z) \leftarrow D]$. Then

$$|P_{\text{left}} - P_{\text{right}}| \leq \sqrt{(q+1) \cdot P_{\text{find}}} \text{ and } \left| \sqrt{P_{\text{left}}} - \sqrt{P_{\text{right}}} \right| \leq \sqrt{(q+1) \cdot P_{\text{find}}}.$$

Remark 3. There are several other definitions of P_{right} in [1, Theorem 1]. In this paper, we only need the above definition in our security proofs.

Theorem 2 (Search in Semi-Classical Oracle [1, Theorem 2]). *Let $S \subseteq \mathcal{X}$ be random and z be a random bitstring. (S, z) may have an arbitrary joint distribution. Let $\mathcal{A}$ be any quantum oracle algorithm making at most q queries to a semi-classical oracle with domain $\mathcal{X}$.*

Let $\mathcal{B}$ be an algorithm that on input z , chooses $i \leftarrow_{\$} [q]$; runs $\mathcal{A}^{\mathcal{O}_S^{SC}}(z)$ until (just before) the i -th query, measures the input register and outputs the measurement outcome x . Then

$$\Pr[\text{Find} : \mathcal{A}^{\mathcal{O}_S^{SC}}(z), (S, z) \leftarrow D] \leq 4q \cdot \Pr[x \in S : x \leftarrow \mathcal{B}(z), (S, z) \leftarrow D].$$

In the above theorem, for the simple case that z is independent of S , we have the following corollary.

Corollary 1 ([1, Corollary 1]). *Suppose that S and z are independent, and that $\mathcal{A}$ is a q -query algorithm. Let $P_{\text{max}} := \max_{x \in \mathcal{X}} \Pr[x \in S]$, then $\Pr[\text{Find} : \mathcal{A}^{\mathcal{O}_S^{SC}}(z), (S, z) \leftarrow D] \leq 4q \cdot P_{\text{max}}$.*

2.4 Compressed Oracle Technique

We briefly introduce the compressed oracle technique presented by Zhandry in [41]. Roughly speaking, the core idea of this technique is to purify the query state of the quantum random oracle and then encode the oracle's internal state into a (efficiently controlled) database state. Here we first introduce the database model, and then introduce one particular version of the compressed oracle called the compressed standard oracle. For simplicity, we further assume that the compressed standard oracle will be queried at most q times.

Definition 2 (Database). *Define $\mathcal{X} := \{0, 1\}^m$ and $\mathcal{Y} := \{0, 1\}^n$. Introduce a symbol $\perp$ that $\perp \notin \mathcal{X} \cup \mathcal{Y}$. Then a database D is an ordered list of q -pairs as:*

$$((x_1, y_1), (x_2, y_2), \dots, (x_l, y_l), (\perp, 0^n), \dots, (\perp, 0^n)),$$

⁹ [1] actually considered the parallel queries and gave a better bound. In our paper, we do not consider such parallel queries for simplicity.

where $(x_i, y_i) \in \mathcal{X} \times \mathcal{Y}$ for $i \in [l]$, $x_1 < \dots < x_l$, and all $(\perp, 0^n)$ pairs are at the end of the list¹⁰. The database set $\mathbf{D}_q$ is a set containing the above databases.

For an $x \in \mathcal{X}$, we write $D(x) = y$ if there exists a $y \in \mathcal{Y}$ s.t. $(x, y) \in D$, otherwise write $D(x) = \perp$. Let $n(D)$ denote the number of (x, y) pairs in the database D such that $D(x) \neq \perp$. For any pair $(x, y) \in \mathcal{X} \times \mathcal{Y}$ and any database D which satisfies $D(x) = \perp$ and $n(D) < q$, let $D \cup (x, y)$ be the updated database which first deletes a pair $(\perp, 0^n)$ and then inserts (x, y) appropriately into D to maintain its order. Conversely, for any pair $(x, y) \in \mathcal{X} \times \mathcal{Y}$ and any database D which satisfies $D(x) = y$, let $D \setminus (x, y)$ denote the new database which first removes the pair (x, y) from D and then adds a pair $(\perp, 0^n)$ at the end.

Denote $\mathbf{D}_q$ as the quantum register defined over the database set $\mathbf{D}_q$. That is, $\mathbf{D}_q$ is the Hilbert space with orthonormal basis $\{|D\rangle\}_{D \in \mathbf{D}_q}$, where the basis state is labeled by elements in $\mathbf{D}_q$. Indeed, this basis is also called the computational basis by the basic quantum computation. In the following, we refer to $\mathbf{D}_q$ as the database register.

Define a superposition of the database state as

$$|D \cup (x, \beta_r)\rangle := \frac{1}{\sqrt{2^n}} \sum_{y \in \{0,1\}^n} (-1)^{y \cdot r} |D \cup (x, y)\rangle, \quad (r \in \{0,1\}^n).$$

Then for any $x \in \mathcal{X}$, define the local decompression procedure StdDecomp_x applied on $\mathbf{D}_q$ as follows:

- For $D \in \mathbf{D}_q$ that $D(x) = \perp$ and $n(D) < q$, $\text{StdDecomp}_x|D\rangle = |D \cup (x, \beta_{0^n})\rangle$;
- For $D \in \mathbf{D}_q$ that $D(x) = \perp$ and $n(D) < q$, $\text{StdDecomp}_x|D \cup (x, \beta_{0^n})\rangle = |D\rangle$ and $\text{StdDecomp}_x|D \cup (x, \beta_r)\rangle = |D \cup (x, \beta_r)\rangle$ for $r \neq 0^n$;
- For $D \in \mathbf{D}_q$ that $D(x) = \perp$ and $n(D) = q$, $\text{StdDecomp}_x|D\rangle = |D\rangle$.

It is obvious that StdDecomp_x is unitary and is an involution, since applying it twice results in the identity operation. Meanwhile, by the above definition, it is easy to check that

$$\text{StdDecomp}_x|D \cup (x, y)\rangle = |D \cup (x, y)\rangle + \frac{1}{\sqrt{2^n}}|D\rangle - \frac{1}{\sqrt{2^n}}|D \cup (x, \beta_{0^n})\rangle, \quad (1)$$

for a $D \in \mathbf{D}_q$ that $D(x) = \perp$ and $n(D) < q$, which is a very useful property.

Definition 3 (Compressed Standard Oracle). Let $\mathbf{X}$ (resp. $\mathbf{Y}$) denote the input (resp. output) register defined over $\mathcal{X}$ (resp. $\mathcal{Y}$). Let $|D^\perp\rangle$ be the initial state on database register $\mathbf{D}_q$, where $D^\perp$ is the database containing q $(\perp, 0^n)$ pairs. A quantum query to the compressed standard oracle is implemented by performing the following unitary operation on registers $\mathbf{XYD}_q$.

$$\text{CStO} := \sum_{x \in \{0,1\}^m} |x\rangle\langle x|_{\mathbf{X}} \otimes \text{StdDecomp}_x \circ \text{CNOT}^x \circ \text{StdDecomp}_x.$$

¹⁰ Here we only give the basic definition of the database, the quantum implementation details of the database can be found in [9,10].

Here $\text{CNOT}^x|y, D\rangle_{\text{YD}_q} = |y \oplus D(x), D\rangle_{\text{YD}_q}$ if $D(x) \neq \perp$ and $|y, D\rangle_{\text{YD}_q}$ otherwise¹¹.

Theorem 3 ([41, Lemma 4]). *For any adversary $\mathcal{A}$ making at most q oracle queries, the compressed standard oracle (Definition 3) and the quantum random oracle $H : \mathcal{X} \rightarrow \mathcal{Y}$ are perfectly indistinguishable.*

Remark 4. As analyzed in [12, Theorem 4.3], using the compressed standard oracle to perfectly simulate the quantum random oracle with query upper bound q leads to an $\mathcal{O}(q^2)$ increase in running time.

3 The Security of OAEP Transform

In this section, we first review the structure of OAEP transform and then give its security theorem (Theorem 4). This theorem shows that, in the QROM, the IND-qCCA security of OAEP can be reduced to the PD-OW security of the underlying trapdoor injective function. We refer to Supplementary Material C for details of cryptographic primitives and security definitions.

OAEP Transform. Let $\text{TDI} = (\text{Gen}, f, f^{-1})$ be a trapdoor injective function with input space $\{0, 1\}^{n+k_1} \times \{0, 1\}^{k_0}$ and output space $\{0, 1\}^m$. Define a parameter $\mathcal{T}_f := \max_{(\text{pk}, \text{sk})} \mathcal{T}_{f_{\text{pk}}}$, where the maximum is according to all the (pk, sk) sampled by Gen , and $\mathcal{T}_{f_{\text{pk}}}$ denotes the running time of computing f_{pk} . Let $G : \{0, 1\}^{k_0} \rightarrow \{0, 1\}^{n+k_1}$ and $H : \{0, 1\}^{n+k_1} \rightarrow \{0, 1\}^{k_0}$ be hash functions. We associate the randomized PKE scheme

$$\text{OAEP}[\text{TDI}, G, H] = (\text{Gen}, \text{Enc}, \text{Dec}).$$

This scheme has message space $\mathcal{M} = \{0, 1\}^n$, randomness space $\mathcal{R} = \{0, 1\}^{k_0}$ and ciphertext space $\mathcal{C} = \{0, 1\}^m$. The constituting algorithms of this scheme are shown in Fig. 6.

Gen	Enc(pk, $m \in \{0, 1\}^n$)	Dec(sk, c)
1 : $(\text{pk}, \text{sk}) \leftarrow \text{Gen}$	1 : $r \leftarrow_{\$} \{0, 1\}^{k_0}$	1 : if $f_{\text{sk}}^{-1}(c) = \perp$, return $\perp$
2 : return (pk, sk)	2 : $s = (m 0^{k_1}) \oplus G(r)$	2 : else parse $f_{\text{sk}}^{-1}(c)$ into $s t$
	3 : $t = r \oplus H(s)$	3 : $r = t \oplus H(s), M = s \oplus G(r)$
	4 : $c = f_{\text{pk}}(s, t)$	4 : if $[M]_{k_1} = 0^{k_1}$, return $[M]^n$
	5 : return c	5 : else return $\perp$

Fig. 6. Public key encryption scheme $\text{OAEP}[\text{TDI}, G, H]$.

¹¹ After performing CNOT^x , the state on YD_q remains unchanged if $D(x) = \perp$. In fact, this is equivalent to the definition that " $y \oplus \perp = y$ " defined in [41].

Theorem 4 (PD-OW of $\text{TDI} \xrightarrow{\text{QROM}} \text{IND-qCCA}$ of $\text{OAEP}[\text{TDI}, \text{G}, \text{H}]$). *Let $\mathcal{A}$ be an IND-qCCA adversary against $\text{OAEP}[\text{TDI}, \text{G}, \text{H}]$ in the QROM, making at most q_D , q_G and q_H quantum queries to the decryption oracle qDec , random oracle G and random oracle H . Let $q := q_G + q_H + 1$. Then we can construct a PD-OW adversary $\mathcal{B}$ against the trapdoor injective function TDI such that*

$$\begin{aligned} \text{Adv}_{\text{OAEP}[\text{TDI}, \text{G}, \text{H}], \mathcal{A}}^{\text{IND-qCCA}} &\leq (32q_D + 4)q \cdot \sqrt{\text{Adv}_{\text{TDI}, \mathcal{B}}^{\text{PD-OW}}} + (42q_D + 8)q \cdot \sqrt{2^{-k_0}} \\ &\quad + 24q_D \cdot \sqrt{2^{-k_1}}, \end{aligned}$$

and $\mathcal{T}_B \leq \mathcal{T}_A + \mathcal{O}(q_G^2 + q_H^2) + \mathcal{O}(q_D q_G q_H) \cdot \mathcal{T}_f$.

3.1 Quantum Accessible Decryption Oracle qDec

Before proving Theorem 4, we first show how to simulate the quantum accessible decryption oracle qDec .

Denote I/O as the input/output registers of qDec , where I is defined over $\{0, 1\}^m$ and O over $\{0, 1\}^n \cup \{\perp\}$ ¹². Let S and T be two ancillary registers, which are defined over $\{0, 1\}^{n+k_1} \cup \{\perp\}$ and $\{0, 1\}^{k_0} \cup \{\perp\}$, respectively. Now we define the unitary operation U_{qDec} acting on registers I , O , S and T :

$$\text{U}_{\text{qDec}} := \text{U}_{f^{-1}} \circ \text{O}_H \circ \text{O}_G \circ \text{U}_{\text{check}} \circ \text{O}_G \circ \text{O}_H \circ \text{U}_{f^{-1}}. \quad (2)$$

Here

$$\text{U}_{f^{-1}} : |c, s, t\rangle_{\text{IST}} \mapsto \begin{cases} |c, s \oplus s', t \oplus t'\rangle_{\text{IST}}, & \text{if } f_{\text{sk}}^{-1}(c) \neq \perp \text{ and } f_{\text{sk}}^{-1}(c) = s' || t', \\ |c, s \oplus \perp, t \oplus \perp\rangle_{\text{IST}}, & \text{otherwise.} \end{cases}$$

focuses on lifting the step 1 and 2 of $\text{Dec}(\text{sk}, c)$ (Fig. 6) to the quantum setting. $\text{O}_H : |x, y\rangle \mapsto |x, y \oplus \text{H}(x)\rangle$ (resp. $\text{O}_G : |x, y\rangle \mapsto |x, y \oplus \text{G}(x)\rangle$) with input/output registers S/T (resp. T/S) focuses on lifting the step 3 of $\text{Dec}(\text{sk}, c)$. As for the U_{check} , it is defined as:

$$\text{U}_{\text{check}} : |M, y\rangle_{\text{SO}} \mapsto \begin{cases} |M, y \oplus [M]^n\rangle_{\text{SO}}, & \text{if } M \in \{0, 1\}^{n+k_1} \wedge [M]_{k_1} = 0^{k_1}, \\ |M, y \oplus \perp\rangle_{\text{SO}}, & \text{otherwise.} \end{cases}$$

and it is used to lift the step 4 and step 5 of $\text{Dec}(\text{sk}, c)$ to the quantum setting. Note that the implementation of $\text{U}_{f^{-1}}$ may introduce $s \oplus \perp$ on register S , which indicates that the computation has failed. So in the above definition of U_{check} , we impose the restriction $M \in \{0, 1\}^{n+k_1}$ to exclude such failure cases. This exclusion is valid because, as shown in Supplementary Material B, our quantum representation of $\perp$ ensures that $s \oplus \perp \notin \{0, 1\}^{n+k_1}$ for any $s \in \{0, 1\}^{n+k_1}$. Additionally, since $\text{U}_{f^{-1}}$ can also introduce $t \oplus \perp$ on register T , WLOG we assume $\text{G}(\perp) = \text{H}(\perp) = 0$ in the definition of O_G and O_H above.

¹² Here and in the following, we always use symbol $\perp$ to denote "computational failure". See Supplementary Material B for a detailed quantum representation of this symbol.

At this point, one can easily check that

$$U_{\text{qDec}}|c, y, 0, 0\rangle_{\text{IOST}} = |c, y \oplus \text{Dec}(\text{sk}, c), 0, 0\rangle_{\text{IOST}}.$$

So if we initialize registers **S** and **T** to the state $|0\rangle$ at the beginning and ensure that they are not accessed by any other operations, then their states will remain $|0\rangle$ before and after each implementation of U_{qDec} . Hence, for (pk, sk) sampled from **Gen**, let **S** and **T** be the internal registers of the quantum-accessible decryption oracle **qDec**. By initializing both registers to $|0\rangle$, we can directly simulate **qDec** using U_{qDec} before the challenge ciphertext c^* is defined. After c^* is defined, we can use a controlled version of U_{qDec} to simulate **qDec**, which is defined as:

$$U_{\text{qDec}}^* := U_{\perp} \circ P_{c^*} + U_{\text{qDec}} \circ (I - P_{c^*}). \quad (3)$$

Here $P_{c^*} := |c^*\rangle\langle c^*|$ is a projector acting on register **I**, and $U_{\perp}$ is a unitary operation acting on the register **O** which maps $|y\rangle$ to $|y \oplus \perp\rangle$ for all y . I is the identity operator.

4 Proof of Theorem 4

We prove Theorem 4 by a sequence of games $\mathbf{G}_0$ to $\mathbf{G}_{17}$ as shown in Fig. 7, Fig. 8 and Fig. 9. These games can be divided into the following three parts:

1. Games $\mathbf{G}_0$ to $\mathbf{G}_5$ analyzed in Section 4.1. This part focuses on changing the simulation of the quantum accessible decryption oracle **qDec** until the secret key **sk** is no longer needed.
2. Games $\mathbf{G}_5$ to $\mathbf{G}_{14}$ analyzed in Section 4.2. This part focuses on reprogramming the oracles **G** and **H** queried before $\mathcal{A}$ outputs (m_0^*, m_1^*) .
3. Games $\mathbf{G}_{14}$ to $\mathbf{G}_{17}$ analyzed in Section 4.3. This part focuses on reprogramming the oracles **G** and **H** queried during the generation of b' .

After completing these three parts, we finally get a trivial game $\mathbf{G}_{17}$ that outputs 1 with probability $1/2$, and combining the security bounds derived in each part, we obtain the security bound claimed in Theorem 4.

In the following proof, for simplicity, we may choose to include or omit the subscripts that denote the quantum registers of a quantum state, e.g., we may simplify $|c\rangle_{\text{I}}$ to $|c\rangle$. Generally, we will include the subscripts when they are crucial to our proof; otherwise, we may choose to omit them.

Game $\mathbf{G}_0$: This is the IND-qCCA game of $\text{OAEP}[\text{TDI}, \text{G}, \text{H}]$ with the adversary $\mathcal{A}$ in the QROM. Before the challenge ciphertext c^* is defined, the oracle **qDec** is simulated by U_{qDec} (Eq. (2)). After c^* is defined, the oracle **qDec** is simulated by U_{qDec}^* (Eq. (3)). Obviously, we have

$$\text{Adv}_{\text{OAEP}[\text{TDI}, \text{G}, \text{H}], \mathcal{A}}^{\text{IND-qCCA}} = \left| \Pr[1 \leftarrow \mathbf{G}_0] - \frac{1}{2} \right|. \quad (4)$$

4.1 Change the Simulation of qDec

In this section, based on game $\mathbf{G}_0$, we introduce games $\mathbf{G}_1$ to $\mathbf{G}_5$ as shown in Fig. 7 to change the simulation of the quantum accessible decryption oracle qDec. In game $\mathbf{G}_5$, the simulation of qDec no longer requires the secret key sk.

GAMES $\mathbf{G}_0$ to $\mathbf{G}_5$		$H(x_H, y_H\rangle)$
1. $(pk, sk) \leftarrow \text{Gen}$	// $\mathbf{G}_0$ - $\mathbf{G}_5$	apply $ x_H, y_H\rangle \mapsto x_H, y_H \oplus H(x_H)\rangle$ // $\mathbf{G}_0$ - $\mathbf{G}_2$
$G \leftarrow_{\$} \Omega_G, H \leftarrow_{\$} \Omega_H$	// $\mathbf{G}_0$ - $\mathbf{G}_2$	apply CStO _H // $\mathbf{G}_3$ - $\mathbf{G}_5$
2. $(m_0^*, m_1^*) \leftarrow \mathcal{A}^{G, H, \text{qDec}}(pk)$	// $\mathbf{G}_0$ - $\mathbf{G}_5$	
3. $b \leftarrow_{\$} \{0, 1\}$	// $\mathbf{G}_0$ - $\mathbf{G}_5$	qDec($ c, y\rangle$)
$r^* \leftarrow_{\$} \{0, 1\}^{k_0}$	// $\mathbf{G}_0$ - $\mathbf{G}_5$	if c^* has not been defined
$s^* = (m_b^* 0^{k_1}) \oplus G(r^*)$	// $\mathbf{G}_0$ - $\mathbf{G}_5$	apply U_{qDec} // $\mathbf{G}_0$
$t^* = r^* \oplus H(s^*)$	// $\mathbf{G}_0$ - $\mathbf{G}_5$	apply U_{Search} // $\mathbf{G}_5$
$c^* = f_{pk}(s^*, t^*)$	// $\mathbf{G}_0$ - $\mathbf{G}_5$	else apply U_{qDec}^* // $\mathbf{G}_0$
4. $b' \leftarrow \mathcal{A}^{G, H, \text{qDec}}(pk, c^*)$	// $\mathbf{G}_0$ - $\mathbf{G}_5$	apply U_{Search}^* // $\mathbf{G}_5$
5. return $[b' = b]$	// $\mathbf{G}_0$ - $\mathbf{G}_5$	qDec($ c, y\rangle$)
$G(x_G, y_G\rangle)$		if c^* has not been defined
apply $ x_G, y_G\rangle \mapsto x_G, y_G \oplus G(x_G)\rangle$	// $\mathbf{G}_0$	apply U_{qDec}^i // $\mathbf{G}_i, i \in [4]$
apply CStO _G	// $\mathbf{G}_1$ - $\mathbf{G}_5$	else apply $U_{\text{qDec}}^{i,*}$ // $\mathbf{G}_i, i \in [4]$

Fig. 7. Games $\mathbf{G}_0$ to $\mathbf{G}_5$ in the proof of Theorem 4. Here Ω_G and Ω_H denote the set of all functions $G : \{0, 1\}^{k_0} \rightarrow \{0, 1\}^{n+k_1}$ and $H : \{0, 1\}^{n+k_1} \rightarrow \{0, 1\}^{k_0}$, respectively.

Game $\mathbf{G}_1$: This game is the same as game $\mathbf{G}_0$, except that G is simulated as a compressed standard oracle with the database register D_{q_G} (Definition 3). Denote CStO_G as the unitary operation used to implement that compressed standard oracle, then in this game, the oracle qDec is simulated by

$$U_{\text{qDec}}^1 := U_{f^{-1}} \circ O_H \circ \text{CStO}_G \circ U_{\text{check}} \circ \text{CStO}_G \circ O_H \circ U_{f^{-1}}, \quad (5)$$

and $U_{\text{qDec}}^{1,*} := U_{\perp} \circ P_{c^*} + U_{\text{qDec}}^1 \circ (I - P_{c^*})$ before and after c^* is defined, respectively. Since the adversary $\mathcal{A}$ makes at most q_G queries to G , the two G oracles in games $\mathbf{G}_0$ and $\mathbf{G}_1$ are perfectly indistinguishable by Theorem 3, and thus

$$\Pr[1 \leftarrow \mathbf{G}_0] = \Pr[1 \leftarrow \mathbf{G}_1]. \quad (6)$$

Game $\mathbf{G}_2$: This game is the same as game $\mathbf{G}_1$, except that U_{qDec}^1 is replaced by

$$U_{\text{qDec}}^2 := U_{f^{-1}} \circ O_H \circ \text{Test}_G \circ U_{\text{check}} \circ \text{Test}_G \circ O_H \circ U_{f^{-1}}, \quad (7)$$

and $U_{\text{qDec}}^{1,*}$ is replaced by $U_{\text{qDec}}^{2,*} := U_{\perp} \circ P_{c^*} + U_{\text{qDec}}^2 \circ (I - P_{c^*})$. Here Test_G is a new unitary operation acting on the registers S , T and D_{q_G} as

$$\text{Test}_G : |s, t, D_G\rangle \mapsto \begin{cases} |s \oplus D_G(t), t, D_G\rangle, & \text{if } D_G(t) \neq \perp, \\ |s \oplus \perp, t, D_G\rangle, & \text{otherwise.} \end{cases}$$

In game $\mathbf{G}_2$, since the Test_G does not change the database state $|D_G\rangle$ in the computational basis and only "read" some information from D_G , the simulation of qDec will no longer update the (computational basis) state of the database register D_{q_G} . As for the difference between game $\mathbf{G}_1$ and $\mathbf{G}_2$, we can prove the following lemma, and the detailed proof is given in Supplementary Material D.1.

Lemma 1. $|\Pr[1 \leftarrow \mathbf{G}_1] - \Pr[1 \leftarrow \mathbf{G}_2]| \leq 8q_D \cdot \sqrt{2^{-k_1}}$.

Game $\mathbf{G}_3$: This game is the same as game $\mathbf{G}_2$, except that H is simulated as a compressed standard oracle with the database register D_{q_H} (Definition 3). Denote CStO_H as the unitary operation used to implement that compressed standard oracle, then in this game, the oracle qDec is simulated by

$$U_{\text{qDec}}^3 := U_{f^{-1}} \circ \text{CStO}_H \circ \text{Test}_G \circ U_{\text{check}} \circ \text{Test}_G \circ \text{CStO}_H \circ U_{f^{-1}}, \quad (8)$$

and $U_{\text{qDec}}^{3,*} := U_{\perp} \circ P_{c^*} + U_{\text{qDec}}^3 \circ (I - P_{c^*})$ before and after c^* is defined, respectively.

Similar to the analysis of the game hop from $\mathbf{G}_0$ to $\mathbf{G}_1$, we have

$$\Pr[1 \leftarrow \mathbf{G}_2] = \Pr[1 \leftarrow \mathbf{G}_3]. \quad (9)$$

Game $\mathbf{G}_4$: This game is the same as game $\mathbf{G}_3$, except that U_{qDec}^3 is replaced by

$$U_{\text{qDec}}^4 := U_{f^{-1}} \circ \text{Test}_H \circ \text{Test}_G \circ U_{\text{check}} \circ \text{Test}_G \circ \text{Test}_H \circ U_{f^{-1}}, \quad (10)$$

and $U_{\text{qDec}}^{3,*}$ is replaced by $U_{\text{qDec}}^{4,*} := U_{\perp} \circ P_{c^*} + U_{\text{qDec}}^4 \circ (I - P_{c^*})$. Here Test_H is a new unitary operation acting on the registers S , T and D_{q_H} as

$$\text{Test}_H : |s, t, D_H\rangle \mapsto \begin{cases} |s, t \oplus D_H(s), D_H\rangle, & \text{if } D_H(s) \neq \perp, \\ |s, t \oplus \perp, D_H\rangle, & \text{otherwise.} \end{cases}$$

In the simulation of qDec in game $\mathbf{G}_4$, both the compressed standard oracle implementation of G and H are changed into a direct database "read" (i.e., Test_G and Test_H). Thus the simulation of qDec in game $\mathbf{G}_4$ will no longer update the (computational basis) state on the database registers D_{q_G} and D_{q_H} . As for the difference between game $\mathbf{G}_3$ and $\mathbf{G}_4$, similar to the proof strategy of Lemma 1, we can prove the following lemma, and the detailed proof is given in Supplementary Material D.3.

Lemma 2. $|\Pr[1 \leftarrow \mathbf{G}_3] - \Pr[1 \leftarrow \mathbf{G}_4]| \leq 8q_D \sqrt{q_G} \cdot \sqrt{2^{-k_0}}$.

Game $\mathbf{G}_5$: This game is the same as game $\mathbf{G}_4$, except that U_{qDec}^4 is replaced by

$$U_{\text{Search}} : |c, y, D_G, D_H\rangle \mapsto |c, y \oplus \text{Search}(\text{pk}, c, D_G, D_H), D_G, D_H\rangle,$$

which acts on the registers $\text{I}, \text{O}, D_{q_H}$ and D_{q_G} , and $U_{\text{qDec}}^{4,*}$ is replaced by $U_{\text{Search}}^* := U_{\perp} \circ P_{c^*} + U_{\text{Search}} \circ (I - P_{c^*})$. Here Search is a classical function defined as

$$\text{Search}(\text{pk}, c, D_G, D_H) = \begin{cases} [s \oplus D_G(r)]^n \exists (s, D_H(s)) \text{ and } (r, D_G(r)) \text{ s.t.} \\ \quad f_{\text{pk}}(s, r \oplus D_H(s)) = c \wedge [s]_{k_1} = [D_G(r)]_{k_1}, \\ \perp \quad \text{otherwise.} \end{cases} \quad (11)$$

One can notice that the computation of Search does not use the secret key sk and only uses the public key pk to compute $f_{\text{pk}}(\cdot)$. So the simulation of qDec in game $\mathbf{G}_5$ no longer uses the secret key sk . Next, in Supplementary Material D.4 we prove that for any joint state $|\Phi\rangle$ just before one decryption query on registers $\text{I}, \text{O}, D_{q_H}$ and D_{q_G} , the following equations hold:

$$U_{\text{qDec}}^4 |\Phi\rangle |0\rangle_{\text{ST}} = U_{\text{Search}} |\Phi\rangle |0\rangle_{\text{ST}} \text{ and } U_{\text{qDec}}^{4,*} |\Phi\rangle |0\rangle_{\text{ST}} = U_{\text{Search}}^* |\Phi\rangle |0\rangle_{\text{ST}}.$$

So it is obvious that adversary $\mathcal{A}$ has an identical view between game $\mathbf{G}_4$ and $\mathbf{G}_5$. Hence, for the difference between game $\mathbf{G}_4$ and $\mathbf{G}_5$, we have

$$\Pr[1 \leftarrow \mathbf{G}_4] = \Pr[1 \leftarrow \mathbf{G}_5]. \quad (12)$$

Now, by combining Eq. (6), (9), (12), Lemma 1 and Lemma 2, we obtain

$$|\Pr[1 \leftarrow \mathbf{G}_0] - \Pr[1 \leftarrow \mathbf{G}_5]| \leq 8q_D \cdot \sqrt{2^{-k_1}} + 8q_D \sqrt{q_G} \cdot \sqrt{2^{-k_0}}. \quad (13)$$

4.2 Reprogram $\mathbf{G}$ and $\mathbf{H}$ Queried before $\mathcal{A}$ Outputs (m_0^*, m_1^*)

Here we introduce games $\mathbf{G}_6$ to $\mathbf{G}_{14}$ as shown in Fig. 8 to reprogram oracles $\mathbf{G}$ and $\mathbf{H}$ queried before $\mathcal{A}$ outputs (m_0^*, m_1^*) . After this reprogramming, the $\mathbf{G}(r^*)$ and $\mathbf{H}(s^*)$ used to generate the challenge ciphertext c^* are independent of oracles $\mathbf{G}$ and $\mathbf{H}$ queried before $\mathcal{A}$ outputs (m_0^*, m_1^*) ¹³. Hence, from now on, we can focus only on reprogramming the oracles $\mathbf{G}$ and $\mathbf{H}$ queried during the generation of b' , which is actually the topic of the next section.

Game $\mathbf{G}_6$: Compared with game $\mathbf{G}_5$, this new game has the following two changes:

- Before $\mathcal{A}$ outputs (m_0^*, m_1^*) , $\mathbf{G}$ is simulated by $\text{CStO}_G \circ \mathcal{O}_{\{r^*\}}^{SC}$, which is an operation that first queries the semi-classical oracle $\mathcal{O}_{\{r^*\}}^{SC}$ (Definition 1) on the input register of $\mathbf{G}$ and then performs CStO_G .
- If Find_{r^*} occurs before $\mathcal{A}$ outputs (m_0^*, m_1^*) , the game finally returns $\perp$. Here Find_{r^*} is the event that the measurement of $\mathcal{O}_{\{r^*\}}^{SC}$ ever has result 1.

We first construct a quantum oracle algorithm $\mathcal{A}_1$ as follows, which has oracle access to a constant zero function $\mathbf{F}$.

¹³ But not independent of the oracles $\mathbf{G}$ and $\mathbf{H}$ queried during the generation of b' .

GAMES $\mathbf{G}_5$ to $\mathbf{G}_{14}$	$\mathbf{G}(x_G, y_G\rangle)$
1. $(pk, sk) \leftarrow \text{Gen}, r^* \leftarrow_{\$} \{0, 1\}^{k_0}$ // $\mathbf{G}_5$ - $\mathbf{G}_{14}$	if (m_0^*, m_1^*) has not been computed
$g^* \leftarrow_{\$} \{0, 1\}^{n+k_1}$ // $\mathbf{G}_7$	apply CStO_G // $\mathbf{G}_5, \mathbf{G}_{14}$
$s^* \leftarrow_{\$} \{0, 1\}^{n+k_1}$ // $\mathbf{G}_8$ - $\mathbf{G}_{14}$	apply $\text{CStO}_G \circ \mathcal{O}_{\{r^*\}}^{SC}$ // $\mathbf{G}_6$ - $\mathbf{G}_{13}$
$h^* \leftarrow_{\$} \{0, 1\}^{k_0}$ // $\mathbf{G}_{10}$	if Find_{r^*} occurs, return $\perp$
$t^* \leftarrow_{\$} \{0, 1\}^{k_0}$ // $\mathbf{G}_{11}$ - $\mathbf{G}_{14}$	else
2. $(m_0^*, m_1^*) \leftarrow \mathcal{A}^{G, H, \text{qDec}}(pk)$ // $\mathbf{G}_5$ - $\mathbf{G}_{14}$	apply CStO_G // $\mathbf{G}_5$ - $\mathbf{G}_{11}$
3. $b \leftarrow_{\$} \{0, 1\}$ // $\mathbf{G}_5$ - $\mathbf{G}_{14}$	apply $\text{CStO}_G^{r^*, g^*}$ // $\mathbf{G}_{12}$ - $\mathbf{G}_{14}$
$s^* = (m_b^* 0^{k_1}) \oplus G(r^*)$ // $\mathbf{G}_5$ - $\mathbf{G}_6$	$H(x_H, y_H\rangle)$
$s^* = (m_b^* 0^{k_1}) \oplus g^*$ // $\mathbf{G}_7$	if (m_0^*, m_1^*) has not been computed
perform $\text{StdDecomp}_{r^*} \circ U_{g^*}^{r^*}$ // $\mathbf{G}_7$ - $\mathbf{G}_{12}$	apply CStO_H // $\mathbf{G}_5$ - $\mathbf{G}_8, \mathbf{G}_{14}$
perform $U_{g^*}^{r^*}$ // $\mathbf{G}_{13}$ - $\mathbf{G}_{14}$	apply $\text{CStO}_H \circ \mathcal{O}_{\{s^*\}}^{SC}$ // $\mathbf{G}_9$ - $\mathbf{G}_{13}$
$g^* = (m_b^* 0^{k_1}) \oplus s^*$ // $\mathbf{G}_8$ - $\mathbf{G}_{14}$	if Find_{s^*} occurs, return $\perp$
$t^* = r^* \oplus H(s^*)$ // $\mathbf{G}_5$ - $\mathbf{G}_9$	else
$t^* = r^* \oplus h^*$ // $\mathbf{G}_{10}$	apply CStO_H // $\mathbf{G}_5$ - $\mathbf{G}_{11}$
$h^* = r^* \oplus t^*$ // $\mathbf{G}_{11}$ - $\mathbf{G}_{14}$	apply $\text{CStO}_H^{s^*, h^*}$ // $\mathbf{G}_{12}$ - $\mathbf{G}_{14}$
perform $\text{StdDecomp}_{s^*} \circ U_{h^*}^{s^*}$ // $\mathbf{G}_{10}$ - $\mathbf{G}_{12}$	$\text{qDec}(c, y\rangle)$
perform $U_{h^*}^{s^*}$ // $\mathbf{G}_{13}$ - $\mathbf{G}_{14}$	if c^* has not been computed
$c^* = f_{pk}(s^*, t^*)$ // $\mathbf{G}_5$ - $\mathbf{G}_{14}$	apply U_{Search} // $\mathbf{G}_5$ - $\mathbf{G}_{14}$
4. $b' \leftarrow \mathcal{A}^{G, H, \text{qDec}}(pk, c^*)$ // $\mathbf{G}_5$ - $\mathbf{G}_{14}$	else apply U_{Search}^* // $\mathbf{G}_5$ - $\mathbf{G}_{14}$
5. return $\llbracket b' = b \rrbracket$ // $\mathbf{G}_5$ - $\mathbf{G}_{14}$	

Fig. 8. Games $\mathbf{G}_5$ to $\mathbf{G}_{14}$ in the proof of Theorem 4. The game $\mathbf{G}_5$ shows in this figure modifies the random selection of r^* to be placed just after the generation of (pk, sk) , which is only a conceptual change compared with the game $\mathbf{G}_5$ shown in Fig. 7.

- $\mathcal{A}_1$ runs game $\mathbf{G}_5$. Specifically, when $\mathcal{A}$ queries $\mathbf{G}$ using an (informal) query state $\sum |x, y\rangle_{X_G Y_G}$ before $\mathcal{A}$ outputs (m_0^*, m_1^*) , $\mathcal{A}_1$ will perform an additional operation: first expand this query state into $\sum |x, y\rangle \otimes_{X_G Y_G} |0\rangle_{O_F}$ and then forward the input/output register X_G/O_F to its function oracle F .

Obviously $\mathcal{A}_1$ makes at most q_G queries to its oracle, and such additional queries do not disturb $\mathcal{A}$'s computation since F is a constant zero function. Hence,

$$\Pr[1 \leftarrow \mathbf{G}_5] = \Pr[b = 1 : b \leftarrow \mathcal{A}_1^F(r^*, z), r^* \leftarrow_{\$} \{0, 1\}^{k_0}, z \leftarrow D_{\mathcal{A}_1}].$$

Here we particularly write r^* and assume (r^*, z) as the classical input of $\mathcal{A}_1$ without loss of generality, where r^* is uniformly sampled from $\{0, 1\}^{k_0}$ and z is sampled according to a distribution $D_{\mathcal{A}_1}$. Then, if we change F into $F \setminus \{r^*\}$, i.e., the oracle that first queries $\mathcal{O}_{\{r^*\}}^{SC}$ on the input register of F and then queries F , it is easy to check that

$$\Pr[1 \leftarrow \mathbf{G}_6] = \Pr[(b = 1) \wedge \neg \text{Find}_{r^*} : b \leftarrow \mathcal{A}_1^{F \setminus \{r^*\}}(r^*, z), r^* \leftarrow_{\$} \{0, 1\}^{k_0}, z \leftarrow D_{\mathcal{A}_1}].$$

Let D be a joint distribution of $(E, F, \{r^*\}, z)$, where E, F are both constant zero functions¹⁴, $r^* \leftarrow_{\$} \{0, 1\}^{k_0}$ and $z \leftarrow D_{\mathcal{A}_1}$. Then by applying Theorem 1 with this joint distribution D , we have

$$\begin{aligned} |\Pr[1 \leftarrow \mathbf{G}_5] - \Pr[1 \leftarrow \mathbf{G}_6]| &\leq \sqrt{(q_G + 1) \cdot \Pr[\text{Find}_{r^*} : \mathcal{A}_1^{F \setminus \{r^*\}}, (E, F, \{r^*\}, z) \leftarrow D]} \\ &= \sqrt{(q_G + 1) \cdot \Pr[\text{Find}_{r^*} : \mathbf{G}_6]}. \end{aligned} \quad (14)$$

The above final equation follows from the definition of $\mathcal{A}_1$ and game $\mathbf{G}_6$.

Game $\mathbf{G}_7$: This game is the same as game $\mathbf{G}_6$ except that the computation of s^* is changed. In this game, s^* is generated as $(m_b^* || 0^{k_1}) \oplus g^*$, where g^* is a new bitstring uniformly sampled from $\{0, 1\}^{n+k_1}$ at the beginning of the game. And after the generation of s^* , the $U_{g^*}^{r^*}$ and StdDecomp_{r^*} are sequentially performed on the database register D_{q_G} , where StdDecomp_{r^*} is the local decompression procedure defined in Section 2.4 and $U_{g^*}^{r^*}$ is an involution defined as:

$$U_{g^*}^{r^*} : |D_G\rangle \mapsto \begin{cases} |D_G \cup (r^*, g^*)\rangle & \text{if } D_G(r^*) = \perp, \\ |D_G \setminus (r^*, g^*)\rangle & \text{if } D_G(r^*) = g^*, \\ |D_G\rangle & \text{otherwise.} \end{cases} \quad (15)$$

In game $\mathbf{G}_6$, s^* is generated as $(m_b^* || 0^{k_1}) \oplus G(r^*)$, so a classical query to $\mathbf{G}$ by r^* is needed. Since $\mathbf{G}$ is simulated by CStO_G , this classical query after (m_0^*, m_1^*) has been computed in game $\mathbf{G}_6$ is actually processed as follows:

1. Initialize the query registers $X_G Y_G$ and prepare the initial state as $|r^*, 0\rangle$.

¹⁴ Here function E is introduced only to ensure that the conditions for applying Theorem 1 are met. In the corresponding games $\mathbf{G}_5$ and $\mathbf{G}_6$, the adversary $\mathcal{A}$ has access to the oracles F and $F \setminus \{r^*\}$, respectively.

2. Perform CStO_G on the registers $X_G Y_G D_{q_G}$, measure $X_G Y_G$ to get the outcome (r^*, y) and then rephrase y as $G(r^*)$.

We note that if Find_{r^*} does not occur in game $\mathbf{G}_6$, this y must be uniformly random in $\{0, 1\}^{n+k_1}$. In fact, by the definition of the semi-classical oracle (Definition 1), if Find_{r^*} does not occur, r^* is never queried by the adversary, and hence the whole (unit) state of the game just before computing s^* can be written as $\sum_{z, D_G(r^*)=\perp} \alpha_{z, D_G} |z, D_G\rangle$, where $|z\rangle$ denotes the other state that may entangle with $|D_G\rangle$. Then, the classical query to G by r^* has the following computation:

$$\begin{aligned}
& \sum_{z, D_G(r^*)=\perp} \alpha_{z, D_G} |z, D_G\rangle \xrightarrow{\text{Initialize } X_G Y_G} |r^*, 0\rangle \sum_{z, D_G(r^*)=\perp} \alpha_{z, D_G} |z, D_G\rangle \xrightarrow{\text{StdDecomp}_{r^*}} \\
& \sum_{z, D_G(r^*)=\perp} \frac{\alpha_{z, D_G}}{\sqrt{2^{n+k_1}}} |r^*, 0\rangle \sum_{y_0 \in \{0, 1\}^{n+k_1}} |z, D_G \cup (r^*, y_0)\rangle \xrightarrow{\text{CNOT}^{r^*}} \\
& \sum_{y_0 \in \{0, 1\}^{n+k_1}} |r^*, y_0\rangle \sum_{z, D_G(r^*)=\perp} \frac{\alpha_{z, D_G}}{\sqrt{2^{n+k_1}}} |z, D_G \cup (r^*, y_0)\rangle \xrightarrow{\text{StdDecomp}_{r^*}} \\
|\psi\rangle := & \sum_{y_0 \in \{0, 1\}^{n+k_1}} |r^*, y_0\rangle \sum_{z, D_G(r^*)=\perp} \frac{\alpha_{z, D_G}}{\sqrt{2^{n+k_1}}} \text{StdDecomp}_{r^*} |z, D_G \cup (r^*, y_0)\rangle.
\end{aligned}$$

At this point, based on the above state $|\psi\rangle$, it is obvious to see that the measurement on registers $X_G Y_G$ will get (r^*, y) for any $y \in \{0, 1\}^{n+k_1}$ with probability $1/2^{n+k_1}$, since $\sum_{z, D_G(r^*)=\perp} \alpha_{z, D_G} |z, D_G\rangle$ is unit and StdDecomp_{r^*} is a unitary operation. So we indeed get a uniformly random y after we make a classical query to G by r^* . Meanwhile, the state $|\psi\rangle$ will collapse into $\sum_{z, D_G(r^*)=\perp} \alpha_{z, D_G} \text{StdDecomp}_{r^*} |z, D_G \cup (r^*, y)\rangle$ ¹⁵ after we get the measurement outcome y and rephrase it as $G(r^*)$.

Instead of querying G to get $G(r^*)$, game $\mathbf{G}_7$ directly chooses a uniformly random g^* from $\{0, 1\}^{n+k_1}$ as $G(r^*)$ and then sequentially performs $U_{g^*}^*$ (Eq. (15)) and StdDecomp_{r^*} to update the database state. In fact, if Find_{r^*} does not occur in game $\mathbf{G}_7$, the whole (unit) state of the game just before computing s^* can also be written as $\sum_{z, D_G(r^*)=\perp} \alpha_{z, D_G} |z, D_G\rangle$, and one can easily check that, after sequentially performing $U_{g^*}^*$ and StdDecomp_{r^*} , that whole state will become $\sum_{z, D_G(r^*)=\perp} \alpha_{z, D_G} \text{StdDecomp}_{r^*} |z, D_G \cup (r^*, g^*)\rangle$, where g^* is uniformly random.

Now we can conclude that, if Find_{r^*} does not occur in games $\mathbf{G}_6$ and $\mathbf{G}_7$, their corresponding whole states just after the simulation of $G(r^*)$ are identical¹⁶ even though they apply different computing methods. This means that these two games must have the same final output statistics. Thus

¹⁵ Since the query registers $X_G Y_G$ are measured, we omit $|r^*, y\rangle$ in this collapsed state.

¹⁶ More rigorously speaking, these two whole states are both mixed states and have the same density matrix.

$$\begin{aligned}
\Pr[b = 1 \wedge \neg \text{Find}_{r^*} : b \leftarrow \mathbf{G}_6] &= \Pr[b = 1 | \neg \text{Find}_{r^*} : b \leftarrow \mathbf{G}_6] \cdot \Pr[\neg \text{Find}_{r^*} : \mathbf{G}_6] \\
&= \Pr[b = 1 | \neg \text{Find}_{r^*} : b \leftarrow \mathbf{G}_7] \cdot \Pr[\neg \text{Find}_{r^*} : \mathbf{G}_6] \\
&\stackrel{(a)}{=} \Pr[b = 1 | \neg \text{Find}_{r^*} : b \leftarrow \mathbf{G}_7] \cdot \Pr[\neg \text{Find}_{r^*} : \mathbf{G}_7] \\
&= \Pr[b = 1 \wedge \neg \text{Find}_{r^*} : b \leftarrow \mathbf{G}_7].
\end{aligned}$$

Here (a) uses the fact that $\Pr[\neg \text{Find}_{r^*} : \mathbf{G}_6] = \Pr[\neg \text{Find}_{r^*} : \mathbf{G}_7]$, which is obvious since the changes made in game $\mathbf{G}_7$ do not affect the probability that Find_{r^*} occurs. Then, since the construction of games $\mathbf{G}_6$ and $\mathbf{G}_7$ ensures that games $\mathbf{G}_6$ and $\mathbf{G}_7$ returning 1 implies that Find_{r^*} does not occur, the above equation means that

$$\Pr[1 \leftarrow \mathbf{G}_6] = \Pr[1 \leftarrow \mathbf{G}_7]. \quad (16)$$

Game $\mathbf{G}_8$: Different from game $\mathbf{G}_7$, which first uniformly samples g^* and then computes $s^* = (m_b^* || 0^{k_1}) \oplus g^*$, this game first uniformly samples s^* at the beginning and then computes $g^* = (m_b^* || 0^{k_1}) \oplus s^*$. Obviously this is only a conceptual change, thus

$$\Pr[1 \leftarrow \mathbf{G}_7] = \Pr[1 \leftarrow \mathbf{G}_8]. \quad (17)$$

Game $\mathbf{G}_9$: Compared with game $\mathbf{G}_8$, this new game has the following two changes:

- Before $\mathcal{A}$ outputs (m_0^*, m_1^*) , $\mathbf{H}$ is simulated by $\text{CStO}_{\mathbf{H}} \circ \mathcal{O}_{\{s^*\}}^{SC}$, which is an operation that first queries the semi-classical oracle $\mathcal{O}_{\{s^*\}}^{SC}$ (Definition 1) on the input register of $\mathbf{H}$ and then performs $\text{CStO}_{\mathbf{H}}$.
- If Find_{s^*} occurs before $\mathcal{A}$ outputs (m_0^*, m_1^*) , the game finally returns $\perp$. Here Find_{s^*} is the event that the measurement of $\mathcal{O}_{\{s^*\}}^{SC}$ ever has result 1.

Following a similar analysis in the game hop from $\mathbf{G}_5$ to $\mathbf{G}_6$, and shift the perspective from oracle $\mathbf{G}$ to oracle $\mathbf{H}$, we can obtain the following formula.

$$|\Pr[1 \leftarrow \mathbf{G}_8] - \Pr[1 \leftarrow \mathbf{G}_9]| \leq \sqrt{(q_{\mathbf{H}} + 1) \cdot \Pr[\text{Find}_{s^*} : \mathbf{G}_9]}. \quad (18)$$

As for the $\Pr[\text{Find}_{r^*} : \mathbf{G}_6]$ used in Eq. (14) and the above $\Pr[\text{Find}_{s^*} : \mathbf{G}_9]$, we can prove the following lemma using Corollary 1, and the detailed proof is given in Supplementary Material D.5.

Lemma 3. $\Pr[\text{Find}_{r^*} : \mathbf{G}_6] \leq 4q_{\mathbf{G}} \cdot 2^{-k_0}$ and $\Pr[\text{Find}_{s^*} : \mathbf{G}_9] \leq 4q_{\mathbf{H}} \cdot 2^{-n-k_1}$.

Game $\mathbf{G}_{10}$: This game is the same as game $\mathbf{G}_9$ except that the computation of t^* is changed. In this game, t^* is generated as $r^* \oplus h^*$, where h^* is a new bitstring uniformly sampled from $\{0, 1\}^{k_0}$ at the beginning of the game. And after the generation of t^* , the $\mathbf{U}_{h^*}^{s^*}$ and StdDecomp_{s^*} are sequentially performed on the database register $\mathbf{D}_{q_{\mathbf{H}}}$, where StdDecomp_{s^*} is the local decompression procedure defined in Section 2.4 and $\mathbf{U}_{h^*}^{s^*}$ is an involution defined as:

$$\mathbf{U}_{h^*}^{s^*} : |D_{\mathbf{H}}\rangle \mapsto \begin{cases} |D_{\mathbf{H}} \cup (s^*, h^*)\rangle & \text{if } D_{\mathbf{H}}(s^*) = \perp, \\ |D_{\mathbf{H}} \setminus (s^*, h^*)\rangle & \text{if } D_{\mathbf{H}}(s^*) = h^*, \\ |D_{\mathbf{H}}\rangle & \text{otherwise.} \end{cases} \quad (19)$$

Following a similar analysis in the game hop from $\mathbf{G}_6$ to $\mathbf{G}_7$, and shift the perspective from oracle $\mathbf{G}$ to oracle $\mathbf{H}$, we can obtain

$$\Pr[1 \leftarrow \mathbf{G}_9] = \Pr[1 \leftarrow \mathbf{G}_{10}]. \quad (20)$$

Game $\mathbf{G}_{11}$: Different from game $\mathbf{G}_{10}$, which first uniformly samples h^* and then computes $t^* = r^* \oplus h^*$, this game first uniformly samples t^* at the beginning and then computes $h^* = r^* \oplus t^*$. Obviously this is only a conceptual change, thus

$$\Pr[1 \leftarrow \mathbf{G}_{10}] = \Pr[1 \leftarrow \mathbf{G}_{11}]. \quad (21)$$

Game $\mathbf{G}_{12}$: This game is the same as game $\mathbf{G}_{11}$, except that the simulation of oracles $\mathbf{G}$ and $\mathbf{H}$ during the generation of b' is changed. In fact, the $\text{CStO}_{\mathbf{G}}$ used to simulate $\mathbf{G}$ is changed into

$$\begin{aligned} \text{CStO}_{\mathbf{G}}^{r^*, g^*} := & \sum_{x \in \{0,1\}^{k_0} \setminus \{r^*\}} |x\rangle\langle x| \otimes \text{StdDecomp}_x \circ \text{CNOT}^x \circ \text{StdDecomp}_x \\ & + |r^*\rangle\langle r^*| \otimes \text{CNOT}_{g^*}, \end{aligned} \quad (22)$$

and the $\text{CStO}_{\mathbf{H}}$ used to simulate $\mathbf{H}$ is changed to

$$\begin{aligned} \text{CStO}_{\mathbf{H}}^{s^*, h^*} := & \sum_{x \in \{0,1\}^{n+k_1} \setminus \{s^*\}} |x\rangle\langle x| \otimes \text{StdDecomp}_x \circ \text{CNOT}^x \circ \text{StdDecomp}_x \\ & + |s^*\rangle\langle s^*| \otimes \text{CNOT}_{h^*}. \end{aligned} \quad (23)$$

Here CNOT_{g^*} (resp. CNOT_{h^*}) acts on the output register of $\mathbf{G}$ (resp. $\mathbf{H}$) and maps $|y\rangle$ into $|y \oplus g^*\rangle$ (resp. $|y \oplus h^*\rangle$).

Similar to the analysis in the game hop from $\mathbf{G}_6$ to $\mathbf{G}_7$, if Find_{r^*} and Find_{s^*} both do not occur in game $\mathbf{G}_{11}$, we know that the whole state of this game just after the computation of c^* can be written as

$$\sum_{\substack{z, D_{\mathbf{G}}(r^*) = \perp \\ D_{\mathbf{H}}(s^*) = \perp}} \alpha_{D_{\mathbf{G}}, D_{\mathbf{H}}}^z |z\rangle \otimes \text{StdDecomp}_{r^*} |D_{\mathbf{G}} \cup (r^*, g^*)\rangle \otimes \text{StdDecomp}_{s^*} |D_{\mathbf{H}} \cup (s^*, h^*)\rangle.$$

Here $|z\rangle$ denotes the other state that may entangle with $|D_{\mathbf{G}}\rangle$ and $|D_{\mathbf{H}}\rangle$. As for the next generation of b' , we note that the $\text{U}_{\text{Search}}^*$ used to simulate the decryption oracle $\mathbf{qDec}$ may disturb the database state and make it not in the form $\text{StdDecomp}_{r^*} |D_{\mathbf{G}} \cup (r^*, g^*)\rangle \otimes \text{StdDecomp}_{s^*} |D_{\mathbf{H}} \cup (s^*, h^*)\rangle$, and the reason is that $\text{U}_{\text{Search}}^*$ may not commute with StdDecomp_{r^*} or StdDecomp_{s^*} ¹⁷. Correspondingly, if we additionally assume that $\text{U}_{\text{Search}}^*$ commutes with StdDecomp_{r^*} and

¹⁷ In fact, $\text{U}_{\text{Search}}^*$ defined in game $\mathbf{G}_5$ makes a database read in the computational basis, while the StdDecomp_{r^*} and StdDecomp_{s^*} defined in Section 2.4 perform some operations under the Fourier basis, making it challenging to directly claim that these operations are commutative.

StdDecomp_{s^*} , the whole state of game $\mathbf{G}_{11}$ just before the first random oracle query (to $\mathbf{G}$ or $\mathbf{H}$) during the generation of b' can be written as

$$\sum_{\substack{z, D_{\mathbf{G}}(r^*)=\perp \\ D_{\mathbf{H}}(s^*)=\perp}} \beta_{D_{\mathbf{G}}, D_{\mathbf{H}}}^z |z\rangle \otimes \text{StdDecomp}_{r^*} |D_{\mathbf{G}} \cup (r^*, g^*)\rangle \otimes \text{StdDecomp}_{s^*} |D_{\mathbf{H}} \cup (s^*, h^*)\rangle,$$

since U_{Search}^* does not change the database state $|D_{\mathbf{G}}\rangle$ and $|D_{\mathbf{H}}\rangle$ in the computational basis. In the following proof we denote the above state as $|\Phi\rangle$. Based on this observation, we can prove the following claim.

Claim. In game $\mathbf{G}_{11}$, if neither Find_{r^*} nor Find_{s^*} occurs and U_{Search}^* commutes with both StdDecomp_{r^*} and StdDecomp_{s^*} , we can conclude that $\mathbf{G}(r^*) = g^*$ and $\mathbf{H}(s^*) = h^*$ in the adversary's view during the generation of b' .

Proof. As analyzed above, under the condition of this claim, the whole state of game $\mathbf{G}_{11}$ just before the first random oracle query during the generation of b' is a particular state $|\Phi\rangle$. Without loss of generality, we assume that the first random oracle query during the generation of b' is a $\mathbf{G}$ query.

Now for this first $\mathbf{G}$ query, the corresponding simulation process of $\mathbf{G}(r^*)$ can be described as follows, which we denote $|r^*, y\rangle$ as the query state.

$$\begin{aligned} & |r^*, y\rangle |\Phi\rangle \xrightarrow{\text{StdDecomp}_{r^*}} \\ & |r^*, y\rangle \sum \beta_{D_{\mathbf{G}}, D_{\mathbf{H}}}^z |z\rangle \otimes |D_{\mathbf{G}} \cup (r^*, g^*)\rangle \otimes \text{StdDecomp}_{s^*} |D_{\mathbf{H}} \cup (s^*, h^*)\rangle \xrightarrow{\text{CNOT}^{r^*}} \\ & |r^*, y \oplus g^*\rangle \sum \beta_{D_{\mathbf{G}}, D_{\mathbf{H}}}^z |z\rangle \otimes |D_{\mathbf{G}} \cup (r^*, g^*)\rangle \otimes \text{StdDecomp}_{s^*} |D_{\mathbf{H}} \cup (s^*, h^*)\rangle \\ & \xrightarrow{\text{StdDecomp}_{r^*}} |r^*, y \oplus g^*\rangle |\Phi\rangle. \end{aligned}$$

Therefore, according to this claim, the response to the adversary's first $\mathbf{G}$ query will consistently satisfy $\mathbf{G}(r^*) = g^*$, and the database state corresponding to the query state $|r^*, y\rangle$ remains unchanged after the simulation of this $\mathbf{G}$ query.

This actually means that, the reply to the subsequent queries to $\mathbf{G}$ will also maintain $\mathbf{G}(r^*) = g^*$. Hence according to this claim, $\mathbf{G}(r^*)$ is exactly g^* in the adversary's view before it outputs b' . A similar analysis applies to $\mathbf{H}(s^*)$, and it is exactly h^* in the adversary's view. Thus we complete the proof.

Based on the above claim, we make a modification in game $\mathbf{G}_{12}$, which is, the simulation of $\mathbf{G}$ (resp. $\mathbf{H}$) is switched from $\text{CStO}_{\mathbf{G}}$ (resp. $\text{CStO}_{\mathbf{H}}$) to $\text{CStO}_{\mathbf{G}}^{r^*, g^*}$ (resp. $\text{CStO}_{\mathbf{H}}^{s^*, h^*}$) defined in Eq. (22) (resp. Eq. (23)). Now for the query $|r^*, y\rangle$ to $\mathbf{G}$ (resp. $|s^*, y\rangle$ to $\mathbf{H}$) during the generation of b' , the simulation of game $\mathbf{G}_{12}$ will not perform any operation on the database state and directly XOR g^* (resp. h^*) onto y , where g^* (resp. h^*) is actually rephrased as $\mathbf{G}(r^*)$ (resp. $\mathbf{H}(s^*)$). Notice that the construction of games $\mathbf{G}_{11}$ and $\mathbf{G}_{12}$ ensures that the two games returning 1 implies that Find_{r^*} and Find_{s^*} do not occur, hence we have $\Pr[1 \leftarrow \mathbf{G}_{11}] = \Pr[1 \leftarrow \mathbf{G}_{12}]$ under the condition that U_{Search}^* commutes with StdDecomp_{r^*} and StdDecomp_{s^*} .

Lemma 4. Let $[A, B]$ be $AB - BA$ for any operations A and B , then we have $\|[\mathbf{U}_{\text{Search}}^*, \text{StdDecomp}_{r^*}]\| \leq 8 \cdot \sqrt{2^{-k_1}}$, $\|[\mathbf{U}_{\text{Search}}^*, \text{StdDecomp}_{s^*}]\| \leq 8\sqrt{q_G} \cdot \sqrt{2^{-k_0}}$.

In the above we introduce Lemma 4, which is proved in Supplementary Material D.6. It shows that $\mathbf{U}_{\text{Search}}^*$ indeed commutes with StdDecomp_{r^*} and StdDecomp_{s^*} except for a fixed error. Since $\mathbf{U}_{\text{Search}}^*$ is implemented at most q_D times during the generation of b' , we finally obtain

$$|\Pr[1 \leftarrow \mathbf{G}_{11}] - \Pr[1 \leftarrow \mathbf{G}_{12}]| \leq 8q_D \cdot \sqrt{2^{-k_1}} + 8q_D\sqrt{q_G} \cdot \sqrt{2^{-k_0}}. \quad (24)$$

Game $\mathbf{G}_{13}$: This game is the same as game $\mathbf{G}_{12}$, except that the performing of StdDecomp_{r^*} (resp. StdDecomp_{s^*}) just after the performing of $\mathbf{U}_{g^*}^*$ (resp. $\mathbf{U}_{h^*}^*$) is canceled.

To show the difference between game $\mathbf{G}_{12}$ and game $\mathbf{G}_{13}$, we present the following Lemma 5. At a high level, the core proof idea of this lemma is to exploit the commutativity of StdDecomp_{r^*} , StdDecomp_{s^*} , $\mathbf{U}_{g^*}^*$, $\mathbf{U}_{h^*}^*$ and $\mathbf{U}_{\text{Search}}^*$. For example, StdDecomp_{r^*} must commute with StdDecomp_{s^*} and $\mathbf{U}_{h^*}^*$ since they act on different database registers. Moreover, by Lemma 4, both StdDecomp_{r^*} and StdDecomp_{s^*} almost commute with $\mathbf{U}_{\text{Search}}^*$. Due to space limitations, we give the detailed proof of Lemma 5 in Supplementary Material D.7.

Lemma 5. $|\Pr[1 \leftarrow \mathbf{G}_{12}] - \Pr[1 \leftarrow \mathbf{G}_{13}]| \leq 8q_D \cdot \sqrt{2^{-k_1}} + 8q_D\sqrt{q_G} \cdot \sqrt{2^{-k_0}}$.

Game $\mathbf{G}_{14}$: Before $\mathcal{A}$ outputs (m_0^*, m_1^*) , the queries to the semi-classical oracles $\mathcal{O}_{\{r^*\}}^{SC}$ and $\mathcal{O}_{\{s^*\}}^{SC}$ performed in game $\mathbf{G}_{13}$ are canceled, meanwhile the two flags "Find $_{r^*}$ " occurs, game finally returns $\perp$ and "Find $_{s^*}$ " occurs, game finally returns $\perp$ are both removed.

By mimicking the analysis given in the game hop from $\mathbf{G}_5$ to $\mathbf{G}_6$ and $\mathbf{G}_8$ to $\mathbf{G}_9$, and then using Lemma 3, we obtain

$$|\Pr[1 \leftarrow \mathbf{G}_{13}] - \Pr[1 \leftarrow \mathbf{G}_{14}]| \leq 2(q_G + 1) \cdot \sqrt{2^{-k_0}} + 2(q_H + 1) \cdot \sqrt{2^{-n-k_1}}. \quad (25)$$

Now by combining Eq. (14), (16), (17), (18), (20), (21), (24), (25) and Lemma 3, 5, we obtain

$$\begin{aligned} |\Pr[1 \leftarrow \mathbf{G}_5] - \Pr[1 \leftarrow \mathbf{G}_{14}]| &\leq 4(q_H + 1) \cdot \sqrt{2^{-n-k_1}} + 16q_D \cdot \sqrt{2^{-k_1}} \\ &\quad + (16q_D\sqrt{q_G} + 4q_G + 4) \cdot \sqrt{2^{-k_0}}. \end{aligned} \quad (26)$$

4.3 Reprogram $\mathbf{G}$ and $\mathbf{H}$ Queried During the Generation of b'

In this section, we introduce games $\mathbf{G}_{15}$ to $\mathbf{G}_{17}$ as shown in Fig. 9 to reprogram the oracles $\mathbf{G}$ and $\mathbf{H}$ queried during the generation of b' . After this reprogramming, we get a trivial game $\mathbf{G}_{17}$ which returns 1 with the probability $1/2$.

GAMES $\mathbf{G}_{14}$ to $\mathbf{G}_{17}$		$G(x_G, y_G\rangle)/H(x_H, y_H\rangle)$	
1. $(pk, sk) \leftarrow \text{Gen}$	// $\mathbf{G}_{14}$ - $\mathbf{G}_{17}$	if (m_0^*, m_1^*) has not been defined	
$r^* \leftarrow_{\$} \{0, 1\}^{k_0}$	// $\mathbf{G}_{14}$ - $\mathbf{G}_{17}$	apply $\text{CStO}_G/\text{CStO}_H$	// $\mathbf{G}_{14}$ - $\mathbf{G}_{17}$
$s^* \leftarrow_{\$} \{0, 1\}^{n+k_1}, t^* \leftarrow_{\$} \{0, 1\}^{k_0}$	// $\mathbf{G}_{14}$ - $\mathbf{G}_{17}$	else	
2. $(m_0^*, m_1^*) \leftarrow \mathcal{A}^{G,H,q\text{Dec}}(pk)$	// $\mathbf{G}_{14}$ - $\mathbf{G}_{17}$	apply $\text{CStO}_G^{r^*, g^*}/\text{CStO}_H^{s^*, h^*}$	// $\mathbf{G}_{14}$ - $\mathbf{G}_{16}$
3. $b \xleftarrow{\$} \{0, 1\}$	// $\mathbf{G}_{14}$ - $\mathbf{G}_{17}$	apply $\text{CStO}_G/\text{CStO}_H$	// $\mathbf{G}_{17}$
$g^* = (m_b^* 0^{k_1}) \oplus s^*, h^* = r^* \oplus t^*$	// $\mathbf{G}_{14}$ - $\mathbf{G}_{15}$	$q\text{Dec}(c, y\rangle)$	
perform $U_{h^*}^{s^*} \circ U_{g^*}^{r^*}$	// $\mathbf{G}_{14}$ - $\mathbf{G}_{15}$	if c^* has not been defined	
$c^* = f_{pk}(s^*, t^*)$	// $\mathbf{G}_{14}$ - $\mathbf{G}_{17}$	apply U_{Search}	// $\mathbf{G}_{14}$ - $\mathbf{G}_{17}$
4. $b' \leftarrow \mathcal{A}^{G,H,q\text{Dec}}(pk, c^*)$	// $\mathbf{G}_{14}$ - $\mathbf{G}_{17}$	else apply U_{Search}^*	// $\mathbf{G}_{14}, \mathbf{G}_{16}$ - $\mathbf{G}_{17}$
5. return $\llbracket b' = b \rrbracket$	// $\mathbf{G}_{14}$ - $\mathbf{G}_{17}$	else apply U_{Search}^{**}	// $\mathbf{G}_{15}$

Fig. 9. Games $\mathbf{G}_{14}$ to $\mathbf{G}_{17}$ in the proof of Theorem 4.

Game $\mathbf{G}_{15}$: This game is the same as game $\mathbf{G}_{14}$, except that the U_{Search}^* used to simulate the decryption oracle $q\text{Dec}$ during the generation of b' is replaced by

$$U_{\text{Search}}^{**} := U_{h^*}^{s^*} \circ U_{g^*}^{r^*} \circ U_{\text{Search}}^* \circ U_{h^*}^{s^*} \circ U_{g^*}^{r^*},$$

where $U_{g^*}^{r^*}$ (Eq. (15)) acts on D_{q_G} and $U_{h^*}^{s^*}$ (Eq. (19)) acts on D_{q_H} .

Here we introduce the following Lemma 6, which shows that the difference between game $\mathbf{G}_{14}$ and game $\mathbf{G}_{15}$ can be upper bounded by the advantage of a PD-OW adversary $\mathcal{B}_1$ against TDI with some additional loss. The proof of this lemma mainly uses Theorem 1 and Theorem 6, and the detailed proof is given in Supplementary Material D.8.

Lemma 6. *There exists a PD-OW adversary $\mathcal{B}_1$ against TDI such that*

$$\begin{aligned} |\Pr[1 \leftarrow \mathbf{G}_{14}] - \Pr[1 \leftarrow \mathbf{G}_{15}]| &\leq 18q_D q \cdot \sqrt{2^{-k_0}} + 8q_D(q_H + 1) \cdot \sqrt{2^{-n-k_1}} \\ &\quad + 8q_D q \cdot \sqrt{\text{Adv}_{\text{TDI}, \mathcal{B}_1}^{\text{PD-OW}}}, \end{aligned}$$

and $\mathcal{T}_{\mathcal{B}_1} \leq \mathcal{T}_A + \mathcal{O}(q_G^2 + q_H^2) + \mathcal{O}(q_D q_G q_H) \cdot \mathcal{T}_f$. Here $q := q_G + q_H + 1$.

Game $\mathbf{G}_{16}$: Compared with game $\mathbf{G}_{15}$, this new game makes two changes: the computing of g^* and h^* , and the performing of $U_{g^*}^{r^*}$ and $U_{h^*}^{s^*}$ just before the generation of c^* are canceled, and the U_{Search}^{**} introduced in game $\mathbf{G}_{15}$ is changed back to U_{Search}^* .

To show the difference between game $\mathbf{G}_{15}$ and game $\mathbf{G}_{16}$, we present the following Lemma 7. At a high level, the core idea of its proof is to use the facts that $U_{g^*}^{r^*}$ and $U_{h^*}^{s^*}$ are both involutions, and that $U_{g^*}^{r^*}$ and $U_{h^*}^{s^*}$ both commute with $\text{CStO}_G^{r^*, g^*}$ and $\text{CStO}_H^{s^*, h^*}$. Due to space limitations, we give the detailed proof of Lemma 7 in Supplementary Material D.9.

Lemma 7. $\Pr[1 \leftarrow \mathbf{G}_{15}] = \Pr[1 \leftarrow \mathbf{G}_{16}]$.

Game $\mathbf{G}_{17}$: This game is the same as game $\mathbf{G}_{16}$, except that the simulation of $\mathbf{G}$ and $\mathbf{H}$ during the generation of b' is changed. In fact, the $\text{CStO}_{\mathbf{G}}^{*,g^*}$ used to simulate $\mathbf{G}$ is changed back to $\text{CStO}_{\mathbf{G}}$, and the $\text{CStO}_{\mathbf{H}}^{s^*,h^*}$ used to simulate $\mathbf{H}$ is changed back to $\text{CStO}_{\mathbf{H}}$.

In fact we can prove the following lemma, which shows that the difference between game $\mathbf{G}_{16}$ and game $\mathbf{G}_{17}$ can be upper bounded by the advantage of a PD-OW adversary $\mathcal{B}_2$ against TDI with some additional loss. The detailed proof of this lemma is given in Supplementary Material D.10.

Lemma 8. *There exists a PD-OW adversary $\mathcal{B}_2$ against TDI such that*

$$|\Pr[1 \leftarrow \mathbf{G}_{16}] - \Pr[1 \leftarrow \mathbf{G}_{17}]| \leq 4q \cdot \sqrt{2^{-k_0}} + 4q \cdot \sqrt{\text{Adv}_{\text{TDI}, \mathcal{B}_2}^{\text{PD-OW}}},$$

and $\mathcal{T}_{\mathcal{B}_2} \leq \mathcal{T}_{\mathcal{A}} + \mathcal{O}(q_{\mathbf{G}}^2 + q_{\mathbf{H}}^2) + \mathcal{O}(q_{\mathbf{D}}q_{\mathbf{G}}q_{\mathbf{H}}) \cdot \mathcal{T}_f$. Here $q := q_{\mathbf{G}} + q_{\mathbf{H}} + 1$.

It is easy to check that $\Pr[1 \leftarrow \mathbf{G}_{17}] = 1/2$, then by combining this equation with Lemma 6, 7 and 8, we obtain (denote $q := q_{\mathbf{G}} + q_{\mathbf{H}} + 1$)

$$\begin{aligned} \left| \Pr[1 \leftarrow \mathbf{G}_{14}] - \frac{1}{2} \right| &\leq (18q_{\mathbf{D}} + 4)q \cdot \sqrt{2^{-k_0}} + 8q_{\mathbf{D}}(q_{\mathbf{H}} + 1) \cdot \sqrt{2^{-n-k_1}} \\ &\quad + 8q_{\mathbf{D}}q \cdot \left(\sqrt{\text{Adv}_{\text{TDI}, \mathcal{B}_1}^{\text{PD-OW}}} + \sqrt{\text{Adv}_{\text{TDI}, \mathcal{B}_2}^{\text{PD-OW}}} \right). \end{aligned} \quad (27)$$

4.4 Putting Everything Together

For the PD-OW adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ involved in Eq. (27), it is evident that we can fold them into one PD-OW adversary $\mathcal{B}$ such that $\text{Adv}_{\text{TDI}, \mathcal{B}_1}^{\text{PD-OW}} + \text{Adv}_{\text{TDI}, \mathcal{B}_2}^{\text{PD-OW}} = 2 \cdot \text{Adv}_{\text{TDI}, \mathcal{B}}^{\text{PD-OW}}$ and $\mathcal{T}_{\mathcal{B}} \leq \max\{\mathcal{T}_{\mathcal{B}_1}, \mathcal{T}_{\mathcal{B}_2}\}$. Here we also have $2^{-n-k_1} \leq \text{Adv}_{\text{TDI}, \mathcal{B}}^{\text{PD-OW}}$ since the aim of $\mathcal{B}$ is to guess a bitstring with length $n + k_1$. Now by combining Eq. (4), Eq. (13), Eq. (26) and Eq. (27), and using the above two properties, we finally prove our Theorem 4, and we restate it below.

Theorem 5 (PD-OW of TDI $\stackrel{\text{QROM}}{\Rightarrow}$ IND-qCCA of OAEP[TDI, $\mathbf{G}$, $\mathbf{H}$]). *Let $\mathcal{A}$ be an IND-qCCA adversary against OAEP[TDI, $\mathbf{G}$, $\mathbf{H}$] in the QROM, making at most $q_{\mathbf{D}}$, $q_{\mathbf{G}}$ and $q_{\mathbf{H}}$ quantum queries to the decryption oracle, random oracle $\mathbf{G}$ and random oracle $\mathbf{H}$. Let $q := q_{\mathbf{G}} + q_{\mathbf{H}} + 1$. Then we can construct a PD-OW adversary $\mathcal{B}$ against the trapdoor injective function TDI such that*

$$\begin{aligned} \text{Adv}_{\text{OAEP}[\text{TDI}, \mathbf{G}, \mathbf{H}], \mathcal{A}}^{\text{IND-qCCA}} &\leq (32q_{\mathbf{D}} + 4)q \cdot \sqrt{\text{Adv}_{\text{TDI}, \mathcal{B}}^{\text{PD-OW}}} + (42q_{\mathbf{D}} + 8)q \cdot \sqrt{2^{-k_0}} \\ &\quad + 24q_{\mathbf{D}} \cdot \sqrt{2^{-k_1}}, \end{aligned}$$

and $\mathcal{T}_{\mathcal{B}} \leq \mathcal{T}_{\mathcal{A}} + \mathcal{O}(q_{\mathbf{G}}^2 + q_{\mathbf{H}}^2) + \mathcal{O}(q_{\mathbf{D}}q_{\mathbf{G}}q_{\mathbf{H}}) \cdot \mathcal{T}_f$.

Acknowledgments. We thank anonymous reviewers for their insightful comments. This work is supported by the National Natural Science Foundation of China (Grants No. 62172405, 62302496).

References

1. Ambainis, A., Hamburg, M., Unruh, D.: Quantum security proofs using semi-classical oracles. In: Advances in Cryptology - CRYPTO 2019 - 39th Annual International Cryptology Conference, Proceedings, Part II. pp. 269–295. Springer (2019). https://doi.org/10.1007/978-3-030-26951-7_10
2. Backes, M., Dürmuth, M., Unruh, D.: OAEP is secure under key-dependent messages. In: Pieprzyk, J. (ed.) Advances in Cryptology - ASIACRYPT 2008, 14th International Conference on the Theory and Application of Cryptology and Information Security, Proceedings. pp. 506–523. Springer (2008). https://doi.org/10.1007/978-3-540-89255-7_31
3. Bellare, M., Rogaway, P.: Optimal asymmetric encryption. In: Santis, A.D. (ed.) Advances in Cryptology - EUROCRYPT '94, Workshop on the Theory and Application of Cryptographic Techniques, Proceedings. Lecture Notes in Computer Science, vol. 950, pp. 92–111. Springer (1994). <https://doi.org/10.1007/BFB0053428>
4. Bindel, N., Hamburg, M., Hövelmanns, K., Hülsing, A., Persichetti, E.: Tighter proofs of CCA security in the quantum random oracle model. In: Theory of Cryptography Conference. pp. 61–90. Springer (2019). https://doi.org/10.1007/978-3-030-36033-7_3
5. Boneh, D.: Simplified OAEP for the RSA and rabin functions. In: Advances in Cryptology - CRYPTO 2001, 21st Annual International Cryptology Conference, Proceedings. pp. 275–291. Springer (2001). https://doi.org/10.1007/3-540-44647-8_17
6. Boneh, D., Dagdelen, Ö., Fischlin, M., Lehmann, A., Schaffner, C., Zhandry, M.: Random oracles in a quantum world. In: Advances in Cryptology - ASIACRYPT 2011 - 17th International Conference on the Theory and Application of Cryptology and Information Security, Seoul, South Korea, December 4-8, 2011. Proceedings. pp. 41–69. Springer (2011). https://doi.org/10.1007/978-3-642-25385-0_3
7. Boyko, V.: On the security properties of OAEP as an all-or-nothing transform. In: Advances in Cryptology - CRYPTO '99, 19th Annual International Cryptology Conference, Proceedings. pp. 503–518. Springer (1999). https://doi.org/10.1007/3-540-48405-1_32
8. Chiesa, A., Manohar, P., Spooner, N.: Succinct arguments in the quantum random oracle model. In: Theory of Cryptography - 17th International Conference, TCC 2019, Proceedings, Part II. pp. 1–29. Springer (2019). https://doi.org/10.1007/978-3-030-36033-7_1
9. Chung, K., Fehr, S., Huang, Y., Liao, T.: On the compressed-oracle technique, and post-quantum security of proofs of sequential work. In: Advances in Cryptology - EUROCRYPT 2021 - 40th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Proceedings, Part II. pp. 598–629. Springer (2021). https://doi.org/10.1007/978-3-030-77886-6_21
10. Czakowski, J., Majenz, C., Schaffner, C., Zur, S.: Quantum lazy sampling and game-playing proofs for quantum indistinguishability. IACR Cryptol. ePrint Arch. p. 428 (2019), <https://eprint.iacr.org/2019/428>
11. Dodis, Y., Fiore, D.: Interactive encryption and message authentication. In: Security and Cryptography for Networks - 9th International Conference, SCN 2014, Proceedings. vol. 8642, pp. 494–513. Springer (2014), https://doi.org/10.1007/978-3-319-10879-7_28

12. Don, J., Fehr, S., Majenz, C., Schaffner, C.: Online-extractability in the quantum random-oracle model. In: *Advances in Cryptology - EUROCRYPT 2022 - 41st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Proceedings, Part III*. pp. 677–706. Springer (2022). https://doi.org/10.1007/978-3-031-07082-2_24
13. Ebrahimi, E.: Post-quantum security of plain OAEP transform. In: *Public-Key Cryptography - PKC 2022 - 25th IACR International Conference on Practice and Theory of Public-Key Cryptography, Proceedings, Part I. Lecture Notes in Computer Science*, vol. 13177, pp. 34–51. Springer (2022). https://doi.org/10.1007/978-3-030-97121-2_2
14. Fujisaki, E., Okamoto, T.: Secure integration of asymmetric and symmetric encryption schemes. In: *Advances in Cryptology - CRYPTO '99, 19th Annual International Cryptology Conference, Proceedings. Lecture Notes in Computer Science*, vol. 1666, pp. 537–554. Springer (1999). https://doi.org/10.1007/3-540-48405-1_34
15. Fujisaki, E., Okamoto, T.: Secure integration of asymmetric and symmetric encryption schemes. *J. Cryptol.* **26**(1), 80–101 (2013). <https://doi.org/10.1007/S00145-011-9114-1>
16. Ge, J., Shan, T., Xue, R.: Tighter qcca-secure key encapsulation mechanism with explicit rejection in the quantum random oracle model. In: *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Proceedings, Part V*. pp. 292–324. Springer (2023). https://doi.org/10.1007/978-3-031-38554-4_10
17. Grilo, A.B., Hövelmanns, K., Hülsing, A., Majenz, C.: Tight adaptive reprogramming in the QROM. In: *Advances in Cryptology - ASIACRYPT 2021 - 27th International Conference on the Theory and Application of Cryptology and Information Security, Proceedings, Part I*. pp. 637–667. Springer (2021). https://doi.org/10.1007/978-3-030-92062-3_22
18. Hashimoto, K., Katsumata, S., Wiggers, T.: Bundled authenticated key exchange: A concrete treatment of (post-quantum) signal’s handshake protocol. *IACR Cryptol. ePrint Arch.* p. 040 (2025), <https://eprint.iacr.org/2025/040>
19. Hosoyamada, A., Iwata, T.: 4-round luby-rackoff construction is a qprp. In: *Advances in Cryptology - ASIACRYPT 2019 - 25th International Conference on the Theory and Application of Cryptology and Information Security, Proceedings, Part I*. pp. 145–174. Springer (2019). https://doi.org/10.1007/978-3-030-34578-5_6
20. Hövelmanns, K., Hülsing, A., Majenz, C.: Failing gracefully: Decryption failures and the fujisaki-okamoto transform. In: *Advances in Cryptology - ASIACRYPT 2022 - 28th International Conference on the Theory and Application of Cryptology and Information Security, Proceedings, Part IV. Lecture Notes in Computer Science*, vol. 13794, pp. 414–443. Springer (2022). https://doi.org/10.1007/978-3-031-22972-5_15
21. Huguenin-Dumittan, L., Vaudenay, S.: On ind-qcca security in the ROM and its applications - CPA security is sufficient for TLS 1.3. In: *Advances in Cryptology - EUROCRYPT 2022 - 41st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Proceedings, Part III*. vol. 13277, pp. 613–642. Springer (2022), https://doi.org/10.1007/978-3-031-07082-2_22
22. Jaeger, J.: Nonadaptive one-way to hiding implies adaptive quantum reprogramming. To appear in *Proceedings of ASIACRYPT 2025*. *IACR Cryptol. ePrint Arch.* p. 797 (2024), <https://eprint.iacr.org/2024/797>

23. Jiang, H., Ma, Z., Zhang, Z.: Post-quantum security of key encapsulation mechanism against CCA attacks with a single decapsulation query. In: Advances in Cryptology - ASIACRYPT 2023 - 29th International Conference on the Theory and Application of Cryptology and Information Security, Proceedings, Part IV. Lecture Notes in Computer Science, vol. 14441, pp. 434–468. Springer (2023), https://doi.org/10.1007/978-981-99-8730-6_14
24. Jiang, H., Zhang, Z., Chen, L., Wang, H., Ma, Z.: Ind-cca-secure key encapsulation mechanism in the quantum random oracle model, revisited. In: Advances in Cryptology - CRYPTO 2018 - 38th Annual International Cryptology Conference, Proceedings, Part III. pp. 96–125. Springer (2018). https://doi.org/10.1007/978-3-319-96878-0_4
25. Jost, D., Maurer, U., Mularczyk, M.: Efficient ratcheting: Almost-optimal guarantees for secure messaging. In: Advances in Cryptology - EUROCRYPT 2019 - 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Proceedings, Part I. vol. 11476, pp. 159–188. Springer (2019), https://doi.org/10.1007/978-3-030-17653-2_6
26. Krämer, J., Struck, P.: On quantum ciphertext indistinguishability, recoverability, and OAEP. In: Post-Quantum Cryptography - 13th International Workshop, PQCrypto 2022, Proceedings. pp. 286–306. Springer (2022). https://doi.org/10.1007/978-3-031-17234-2_14
27. Kuchta, V., Sakzad, A., Stehlé, D., Steinfeld, R., Sun, S.: Measure-rewind-measure: Tighter quantum random oracle model proofs for one-way to hiding and CCA security. In: Advances in Cryptology - EUROCRYPT 2020 - 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Proceedings, Part III. pp. 703–728. Springer (2020). https://doi.org/10.1007/978-3-030-45727-3_24
28. Nielsen, M.A., Chuang, I.L.: Quantum Computation and Quantum Information (10th Anniversary edition). Cambridge University Press (2016)
29. NIST: National institute for standards and technology. post-quantum cryptography standardization process. <https://csrc.nist.gov/Projects/post-quantum-cryptography/post-quantum-cryptography-standardization> (2017)
30. NIST: National institute for standards and technology. post quantum crypto project. selected algorithms. <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022> (2022)
31. NTRU: A submission to the nist post-quantum standardization effort. <https://ntru.org/> (2018)
32. Pan, J., Zeng, R.: Selective opening security in the quantum random oracle model, revisited. In: Public-Key Cryptography - PKC 2024 - 27th IACR International Conference on Practice and Theory of Public-Key Cryptography, Proceedings, Part III. pp. 92–122. Springer (2024). https://doi.org/10.1007/978-3-031-57725-3_4
33. Phan, D.H., Pointcheval, D.: OAEP 3-round: A generic and secure asymmetric encryption padding. In: Advances in Cryptology - ASIACRYPT 2004, 10th International Conference on the Theory and Application of Cryptology and Information Security, Proceedings. pp. 63–77. Springer (2004). https://doi.org/10.1007/978-3-540-30539-2_5
34. Shan, T., Ge, J., Xue, R.: Qcca-secure generic transformations in the quantum random oracle model. In: Public-Key Cryptography - PKC 2023 - 26th IACR International Conference on Practice and Theory of Public-Key Cryptogra-

- phy, Proceedings, Part I. pp. 36–64. Springer (2023). https://doi.org/10.1007/978-3-031-31368-4_2
35. Shor, P.W.: Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Rev.* **41**(2), 303–332 (1999). <https://doi.org/10.1137/S0036144598347011>
 36. Shoup, V.: OAEP reconsidered. *J. Cryptol.* **15**(4), 223–249 (2002). <https://doi.org/10.1007/S00145-002-0133-9>
 37. Targhi, E.E., Unruh, D.: Post-quantum security of the fujisaki-okamoto and OAEP transforms. In: *Theory of Cryptography Conference*. pp. 192–216 (2016). https://doi.org/10.1007/978-3-662-53644-5_8
 38. Ueno, R., Xagawa, K., Tanaka, Y., Ito, A., Takahashi, J., Homma, N.: Curse of re-encryption: A generic power/em analysis on post-quantum kems. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* **2022**, 296–322 (2022). <https://doi.org/10.46586/TCHES.V2022.I1.296-322>
 39. Winter, A.J.: Coding theorem and strong converse for quantum channels. *IEEE Trans. Inf. Theory* **45**(7), 2481–2485 (1999). <https://doi.org/10.1109/18.796385>
 40. Yang, Z., Chen, Y., Luo, S.: Two-message key exchange with strong security from ideal lattices. In: *Topics in Cryptology - CT-RSA 2018 - The Cryptographers’ Track at the RSA Conference 2018, Proceedings*. vol. 10808, pp. 98–115. Springer (2018), https://doi.org/10.1007/978-3-319-76953-0_6
 41. Zhandry, M.: How to record quantum queries, and applications to quantum indistinguishability. In: *Advances in Cryptology - CRYPTO 2019 - 39th Annual International Cryptology Conference, Proceedings, Part II*. pp. 239–268. Springer (2019). https://doi.org/10.1007/978-3-030-26951-7_9

A A Potential Gap in the Security Proof in [13]

In this section, to make it more accurate and coherent, we adhere to the original symbols used in [13] as much as possible.

Game 7: This is identical to Game 6, except it measures all the queries to CStO_G with the projective measurements $\mathbb{M}_{r^*}$. If there is an 1-output measurement, it aborts and returns a random bit.

Game 7:

```

let  $(pk, sk) \leftarrow \text{Gen}(1^n)$ ,  $r^* \xleftarrow{\$} \{0, 1\}^{k_0}$ ,  $b \xleftarrow{\$} \{0, 1\}$ 
 $\mathbb{M}_{r^*} = \{\mathbb{P}_1 = |r^*\rangle\langle r^*|, \mathbb{P}_0 = \mathbb{I} - \mathbb{P}_1\}$ ,
run until there is an 1-output measurement with  $\mathbb{M}_{r^*}$ 
    let  $m_0, m_1 \leftarrow A^{\text{CStO}_H, \text{CStO}_G, \mathbb{U}_{\text{Dec}(4)}}(pk)$ 
    let  $s^* := m_b || 0^{k_1} \oplus G(r^*)$ ,  $t^* := r^* \oplus H(s^*)$ ,  $c^* := f(s^*, t^*)$ 
    let  $b' \leftarrow A^{\text{CStO}_H, \text{CStO}_G, \mathbb{U}_{\text{Dec}(4)}}(c^*)$ 
return  $[b = b']$ 

```

Fig. 10. Game 7 in the proof of [13].

In [13], the author first changed the simulation of the decryption oracle and finally constructed Game 6, which uses $\mathbb{U}_{\text{Dec}(4)}$ to simulate the decryption oracle, and the running of $\mathbb{U}_{\text{Dec}(4)}$ no longer required the secret key sk . In order to bound the probability $\Pr[1 \leftarrow \text{Game 6}]$, [13] designed Game 7 as shown in Fig. 10, and then proved the following formula using the gentle-measurement lemma [39]:

$$|\Pr[1 \leftarrow \text{Game 6}] - \Pr[1 \leftarrow \text{Game 7}]| \leq 2q_{G_1} \sqrt{2^{-k_0}} + 2q_{G_2} \sqrt{\epsilon}.$$

Here ϵ denotes the security bound of the underlying scheme and $q_{G_1} + q_{G_2} = q_G + 3q_{\text{dec}}$, where q_G (resp. q_{dec}) is the total number of queries to the random oracle G (resp. decryption oracle). In fact, the above bound corresponds to the highlighted $q \cdot \sqrt{\epsilon} + q \cdot \sqrt{2^{-k_0}}$ in our Table 1. Next, the following analysis is given in [13] to argue that $\Pr[1 \leftarrow \text{Game 7}] = 1/2$.

"it is clear that Game 7 returns 1 with the probability 1/2 because if one of the measurements returns 1, the output of the game is a random bit. If none of the measurements return 1, $G(r^)$ remains an uniformly random value for A and consequently $m_b || 0^{k_1} \oplus G(r^*)$ is an uniformly random value for A . So the probability that A guesses b is 1/2."*

However, the above analysis is not complete. Note that the random oracle G in Game 7 is simulated as a compressed oracle, which means that the $G(r^*)$ used to compute s^* is actually obtained by making the classical query r^* to the compressed oracle. Next, by the basic rules of compressed oracle, the simulation of $G(r^*)$ will process the database state and eventually insert the pair $(r^*, G(r^*))$ into the database state¹⁸. At this time the key point is that, even if we ensure

¹⁸ More precisely speaking, after this simulation, the database state is a superposition of $\text{StdDecomp}_{r^*} | \mathcal{D}_G \cup (r^*, G(r^*))$ for $\mathcal{D}_G(r^*) = \perp$.

that the adversary A does not query G by r^* to get $G(r^*)$ in Game 7, A can still obtain some information about $G(r^*)$ by querying the decryption oracle, because this oracle is simulated by $\mathbb{U}_{\text{Dec}(4)}$, it will search the whole database state in the computational basis, which may involve the $(r^*, G(r^*))$ pair inserted by the compressed oracle simulation of $G(r^*)$, thereby making A 's view not independent of $G(r^*)$. In the following we also give a simple example to support our observation.

- Let database $\mathfrak{D}_H$ satisfy that there exists an element s such that $\mathfrak{D}_H(s) \neq \perp$, $[s]_{k_1} = [s^*]_{k_1}$ and $s \neq s^*$. Then for this s , define $c := f(s, r^* \oplus \mathfrak{D}_H(s))$. Obviously, we have $[G(r^*) \oplus s]_{k_1} = 0^{k_1}$ and $c \neq c^*$ since $s^* := m_b || 0^{k_1} \oplus G(r^*)$ and $s \neq s^*$. Now by the definition of $\mathbb{U}_{\text{Dec}(4)}$ given in [13], the search it performed will transform the state $|c, y, \mathfrak{D}_G \cup (r^*, G(r^*)), \mathfrak{D}_H\rangle$ into $|c, y \oplus [G(r^*) \oplus s]^n, \mathfrak{D}_G \cup (r^*, G(r^*)), \mathfrak{D}_H\rangle$, where the first two parts denote the state on the query register of the decryption oracle. This actually means that the simulation of the decryption oracle on $|c, y, \mathfrak{D}_G \cup (r^*, G(r^*)), \mathfrak{D}_H\rangle$ will leak some information about $G(r^*)$ (i.e. the $[G(r^*) \oplus s]^n$) to the adversary A , and makes its view not independent of $G(r^*)$.

Hence the argument given in the above paragraph is actually not sufficient to demonstrate that $\Pr[1 \leftarrow \text{Game 7}] = 1/2$. In other words, When trying to transform $G(r^*)$ into a fresh random value, it is necessary not only to restrict A not to query G by r^* , but also to consider the potential detection of $G(r^*)$ indirectly derived from the simulation of the decryption oracle.

In addition, it seems non-trivial to fill the above gap in [13] since there are two main obstacles: first, the adversary A can make quantum queries to the decryption oracle since [13] considered the IND-qCCA security, which makes it hard to restrict A not to query the decryption oracle by some particular c ¹⁹; second, the powerful Semi-Classical One-Way to Hiding theorem [1] cannot be used here, as it only focuses on the function-like oracle (i.e., $|x, y\rangle \mapsto |x, y \oplus H(x)\rangle$), while the above gap involves the simulation of the compressed oracle and it cannot be rewritten in a function-like form by its definition given in [41].

In our paper, we did not fill the above gap in [13], but instead avoided it by using a new proof route. Roughly speaking, thanks to our step-by-step G and H reprogramming, after replacing $G(r^*)$ with a fresh random value g^* in our game **G7** (Fig. 8), we can directly use "perform $\text{StdDecomp}_r^* \circ U_{g^*}^*$ " to visualize the potential detection of the simulation of the decryption oracle, and then eventually bound it by using the Fixed Permutation One-Way to Hiding theorem [22] proposed very recently. Here we emphasize that our method will introduce an additional loss of q_D , resulting in the OAEP bound shown in our Table 1 appearing $q_D \cdot q \cdot \sqrt{\epsilon} + q_D \cdot q \cdot \sqrt{2^{-k_0}}$, which is a tolerable drawback when q_D is small. Nevertheless, either proposing concrete fixes to eliminate this drawback or directly filling the gap in [13], are still very interesting theoretical questions, and we leave them for further investigation.

¹⁹ For instance, the c we designed in the above example.

B Quantum Background

A quantum system (register) Q is a complex Hilbert space $\mathcal{H}_Q$ with an inner product $\langle \cdot | \cdot \rangle$, and notation like $|\cdot\rangle$ or $\langle \cdot |$ is called Dirac notation. A quantum state on Q is a vector on $\mathcal{H}_Q$. We denote $\mathcal{H}_Q = \mathbb{C}[X]$ if Q is defined over a finite set X . The orthonormal basis of $\mathbb{C}[X]$ is $\{|x\rangle\}_{x \in X}$, where the basis state $|x\rangle$ is labeled by the element x of X . We also refer to $\{|x\rangle\}_{x \in X}$ as the computational basis. The norm of a state $|\psi\rangle$ is defined as $\| |\psi\rangle \| := \sqrt{\langle \psi | \psi \rangle}$, and we say $|\psi\rangle$ is a unit state if $\| |\psi\rangle \| = 1$. The norm of an operator A is defined as $\|A\| := \max_{|\psi\rangle} \|A|\psi\rangle\|$, where max is over all unit states on Q . Sometimes we also write $|\psi\rangle$ as $|\psi\rangle_Q$ to emphasize that $|\psi\rangle$ is a quantum state on Q .

Composite quantum system. Given quantum systems Q_1 and Q_2 , we call the tensor product $Q_1 \otimes Q_2$ the composite quantum system, the state of which is written as $\sum_{x,y} \alpha_{x,y} |x\rangle \otimes |y\rangle \in Q_1 \otimes Q_2$ where $|x\rangle \in Q_1$, $|y\rangle \in Q_2$ are the basis states of Q_1 and Q_2 . We sometimes also abbreviate $|x\rangle \otimes |y\rangle$ into $|x, y\rangle$ for simplicity. If the composite quantum system $Q_1 \otimes Q_2$ is in state $|\psi\rangle$, we say that systems Q_1 and Q_2 are entangled if $|\psi\rangle$ can not be written as a product state $|\psi_1\rangle \otimes |\psi_2\rangle$ (here $|\psi_1\rangle \in Q_1$, $|\psi_2\rangle \in Q_2$).

A single qubit is a quantum system defined over $\{0, 1\}$. A single qubit in superposition is a linear combination vector $|b\rangle = \alpha|0\rangle + \beta|1\rangle$ of two computational basis states $|0\rangle$ and $|1\rangle$ with $(\alpha, \beta) \in \mathbb{C}^2$ and $|\alpha|^2 + |\beta|^2 = 1$, α, β are the probability amplitudes of $|b\rangle$. An n -qubit system is $Q^{\otimes n}$ where Q is a single-qubit system.

Evolution of quantum systems. A pure state $|\psi\rangle$ can be manipulated by performing a unitary operation U , and the resulting state can be denoted as $|\psi'\rangle = U|\psi\rangle$. For any unitary operation U , we have $UU^\dagger = I$, where $U^\dagger$ is the Hermitian transpose of U and I is the identity operator.

Projector on the quantum system. We introduce a special operation called projector. For the state $|\psi\rangle$ of a quantum system defined over a finite set X , a projector M_S ($S \subseteq X$) applies the operation $\sum_{y \in S} |y\rangle\langle y|$ to $|\psi\rangle$ to get the new state $\sum_{y \in S} |y\rangle\langle y|\psi\rangle$. We stress that any projector M is Hermitian (i.e., we have $M^\dagger = M$) and idempotent (i.e., we have $M^2 = M$).

Basic measurement. The state $|\psi\rangle$ in a quantum system can be measured in the computational basis. For example, suppose $|\psi\rangle = \sum_{x \in X} \alpha_x |x\rangle$ ($\sum_{x \in X} |\alpha_x|^2 = 1$) with computational basis $\{|x\rangle\}_{x \in X}$, if we measure $|\psi\rangle$ in basis $\{|x\rangle\}_{x \in X}$, we obtain the measurement result x with probability $|\langle x | \psi \rangle|^2 = |\alpha_x|^2$. Given that result x is obtained, the state $|\psi\rangle$ collapses to state $|x\rangle$. Moreover, we explain an important special class of measurements called projective measurement which is defined by a set of projectors $M_1, \dots, M_n$ where M_i are mutually orthogonal and $\sum_{i=1}^n M_i = I$. Upon measuring the state $|\psi\rangle$, the probability of obtaining i ($i \in \{1, \dots, n\}$) is $\langle \psi | M_i | \psi \rangle$ ($= \|M_i |\psi\rangle\|^2$). Given that result i is obtained, the state $|\psi\rangle$ collapses to state $M_i |\psi\rangle / \sqrt{\langle \psi | M_i | \psi \rangle}$.

Quantum oracle algorithm. A quantum oracle algorithm $\mathcal{A}$ is an algorithm that has quantum access to oracles, and that can perform a mix of classical and quantum unitary algorithms. In this paper, without loss of generality, we default that $\mathcal{A}$'s final output is a single bit $b \in \{0, 1\}$. We say that a quantum oracle algorithm $\mathcal{A}^H(z)$ making q oracle queries is a unitary quantum oracle algorithm if its entire execution can be described as

$$\mathbb{M}_{\mathcal{A}} \circ U_q \circ O_H \circ U_{q-1} \circ O_H \cdots U_2 \circ O_H \circ U_1 \circ O_H |\psi_z\rangle.$$

Here $|\psi_z\rangle$ is an initial pure state that may depend on input z , $U_1, \dots, U_q$ are the fixed unitary operations applied between oracle queries, and O_H is the unitary operation used to implement the oracle H . $\mathbb{M}_{\mathcal{A}} := \{M_0, M_1\}$ is the final binary projective measurement performed by $\mathcal{A}$ on its quantum register, and the measurement result of $\mathbb{M}_{\mathcal{A}}$ (0 or 1) is the final output of $\mathcal{A}$. Next, we introduce the following well-known fact about the quantum oracle algorithm. This fact implies that we can directly assume the quantum oracle algorithms to be unitary.

Fact 1. *Any quantum oracle algorithm can be transformed into a unitary quantum oracle algorithm with constant factor computational overhead and the same query number.*

Representation of the special symbol $\perp$. In this paper, we sometimes use a special symbol $\perp$ to expand a finite set $\{0, 1\}^n$. Roughly speaking, the reason is that when we define a particular unitary operation, we need " $\perp$ " to denote "not defined (yet)" or "computational failure".

As for the detailed representation of $\{0, 1\}^n \cup \{\perp\}$, we use the extension method used in [9, 19]. That is to say, we use a classical encoding function enc that $enc(\perp) := 1||0^n \in \{0, 1\}^{n+1}$ and $enc(x) := 0||x \in \{0, 1\}^{n+1}$ for any $x \in \{0, 1\}^n$. Then the set $\{0, 1\}^n \cup \{\perp\}$ can be embedded into the set $\{0, 1\}^{n+1}$. Under this representation, the binary operation $x \oplus y$ for $x, y \in \{0, 1\}^n \cup \{\perp\}$ actually refers to $enc(x) \oplus enc(y)$, and the quantum register defined over the set $\{0, 1\}^n \cup \{\perp\}$ actually refers to the register defined over the set $\{0, 1\}^{n+1}$.

Here we also give a more detailed example. Let c_1, c_2 be two ciphertexts with $\text{Dec}(sk, c_1) = m \in \{0, 1\}^n$ and $\text{Dec}(sk, c_2) = \perp$. So when making one quantum query to the decryption oracle using the uniform superposition of c_1 and c_2 , the adversary can prepare the query state as $1/\sqrt{2}|c_1\rangle_I|0||y\rangle_O + 1/\sqrt{2}|c_2\rangle_I|0||y\rangle_O$, where I/O is the input/output register and y is a fixed value in $\{0, 1\}^n$. In fact, the 0 padded in front of y denotes that the current value of the output register is exactly y , not a value like $y \oplus \perp$. After the decryption query, the adversary will get $1/\sqrt{2}|c_1\rangle_I|0||y \oplus m\rangle_O + 1/\sqrt{2}|c_2\rangle_I|1||y\rangle_O$ since we denote $\perp$ by $1||0^n$. Now the adversary "knows"²⁰ that c_2 failed to decrypt, because it gets the output $1||y$ and finds that this value is actually $y \oplus \perp$.

²⁰ In fact, an adversary needs to perform a measurement to know something, so here is an intuitive explanation, but this does not lose generality.

C Cryptographic Primitives and Security Definitions

Definition 4 (Public key encryption). A public key encryption (PKE) scheme $P = (\text{Gen}, \text{Enc}, \text{Dec})$ consists of three polynomial-time algorithms (in the security parameter λ) with a finite message space $\mathcal{M}$ such that

1. **Gen**, the key generation algorithm, is a probabilistic algorithm that on input 1^λ outputs a public/secret key pair (pk, sk) .
2. **Enc**, the encryption algorithm, on input pk and a message $m \in \mathcal{M}$, it chooses $r \leftarrow_{\$} \mathcal{R}$ ($\mathcal{R}$ is the randomness space), computes $c = \text{Enc}_{pk}(m; r) \in \mathcal{C}$ ($\mathcal{C}$ is the ciphertext space) and outputs the ciphertext c . If **Enc** does not use randomness r to compute c , **Enc** is a deterministic algorithm and outputs $c = \text{Enc}_{pk}(m)$.
3. **Dec**, the decryption algorithm, is a deterministic algorithm that on input ciphertext $c \in \mathcal{C}$ outputs a message $m = \text{Dec}_{sk}(c)$, or a special symbol $\perp \notin \mathcal{M}$ to indicate that c is not a valid ciphertext.

Definition 5 (IND-qCCA security for PKE scheme). For any polynomial-time quantum adversary $\mathcal{A}$, we define its IND-qCCA advantage against the PKE scheme $P = (\text{Gen}, \text{Enc}, \text{Dec})$ as follows:

$$\text{Adv}_{P, \mathcal{A}}^{\text{IND-qCCA}} := \left| \Pr[1 \leftarrow \mathbf{G}_{P, \mathcal{A}}^{\text{IND-qCCA}}] - \frac{1}{2} \right|.$$

Here game $\mathbf{G}_{P, \mathcal{A}}^{\text{IND-qCCA}}$ is shown in Fig. 11. We say that scheme P is IND-qCCA secure if for any polynomial-time quantum adversary $\mathcal{A}$, the $\text{Adv}_{P, \mathcal{A}}^{\text{IND-qCCA}}$ is negligible.

$\mathbf{G}_{P, \mathcal{A}}^{\text{IND-qCCA}}$	$\text{qDec}(c, z\rangle)$
$(pk, sk) \leftarrow \text{Gen}$	if c^* has not been defined
$(m_0^*, m_1^*) \leftarrow \mathcal{A}^{\text{qDec}}(pk)$	return $ c, z \oplus \text{Dec}_{sk}(c)\rangle$
$b \leftarrow_{\$} \{0, 1\}$	else return $ c, z \oplus f_{c^*}(c)\rangle$
$c^* = \text{Enc}_{pk}(m_b^*)$	$f_{c^*}(c)$
$b' \leftarrow \mathcal{A}^{\text{qDec}}(pk, c^*)$	if $c = c^*$
return $\llbracket b' = b \rrbracket$	return $\perp$
	else return $\text{Dec}_{sk}(c)$

Fig. 11. Game $\mathbf{G}_{P, \mathcal{A}}^{\text{IND-qCCA}}$.

We also define IND-qCCA security of PKE schemes in the QROM, where the adversary $\mathcal{A}$ can make quantum queries to random oracles.

Definition 6 (Trapdoor injective function). A trapdoor injective function $\text{TDI} = (\text{Gen}, f, f^{-1})$ consists of one polynomial-time algorithm and two functions (in the security parameter λ) such that:

- Gen is the key generation algorithm, which generates a public/secret key pair (pk, sk) on input 1^λ .
- $f_{\text{pk}}(\cdot)$ is an injective function with domain/range space $\mathcal{X}/\mathcal{Y}$ for all pk .
- $f_{\text{sk}}^{-1}(\cdot)$ is the inverse function of $f_{\text{pk}}(\cdot)$. That is to say, for any $y \in \mathcal{Y}$,

$$f_{\text{sk}}^{-1}(y) = \begin{cases} x, & \text{if } \exists x \in \mathcal{X} \text{ s.t. } f_{\text{pk}}(x) = y, \\ \perp, & \text{otherwise.} \end{cases}$$

The above definition of f_{sk}^{-1} introduces no ambiguity since f is an injective function.

Definition 7 (Quantum partial domain one-wayness for trapdoor injective function). For any polynomial-time quantum adversary $\mathcal{A}$, we define its PD-OW advantage against the trapdoor injective function $\text{TDI} = (\text{Gen}, f, f^{-1})$ as follows:

$$\text{Adv}_{\text{TDI}, \mathcal{A}}^{\text{PD-OW}} := \Pr[1 \leftarrow \mathbf{G}_{\text{TDI}, \mathcal{A}}^{\text{PD-OW}}].$$

Here game $\mathbf{G}_{\text{TDI}, \mathcal{A}}^{\text{PD-OW}}$ is shown in Fig. 12. We say that TDI is PD-OW secure if for any polynomial-time quantum adversary $\mathcal{A}$, $\text{Adv}_{\text{TDI}, \mathcal{A}}^{\text{PD-OW}}$ is negligible.

$\mathbf{G}_{\text{TDI}, \mathcal{A}}^{\text{PD-OW}}$
1 : $(\text{pk}, \text{sk}) \leftarrow \text{Gen}$
2 : $(s^*, t^*) \leftarrow_{\mathcal{S}} \mathcal{X}$
3 : $s \leftarrow \mathcal{A}(\text{pk}, f_{\text{pk}}(s^*, t^*))$
4 : return $\llbracket s = s^* \rrbracket$

Fig. 12. Game $\mathbf{G}_{\text{TDI}, \mathcal{A}}^{\text{PD-OW}}$.

D Missing Proofs in Section 4

Lemma 9 (Measurement Prob v.s. Norm Distance [1, Lemma 3, 4]).

Let $|\Phi_1\rangle, |\Phi_2\rangle$ be two pure unit states. Let $\mathbb{M} = \{\mathbb{M}_0, \mathbb{M}_1\}$ be a binary projective measurement. Let P_i be the probability that $\mathbb{M}$ returns 1 when measuring $|\Phi_i\rangle$ ($i \in \{1, 2\}$). Then we have $|P_1 - P_2| \leq \| |\Phi_1\rangle - |\Phi_2\rangle \|$.

Lemma 10 ([16, Corollary 2]). For any quantum states $|\psi_1\rangle, \dots, |\psi_q\rangle$, we have $\|\sum_{i=1}^q |\psi_i\rangle\|^2 \leq q \cdot \sum_{i=1}^q \| |\psi_i\rangle \|^2$.

D.1 Proof of Lemma 1

Let qDec_1 denote the oracle that simulated by U_{qDec}^1 (resp. $U_{\text{qDec}}^{1,*}$) before (resp. after) c^* is defined. Then, the running process of game $\mathbf{G}_1$ under a fixed $(\text{pk}, \text{sk}, \text{H})$ pair can be rewritten as the following artificial quantum oracle algorithm $\mathcal{B}^{\text{qDec}_1}$:

- After receiving $(\text{pk}, \text{sk}, \text{H})$, $\mathcal{B}$ runs the adversary $\mathcal{A}$ in game $\mathbf{G}_1$. When $\mathcal{A}$ queries oracle G or H , $\mathcal{B}$ uses the method of game $\mathbf{G}_1$ to simulate. When $\mathcal{A}$ queries oracle qDec , $\mathcal{B}$ replies by querying its oracle qDec_1 . $\mathcal{B}$ finally gets b' from $\mathcal{A}$ and outputs $\llbracket b = b' \rrbracket$.

Next, if we change the oracle qDec_1 into qDec_2 , which is the same as qDec_1 except that the U_{qDec}^1 and $U_{\text{qDec}}^{1,*}$ is replaced by U_{qDec}^2 and $U_{\text{qDec}}^{2,*}$, respectively, the resulting quantum oracle algorithm $\mathcal{B}^{\text{qDec}_2}(\text{pk}, \text{sk}, \text{H})$ will run game $\mathbf{G}_2$. Hence,

$$\begin{aligned} \Pr[1 \leftarrow \mathcal{B}^{\text{qDec}_1}(\text{pk}, \text{sk}, \text{H})] &= \Pr[1 \leftarrow \mathbf{G}_1 : (\text{pk}, \text{sk}, \text{H})], \\ \Pr[1 \leftarrow \mathcal{B}^{\text{qDec}_2}(\text{pk}, \text{sk}, \text{H})] &= \Pr[1 \leftarrow \mathbf{G}_2 : (\text{pk}, \text{sk}, \text{H})]. \end{aligned} \quad (28)$$

Here $\Pr[1 \leftarrow \mathbf{G}_i : (\text{pk}, \text{sk}, \text{H})]$ denotes the probability of game $\mathbf{G}_i$ ($i \in [2]$) outputs 1 under a fixed $(\text{pk}, \text{sk}, \text{H})$ pair.

As introduced in Supplementary Material B, any quantum oracle algorithm can be transformed into a unitary one. Thus, for the $\mathcal{B}^{\mathcal{O}}(\text{pk}, \text{sk}, \text{H})$, we can construct its unitary variant $\mathcal{B}_{\text{U}}^{\mathcal{O}}(\text{pk}, \text{sk}, \text{H})$ that acts on the registers $\text{Z}, \text{I}, \text{O}$ and $\text{D}_{q_{\text{G}}}$, where I/O is the input/output register of the oracle $\mathcal{O}$, $\text{D}_{q_{\text{G}}}$ is the database register used to reply $\mathcal{A}$'s G queries, and Z denotes some other ancillary registers used during the execution of $\mathcal{B}_{\text{U}}^{\mathcal{O}}(\text{pk}, \text{sk}, \text{H})$. Now the joint state of $\mathcal{B}_{\text{U}}^{\text{qDec}_1}(\text{pk}, \text{sk}, \text{H})$ and $\mathcal{B}_{\text{U}}^{\text{qDec}_2}(\text{pk}, \text{sk}, \text{H})$ just before the final binary projective measurement $\mathbb{M} := \{\text{M}_0, \text{M}_1\}$ can be written as:

$$\begin{aligned} \mathcal{B}_{\text{U}}^{\text{qDec}_1} : |\Psi_1\rangle|0\rangle_{\text{ST}} &= U_{q_{\text{D}}} \circ U_{\text{qDec}}^{1,*} \circ \dots \circ U_1 \circ U_{\text{qDec}}^1 |\psi\rangle|0\rangle_{\text{ST}}, \\ \mathcal{B}_{\text{U}}^{\text{qDec}_2} : |\Psi_2\rangle|0\rangle_{\text{ST}} &= U_{q_{\text{D}}} \circ U_{\text{qDec}}^{2,*} \circ \dots \circ U_1 \circ U_{\text{qDec}}^2 |\psi\rangle|0\rangle_{\text{ST}}. \end{aligned} \quad (29)$$

Here $|\psi\rangle$ is the initial state on the registers $\text{Z}, \text{I}, \text{O}$ and $\text{D}_{q_{\text{G}}}$, and $U_1, \dots, U_{q_{\text{D}}}$ are unitary operations acting on these registers between the $\text{qDec}_1/\text{qDec}_2$ queries. Obviously, we have

$$\begin{aligned} \Pr[1 \leftarrow \mathcal{B}^{\text{qDec}_1}(\text{pk}, \text{sk}, \text{H})] &= \Pr[1 \leftarrow \mathcal{B}_{\text{U}}^{\text{qDec}_1}(\text{pk}, \text{sk}, \text{H})] = \|\text{M}_1|\Psi_1\rangle|0\rangle_{\text{ST}}\|^2, \\ \Pr[1 \leftarrow \mathcal{B}^{\text{qDec}_2}(\text{pk}, \text{sk}, \text{H})] &= \Pr[1 \leftarrow \mathcal{B}_{\text{U}}^{\text{qDec}_2}(\text{pk}, \text{sk}, \text{H})] = \|\text{M}_1|\Psi_2\rangle|0\rangle_{\text{ST}}\|^2. \end{aligned} \quad (30)$$

As proved in Supplementary Material D.2, $\|\Psi_1\rangle|0\rangle_{\text{ST}} - \Psi_2\rangle|0\rangle_{\text{ST}}\| \leq 8q_{\text{D}} \cdot \sqrt{2^{-k_1}}$. Then by using Lemma 9 and Eq. (30), we have

$$|\Pr[1 \leftarrow \mathcal{B}^{\text{qDec}_1}(\text{pk}, \text{sk}, \text{H})] - \Pr[1 \leftarrow \mathcal{B}^{\text{qDec}_2}(\text{pk}, \text{sk}, \text{H})]| \leq 8q_{\text{D}} \cdot \sqrt{2^{-k_1}}.$$

By combining the above equation with Eq. (28) and averaging the $(\text{pk}, \text{sk}, \text{H})$, we finally get $|\Pr[1 \leftarrow \mathbf{G}_1] - \Pr[1 \leftarrow \mathbf{G}_2]| \leq 8q_{\text{D}} \cdot \sqrt{2^{-k_1}}$.

D.2 Bound on $\| |\Psi_1\rangle|0\rangle_{\text{ST}} - |\Psi_2\rangle|0\rangle_{\text{ST}} \|$

In the running process of $\mathcal{B}_U^{\text{qDec}_1}$ and $\mathcal{B}_U^{\text{qDec}_2}$, the compressed standard oracle is implemented based on D_{q_G} . Recall that the database state $|D_G\rangle$ on the database register D_{q_G} must satisfy $n(D_G) \leq q_G$, and the simulation of the decryption query²¹ also needs to perform two compressed standard oracle queries. Hence, we can conclude that the database state $|D_G\rangle$ just before one decryption query must satisfy $n(D_G) \leq q_G - 2$. Based on this fact, we denote the unit joint state on the registers Z, I, O, D_{q_G} just before performing one decryption query as:

$$|\Phi\rangle := \sum_{\substack{z \in \{0,1\}^*, c \in \mathcal{C}, y \in \{0,1\}^{n+k_1} \cup \{\perp\}, D_G \in \mathbf{D}_{q_G} \\ n(D_G) \leq q_G - 2}} \alpha_{y,D_G}^{z,c} |z\rangle_Z |c, y\rangle_{IO} |D_G\rangle_{D_{q_G}}.$$

Here $\mathcal{C}$ is the ciphertext space and set $\mathbf{D}_{q_G}$ follows Definition 2. In the following proof, for simplicity, we omit the set notations such as " $\in \{0,1\}^*$ " used in the sum of the above equation, and abbreviate $|z\rangle_Z |c, y\rangle_{IO} |D_G\rangle_{D_{q_G}}$ into $|z, c, y, D_G\rangle$.

In order to compute the bound separately, we divide $|\Phi\rangle$ into four complete and mutually orthogonal parts:

$$|\Phi\rangle = |\Phi_1\rangle + |\Phi_2\rangle + |\Phi_3\rangle + |\Phi_4\rangle,$$

where

$$\begin{aligned} |\Phi_1\rangle &:= \sum_{\substack{z, c, y, D_G, n(D_G) \leq q_G - 2 \\ f_{\text{sk}}^{-1}(c) = \perp}} \beta_{y,D_G}^{z,c} |z, c, y, D_G\rangle, \\ |\Phi_2\rangle &:= \sum_{\substack{z, c, y, D_G, n(D_G) \leq q_G - 2 \\ f_{\text{sk}}^{-1}(c) = (s, t) \neq \perp \\ r := t \oplus H(s), D_G(r) = \perp}} \beta_{y,D_G}^{z,c} |z, c, y, D_G\rangle, \\ |\Phi_3\rangle &:= \sum_{\substack{z, c, y, D_G, n(D_G) \leq q_G - 3 \\ f_{\text{sk}}^{-1}(c) = (s, t) \neq \perp \\ r := t \oplus H(s), D_G(r) = \perp \\ u \in \{0,1\}^{n+k_1}, u \neq 0^{n+k_1}}} \frac{\beta_{y,D_G,u}^{z,c}}{\sqrt{2^{n+k_1}}} \sum_{y_1 \in \{0,1\}^{n+k_1}} (-1)^{y_1 \cdot u} |z, c, y, D_G \cup (r, y_1)\rangle, \\ |\Phi_4\rangle &:= \sum_{\substack{z, c, y, D_G, n(D_G) \leq q_G - 3 \\ f_{\text{sk}}^{-1}(c) = (s, t) \neq \perp \\ r := t \oplus H(s), D_G(r) = \perp \\ u = 0^{n+k_1}}} \frac{\beta_{y,D_G,u}^{z,c}}{\sqrt{2^{n+k_1}}} \sum_{y_1 \in \{0,1\}^{n+k_1}} |z, c, y, D_G \cup (r, y_1)\rangle. \end{aligned}$$

We first compute the upper bound of $\| (U_{\text{qDec}}^1 - U_{\text{qDec}}^2) |\Phi\rangle |0\rangle_{\text{ST}} \|$, which corresponds to the case that the challenge ciphertext c^* has not been computed yet. For the fixed tuple (z, c, y, D_G) with $f_{\text{sk}}^{-1}(c) = (s, t) \neq \perp$, and $D_G(r) = \perp$

²¹ In our setting, this query is simulated by using $U_{\text{qDec}}^1 / U_{\text{qDec}}^{1,*}$ or $U_{\text{qDec}}^2 / U_{\text{qDec}}^{2,*}$.

($r := t \oplus \mathbf{H}(s)$), we define the following states that will be used later. Here $w \in \{0, 1\}^{n+k_1}$ and $\nu \in \{0, 1\}^n \cup \{\perp\}$.

$$\begin{aligned}
|\gamma_1[w, \nu]\rangle_{y, D_G}^{z, c} &:= \sum_{[y_1]_{k_1}=[s]_{k_1}} (-1)^{y_1 \cdot w} |z, c, y \oplus \nu, D_G \cup (r, y_1)\rangle, \\
|\gamma_2[w, \nu]\rangle_{y, D_G}^{z, c} &:= \sum_{[y_1]_{k_1}=[s]_{k_1}} (-1)^{y_1 \cdot w} |z, c, y \oplus \nu, D_G\rangle, \\
|\gamma_3[w, \nu]\rangle_{y, D_G}^{z, c} &:= \sum_{[y_1]_{k_1}=[s]_{k_1}} \sum_{y_2 \in \{0, 1\}^{n+k_1}} \frac{(-1)^{y_1 \cdot w}}{\sqrt{2^{n+k_1}}} |z, c, y \oplus \nu, D_G \cup (r, y_2)\rangle.
\end{aligned} \tag{31}$$

Let S_x be an abbreviation of the local decompression procedure StdDecomp_x . Then by the quantum simulation of the unitary operation U_{qDec}^1 (resp. U_{qDec}^2) given in Eq. (5) (resp. Eq. (7)), we can compute

$$\begin{aligned}
U_{\text{qDec}}^1 |\Phi_1\rangle |0\rangle_{\text{ST}} &= U_{\text{qDec}}^2 |\Phi_1\rangle |0\rangle_{\text{ST}} = \sum_{\substack{n(D_G) \leq q_G - 2 \\ f_{\text{sk}}^{-1}(c) = \perp}}^{z, c, y, D_G} \beta_{y, D_G}^{z, c} |z, c, y \oplus \perp, D_G\rangle |0\rangle_{\text{ST}}, \\
(U_{\text{qDec}}^1 - U_{\text{qDec}}^2) |\Phi_2\rangle |0\rangle_{\text{ST}} &= \sum_{\substack{n(D_G) \leq q_G - 2 \\ f_{\text{sk}}^{-1}(c) = (s, t) \neq \perp \\ r := t \oplus \mathbf{H}(s), D_G(r) = \perp}}^{z, c, y, D_G} \frac{\beta_{y, D_G}^{z, c}}{\sqrt{2^{n+k_1}}} S_r \left[\begin{array}{c} |\gamma_1[0^{n+k_1}, [s \oplus y_1]^n]\rangle \\ - |\gamma_1[0^{n+k_1}, \perp]\rangle \end{array} \right] |0\rangle_{\text{ST}}, \\
(U_{\text{qDec}}^1 - U_{\text{qDec}}^2) |\Phi_3\rangle |0\rangle_{\text{ST}} &= \sum_{\substack{n(D_G) \leq q_G - 3 \\ f_{\text{sk}}^{-1}(c) = (s, t) \neq \perp \\ r := t \oplus \mathbf{H}(s), D_G(r) = \perp \\ u \in \{0, 1\}^{n+k_1}, u \neq 0^{n+k_1}}}^{z, c, y, D_G} \frac{\beta_{D_G, u}^{z, c, y}}{2^{n+k_1}} \left[\begin{array}{c} |\gamma_2[u, [s \oplus y_1]^n]\rangle - |\gamma_2[u, \perp]\rangle \\ - |\gamma_3[u, [s \oplus y_1]^n]\rangle + |\gamma_3[u, \perp]\rangle \end{array} \right] |0\rangle_{\text{ST}}.
\end{aligned}$$

Next, we note that the state in the form $\sqrt{1/2^{n+k_1}} \sum_{y_1} |z, c, y, D_G \cup (r, y_1)\rangle$ cannot emerge if we use $U_{\text{qDec}}^1/U_{\text{qDec}}^{1,*}$ to simulate the decryption query, since the quantum simulation of $U_{\text{qDec}}^1/U_{\text{qDec}}^{1,*}$ as shown in Eq. (5) only queries the compressed standard oracle and does not disturb its implementation²². However, the state in that form might emerge if we use $U_{\text{qDec}}^2/U_{\text{qDec}}^{2,*}$ to simulate the decryption query, since the unitary operation Test_G used by them will disturb the implementation of the compressed standard oracle. Hence, in order to compute the difference between $U_{\text{qDec}}^1 |\Phi_4\rangle |0\rangle_{\text{ST}}$ and $U_{\text{qDec}}^2 |\Phi_4\rangle |0\rangle_{\text{ST}}$, for any z, c, y, D_G, r

²² In fact, the uniform superposition of database state like $\sqrt{1/2^{n+k_1}} \sum_{y_1} |D_G \cup (r, y_1)\rangle$ cannot emerge when we implement a compressed standard oracle without being disturbed. This property is also used in the proof of Lemma 5 in [41].

such that $D_G(r) = \perp$ and $n(D_G) \leq q_G - 1$, we add a complement of U_{qDec}^1 as:

$$\begin{aligned} U_{\text{qDec}}^1 & \frac{1}{\sqrt{2^{n+k_1}}} \sum_{y_1 \in \{0,1\}^{n+k_1}} |z, c, y, D_G \cup (r, y_1)\rangle |0\rangle_{\text{ST}} \\ & := \frac{1}{\sqrt{2^{n+k_1}}} \sum_{y_1 \in \{0,1\}^{n+k_1}} |z, c, y \oplus \perp, D_G \cup (r, y_1)\rangle |0\rangle_{\text{ST}}. \end{aligned}$$

Since the (uniform superposition of the database) state $\frac{1}{\sqrt{2^{n+k_1}}} \sum_{y_1 \in \{0,1\}^{n+k_1}} |z, c, y, D_G \cup (r, y_1)\rangle$ must be orthogonal with $|\Phi_1\rangle, |\Phi_2\rangle, |\Phi_3\rangle$ and applying U_{qDec}^1 on all $|\Phi_1\rangle, |\Phi_2\rangle, |\Phi_3\rangle$ cannot generate such uniform superposition of the database, we know that the above complement causes no ambiguity and is workable. With this complement, we can compute

$$\begin{aligned} & (U_{\text{qDec}}^1 - U_{\text{qDec}}^2) |\Phi_4\rangle |0\rangle_{\text{ST}} \\ & = \sum_{\substack{z,c,y,D_G \\ n(D_G) \leq q_G-3 \\ f_{\text{sk}}^{-1}(c)=(s,t) \neq \perp \\ r:=t \oplus H(s), D_G(r)=\perp \\ u=0^{n+k_1}}} \frac{\beta_{y,D_G,u}^{z,c}}{\sqrt{2^{n+k_1}}} \left[\frac{|\gamma_1[0^{n+k_1}, \perp]\rangle - |\gamma_1[0^{n+k_1}, [s \oplus y_1]^n]\rangle}{\sqrt{2^{n+k_1}}} \right] |0\rangle_{\text{ST}}. \end{aligned}$$

At this point, we have $\|(U_{\text{qDec}}^1 - U_{\text{qDec}}^2) |\Phi_1\rangle |0\rangle_{\text{ST}}\| = 0$, and we can compute

$$\begin{aligned} & \|(U_{\text{qDec}}^1 - U_{\text{qDec}}^2) |\Phi_2\rangle |0\rangle_{\text{ST}}\|^2 \\ & = \left\| \sum_{\substack{z,c,y,D_G \\ n(D_G) \leq q_G-2 \\ f_{\text{sk}}^{-1}(c)=(s,t) \neq \perp \\ r:=t \oplus H(s), D_G(r)=\perp}} \frac{\beta_{y,D_G}^{z,c}}{\sqrt{2^{n+k_1}}} S_r \left[|\gamma_1[0^{n+k_1}, [s \oplus y_1]^n]\rangle - |\gamma_1[0^{n+k_1}, \perp]\rangle \right] \right\|^2 \\ & \stackrel{(a)}{\leq} 2 \left\| \sum_{\substack{z,c,y,D_G \\ n(D_G) \leq q_G-2 \\ f_{\text{sk}}^{-1}(c)=(s,t) \neq \perp \\ r:=t \oplus H(s), D_G(r)=\perp}} \frac{\beta_{y,D_G}^{z,c}}{\sqrt{2^{n+k_1}}} S_r |\gamma_1[0^{n+k_1}, [s \oplus y_1]^n]\rangle \right\|^2 \\ & \quad + 2 \left\| \sum_{\substack{z,c,y,D_G \\ n(D_G) \leq q_G-2 \\ f_{\text{sk}}^{-1}(c)=(s,t) \neq \perp \\ r:=t \oplus H(s), D_G(r)=\perp}} \frac{\beta_{y,D_G}^{z,c}}{\sqrt{2^{n+k_1}}} S_r |\gamma_1[0^{n+k_1}, \perp]\rangle \right\|^2 \\ & \stackrel{(b)}{\leq} 2 \sum_{\substack{z,c,D_G \\ n(D_G) \leq q_G-2 \\ f_{\text{sk}}^{-1}(c)=(s,t) \neq \perp \\ r:=t \oplus H(s), D_G(r)=\perp}} \left\| \sum_y \frac{\beta_{y,D_G}^{z,c}}{\sqrt{2^{n+k_1}}} |\gamma_1[0^{n+k_1}, [s \oplus y_1]^n]\rangle \right\|^2 \end{aligned}$$

$$\begin{aligned}
& + 2 \sum_{\substack{n(D_G) \leq q_G - 2 \\ f_{sk}^{-1}(c) = (s,t) \neq \perp \\ r := t \oplus H(s), D_G(r) = \perp}}^{z,c,D_G} \left\| \sum_y \frac{\beta_{y,D_G}^{z,c}}{\sqrt{2^{n+k_1}}} |\gamma_1[0^{n+k_1}, \perp]\rangle \right\|^2 \\
& \stackrel{(c)}{=} 2 \sum_{\substack{n(D_G) \leq q_G - 2 \\ f_{sk}^{-1}(c) = (s,t) \neq \perp \\ r := t \oplus H(s), D_G(r) = \perp}}^{z,c,y,D_G} \sum_{[y_1]_{k_1} = [s]_{k_1}} \left\| \frac{\beta_{y,D_G}^{z,c}}{\sqrt{2^{n+k_1}}} |z, c, y \oplus [s \oplus y_1]^n, D_G \cup (r, y_1)\rangle \right\|^2 \\
& \quad + 2 \sum_{\substack{n(D_G) \leq q_G - 2 \\ f_{sk}^{-1}(c) = (s,t) \neq \perp \\ r := t \oplus H(s), D_G(r) = \perp}}^{z,c,y,D_G} \sum_{[y_1]_{k_1} = [s]_{k_1}} \left\| \frac{\beta_{y,D_G}^{z,c}}{\sqrt{2^{n+k_1}}} |z, c, y \oplus \perp, D_G \cup (r, y_1)\rangle \right\|^2 \\
& = 4 \sum_{\substack{n(D_G) \leq q_G - 2 \\ f_{sk}^{-1}(c) = (s,t) \neq \perp \\ r := t \oplus H(s), D_G(r) = \perp}}^{z,c,y,D_G} \frac{2^n |\beta_{y,D_G}^{z,c}|^2}{2^{n+k_1}} = 4 \cdot 2^{-k_1} \|\Phi_2\|^2.
\end{aligned} \tag{32}$$

Here (a) uses Lemma 10, (b) follows from the definition of $|\gamma_1[w, \nu]\rangle_{y,D_G}^{z,c}$ as shown in Eq. (31) and the fact that S_r is a unitary operation. (c) also follows from the definition of $|\gamma_1[w, \nu]\rangle_{y,D_G}^{z,c}$. Additionally, similar to the above computation process we can get:

$$\begin{aligned}
& \left\| (U_{qDec}^1 - U_{qDec}^2) |\Phi_4\rangle |0\rangle_{ST} \right\|^2 \\
& = \left\| \sum_{\substack{n(D_G) \leq q_G - 3 \\ f_{sk}^{-1}(c) = (s,t) \neq \perp \\ r := t \oplus H(s), D_G(r) = \perp, u = 0^{n+k_1}}}^{z,c,y,D_G} \frac{\beta_{y,D_G,u}^{z,c}}{\sqrt{2^{n+k_1}}} \left[|\gamma_1[0^{n+k_1}, \perp]\rangle - |\gamma_1[0^{n+k_1}, [s \oplus y_1]^n]\rangle \right] \right\|^2 \\
& \stackrel{(d)}{\leq} 2 \sum_{\substack{n(D_G) \leq q_G - 3 \\ f_{sk}^{-1}(c) = (s,t) \neq \perp \\ r := t \oplus H(s), D_G(r) = \perp, u = 0^{n+k_1}}}^{z,c,D_G} \left\| \sum_y \frac{\beta_{y,D_G,u}^{z,c}}{\sqrt{2^{n+k_1}}} |\gamma_1[0^{n+k_1}, \perp]\rangle \right\|^2 \\
& \quad + 2 \sum_{\substack{n(D_G) \leq q_G - 3 \\ f_{sk}^{-1}(c) = (s,t) \neq \perp \\ r := t \oplus H(s), D_G(r) = \perp, u = 0^{n+k_1}}}^{z,c,D_G} \left\| \sum_y \frac{\beta_{y,D_G,u}^{z,c}}{\sqrt{2^{n+k_1}}} |\gamma_1[0^{n+k_1}, [s \oplus y_1]^n]\rangle \right\|^2 \\
& \stackrel{(e)}{=} 2 \sum_{\substack{n(D_G) \leq q_G - 3 \\ f_{sk}^{-1}(c) = (s,t) \neq \perp \\ r := t \oplus H(s), D_G(r) = \perp, u = 0^{n+k_1}}}^{z,c,y,D_G} \sum_{[y_1]_{k_1} = [s]_{k_1}} \left\| \frac{\beta_{y,D_G,u}^{z,c}}{\sqrt{2^{n+k_1}}} |z, c, y \oplus \perp, D_G \cup (r, y_1)\rangle \right\|^2
\end{aligned}$$

$$\begin{aligned}
& + 2 \sum_{\substack{z,c,y,D_G \\ n(D_G) \leq q_G - 3 \\ f_{sk}^{-1}(c) = (s,t) \neq \perp \\ r := t \oplus H(s), D_G(r) = \perp \\ u = 0^{n+k_1}}} \sum_{[y_1]_{k_1} = [s]_{k_1}} \left\| \frac{\beta_{y,D_G,u}^{z,c}}{\sqrt{2^{n+k_1}}} |z, c, y \oplus [s \oplus y_1]^n, D_G \cup (r, y_1)\rangle \right\|^2 \\
& = 4 \sum_{\substack{z,c,y,D_G \\ n(D_G) \leq q_G - 3 \\ f_{sk}^{-1}(c) = (s,t) \neq \perp \\ r := t \oplus H(s), D_G(r) = \perp, u = 0^{n+k_1}}} \frac{2^n |\beta_{y,D_G,u}^{z,c}|^2}{2^{n+k_1}} = 4 \cdot 2^{-k_1} \|\Phi_4\|^2.
\end{aligned} \tag{33}$$

Here (d) uses Lemma 10 and the definition of $|\gamma_1[w, \nu]\rangle_{y,D_G}^{z,c}$ as shown in Eq. (31), (e) also follows from the definition of $|\gamma_1[w, \nu]\rangle_{y,D_G}^{z,c}$.

As for $\|(U_{qDec}^1 - U_{qDec}^2)|\Phi_3\rangle|0\rangle_{ST}\|$, we have

$$\begin{aligned}
& \|(U_{qDec}^1 - U_{qDec}^2)|\Phi_3\rangle|0\rangle_{ST}\|^2 \\
& = \left\| \sum_{\substack{z,c,y,D_G \\ n(D_G) \leq q_G - 3 \\ f_{sk}^{-1}(c) = (s,t) \neq \perp \\ r := t \oplus H(s), D_G(r) = \perp \\ u \in \{0,1\}^{n+k_1}, u \neq 0^{n+k_1}}} \frac{\beta_{y,D_G,u}^{z,c}}{2^{n+k_1}} \begin{bmatrix} |\gamma_2[u, [s \oplus y_1]^n]\rangle - |\gamma_2[u, \perp]\rangle \\ -|\gamma_3[u, [s \oplus y_1]^n]\rangle + |\gamma_3[u, \perp]\rangle \end{bmatrix} \right\|^2 \\
& \stackrel{(f)}{\leq} \frac{4}{2^{k_1}} \sum_{\substack{z,c,y,D_G \\ n(D_G) \leq q_G - 3 \\ f_{sk}^{-1}(c) = (s,t) \neq \perp \\ r := t \oplus H(s), D_G(r) = \perp \\ [y_1]_{k_1} = [s]_{k_1}}} \left\| \sum_{u \neq 0^{n+k_1}}^{u \in \{0,1\}^{n+k_1}} \frac{(-1)^{y_1 \cdot u} \beta_{y,D_G,u}^{z,c}}{\sqrt{2^{n+k_1}}} |z, c, y \oplus [s \oplus y_1]^n, D_G\rangle \right\|^2 \\
& + \frac{4}{2^{k_1}} \sum_{\substack{z,c,y,D_G \\ n(D_G) \leq q_G - 3 \\ f_{sk}^{-1}(c) = (s,t) \neq \perp \\ r := t \oplus H(s), D_G(r) = \perp \\ [y_1]_{k_1} = [s]_{k_1}}} \left\| \sum_{u \neq 0^{n+k_1}}^{u \in \{0,1\}^{n+k_1}} \frac{(-1)^{y_1 \cdot u} \beta_{D_G,u}^{z,c,y}}{\sqrt{2^{n+k_1}}} |z, c, y \oplus \perp, D_G\rangle \right\|^2 \\
& + \frac{4}{2^{k_1}} \sum_{\substack{z,c,y,D_G \\ n(D_G) \leq q_G - 3 \\ f_{sk}^{-1}(c) = (s,t) \neq \perp \\ r := t \oplus H(s), D_G(r) = \perp \\ [y_1]_{k_1} = [s]_{k_1} \\ y_2 \in \{0,1\}^{n+k_1}}} \left\| \sum_{u \neq 0^{n+k_1}}^{u \in \{0,1\}^{n+k_1}} \frac{(-1)^{y_1 \cdot u} \beta_{y,D_G,u}^{z,c}}{2^{n+k_1}} \left| z, c, y \oplus [s \oplus y_1]^n, D_G \cup (r, y_2) \right\rangle \right\|^2 \\
& + \frac{4}{2^{k_1}} \sum_{\substack{z,c,y,D_G \\ n(D_G) \leq q_G - 3 \\ f_{sk}^{-1}(c) = (s,t) \neq \perp \\ r := t \oplus H(s), D_G(r) = \perp \\ [y_1]_{k_1} = [s]_{k_1} \\ y_2 \in \{0,1\}^{n+k_1}}} \left\| \sum_{u \neq 0^{n+k_1}}^{u \in \{0,1\}^{n+k_1}} \frac{(-1)^{y_1 \cdot u} \beta_{y,D_G,u}^{z,c}}{2^{n+k_1}} |z, c, y \oplus \perp, D_G \cup (r, y_2)\rangle \right\|^2
\end{aligned}$$

$$\begin{aligned}
&= \frac{16}{2^{k_1}} \sum_{\substack{z,c,y,D_G \\ n(D_G) \leq q_G - 3 \\ f_{sk}^{-1}(c) = (s,t) \neq \perp \\ r := t \oplus H(s), D_G(r) = \perp}} \sum_{[y_1]_{k_1} = [s]_{k_1}} \left| \sum_{u \in \{0,1\}^{n+k_1}, u \neq 0^{n+k_1}} \frac{(-1)^{y_1 \cdot u} \beta_{y,D_G,u}^{z,c}}{\sqrt{2^{n+k_1}}} \right|^2 \\
&\stackrel{(g)}{\leq} 16 \cdot 2^{-k_1} \|\Phi_3\|^2.
\end{aligned} \tag{34}$$

Here (f) firstly uses Lemma 10 and then follows the definition of $|\gamma_2[w, \nu]\rangle_{y,D_G}^{z,c}$ and $|\gamma_3[w, \nu]\rangle_{y,D_G}^{z,c}$ as shown in Eq. (31), while (g) follows from the definition of $|\Phi_3\rangle$.

Thus, by combining Eq. (32), Eq. (33) and Eq. (34), we finally obtain

$$\begin{aligned}
\|(U_{qDec}^1 - U_{qDec}^2)|\Phi\rangle|0\rangle_{ST}\| &\stackrel{(h)}{\leq} \sum_{i=1}^4 \|(U_{qDec}^1 - U_{qDec}^2)|\Phi_i\rangle|0\rangle_{ST}\| \\
&\leq 2 \cdot \sqrt{2^{-k_1}} (\|\Phi_2\| + \|\Phi_4\|) + 4 \cdot \sqrt{2^{-k_1}} \|\Phi_3\| \\
&\stackrel{(i)}{\leq} 8 \cdot \sqrt{2^{-k_1}} \|\Phi\| = 8 \cdot \sqrt{2^{-k_1}}.
\end{aligned}$$

Here (h) uses the triangle inequality, and (i) comes from the fact that $\|\Phi_i\| \leq \|\Phi\|$ for $i \in [4]$. Recall that $U_{qDec}^{1,*} = U_{\perp} \circ P_{c^*} + U_{qDec}^1 \circ (I - P_{c^*})$ and $U_{qDec}^{2,*} = U_{\perp} \circ P_{c^*} + U_{qDec}^2 \circ (I - P_{c^*})$, thus it is easy to check that

$$\begin{aligned}
\|(U_{qDec}^{1,*} - U_{qDec}^{2,*})|\Phi\rangle|0\rangle_{ST}\| &= \|(U_{qDec}^1 - U_{qDec}^2) \circ (I - P_{c^*})|\Phi\rangle|0\rangle_{ST}\| \\
&\leq 8 \cdot \sqrt{2^{-k_1}} \|(I - P_{c^*})|\Phi\rangle\| \leq 8 \cdot \sqrt{2^{-k_1}}.
\end{aligned}$$

Now for any unit joint state $|\Phi\rangle$ on the registers Z, I, O and D_{q_G} just before one decryption query, we have

$$\max \left\{ \|(U_{qDec}^1 - U_{qDec}^2)|\Phi\rangle|0\rangle_{ST}\|, \|(U_{qDec}^{1,*} - U_{qDec}^{2,*})|\Phi\rangle|0\rangle_{ST}\| \right\} \leq 8 \cdot \sqrt{2^{-k_1}}.$$

Then by the hybrid argument and the definition of $|\Psi_1\rangle$ and $|\Psi_2\rangle$ given in Eq. (29), it is straightforward to obtain that $\| |\Psi_1\rangle|0\rangle_{ST} - |\Psi_2\rangle|0\rangle_{ST} \| \leq 8q_D \cdot \sqrt{2^{-k_1}}$.

D.3 Proof of Lemma 2

Similar to the proof of Lemma 1, the key point of this proof is to prove that the difference between the operations $U_{qDec}^3/U_{qDec}^{3,*}$ and $U_{qDec}^4/U_{qDec}^{4,*}$ can be upper bounded by some parameters. In more detail, for any unit joint state $|\Phi\rangle$ on the registers Z, I, O, D_{q_G} and D_{q_H} just before one decryption query, we need to bound the following value:

$$\max \left\{ \|(U_{qDec}^3 - U_{qDec}^4)|\Phi\rangle|0\rangle_{ST}\|, \|(U_{qDec}^{3,*} - U_{qDec}^{4,*})|\Phi\rangle|0\rangle_{ST}\| \right\}.$$

In the following proof, we first give an upper bound of $\|(U_{qDec}^3 - U_{qDec}^4)|\Phi\rangle|0\rangle_{ST}\|$.

Similar to Supplementary Material D.2, we divide $|\Phi\rangle$ into the following four complete and mutually orthogonal parts:

$$\begin{aligned}
|\Phi_1\rangle &:= \sum_{\substack{z,c,y,D_G,D_H \\ n(D_G) \leq q_G, n(D_H) \leq q_H-2 \\ f_{sk}^{-1}(c) = \perp}} \beta_{D_G,D_H}^{z,c,y} |z, c, y, D_G, D_H\rangle, \\
|\Phi_2\rangle &:= \sum_{\substack{z,c,y,D_G,D_H \\ n(D_G) \leq q_G, n(D_H) \leq q_H-2 \\ f_{sk}^{-1}(c) = (s,t) \neq \perp \\ D_H(s) = \perp}} \beta_{D_G,D_H}^{z,c,y} |z, c, y, D_G, D_H\rangle, \\
|\Phi_3\rangle &:= \sum_{\substack{z,c,y,D_G,D_H \\ n(D_G) \leq q_G, n(D_H) \leq q_H-3 \\ f_{sk}^{-1}(c) = (s,t) \neq \perp \\ D_H(s) = \perp \\ u \in \{0,1\}^{k_0}, u \neq 0^{k_0}}} \frac{\beta_{D_G,D_H}^{z,c,y,u}}{\sqrt{2^{k_0}}} \sum_{y_1 \in \{0,1\}^{k_0}} (-1)^{y_1 \cdot u} |z, c, y, D_G, D_H \cup (s, y_1)\rangle, \\
|\Phi_4\rangle &:= \sum_{\substack{z,c,y,D_G,D_H \\ n(D_G) \leq q_G, n(D_H) \leq q_H-3 \\ f_{sk}^{-1}(c) = (s,t) \neq \perp \\ D_H(s) = \perp, u = 0^{k_0}}} \frac{\beta_{D_G,D_H}^{z,c,y,u}}{\sqrt{2^{k_0}}} \sum_{y_1 \in \{0,1\}^{k_0}} |z, c, y, D_G, D_H \cup (s, y_1)\rangle,
\end{aligned}$$

and additionally introduce a complement of U_{qDec}^3 for any z, c, y, D_G, D_H, s such that $D_H(s) = \perp$ and $n(D_H) \leq q_H - 1$ as:

$$\begin{aligned}
U_{\text{qDec}}^3 \frac{1}{\sqrt{2^{k_0}}} \sum_{y_1 \in \{0,1\}^{k_0}} |z, c, y, D_G, D_H \cup (s, y_1)\rangle |0\rangle_{\text{ST}} \\
:= \frac{1}{\sqrt{2^{k_0}}} \sum_{y_1 \in \{0,1\}^{k_0}} |z, c, y \oplus \perp, D_G, D_H \cup (s, y_1)\rangle |0\rangle_{\text{ST}}.
\end{aligned}$$

For the fixed tuple (z, c, y, D_G, D_H) with $f_{sk}^{-1}(c) = (s, t) \neq \perp$ and $D_H(s) = \perp$, we define the following states that will be used later. Here $w \in \{0,1\}^{k_0}$, $\nu \in \{0,1\}^n \cup \{\perp\}$, and $L_{D_G}^t := \{y \in \{0,1\}^{k_0} : D_G(t \oplus y) \neq \perp\}$.

$$\begin{aligned}
|\gamma_1[w, \nu]\rangle_{D_G, D_H}^{z,c,y} &:= \sum_{\substack{y_1 \in L_{D_G}^t \\ [D_G(t \oplus y_1)]_{k_1} = [s]_{k_1}}} (-1)^{y_1 \cdot w} |z, c, y \oplus \nu, D_G, D_H \cup (s, y_1)\rangle, \\
|\gamma_2[w, \nu]\rangle_{D_G, D_H}^{z,c,y} &:= \sum_{\substack{y_1 \in L_{D_G}^t \\ [D_G(t \oplus y_1)]_{k_1} = [s]_{k_1}}} (-1)^{y_1 \cdot w} |z, c, y \oplus \nu, D_G, D_H\rangle, \\
|\gamma_3[w, \nu]\rangle_{D_G, D_H}^{z,c,y} &:= \sum_{\substack{y_1 \in L_{D_G}^t, y_2 \in \{0,1\}^{k_0} \\ [D_G(t \oplus y_1)]_{k_1} = [s]_{k_1}}} \frac{(-1)^{y_1 \cdot w}}{\sqrt{2^{k_0}}} |z, c, y \oplus \nu, D_G, D_H \cup (s, y_2)\rangle.
\end{aligned} \tag{35}$$

Let S_x be an abbreviation of the local decompression procedure StdDecomp_x . Then by the quantum simulation of unitary operation U_{qDec}^3 (resp. U_{qDec}^4) given in Eq. (8) (resp. Eq. (10)), we have $\|(U_{\text{qDec}}^3 - U_{\text{qDec}}^4)|\Phi_1\rangle|0\rangle_{\text{ST}}\| = 0$ and

$$\begin{aligned}
& \|(U_{\text{qDec}}^3 - U_{\text{qDec}}^4)|\Phi_2\rangle|0\rangle_{\text{ST}}\|^2 \\
&= \left\| \sum_{\substack{n(D_G) \leq q_G, n(D_H) \leq q_H - 2 \\ f_{\text{sk}}^{-1}(c) = (s, t) \neq \perp \\ D_H(s) = \perp}} \frac{\beta_{D_G, D_H}^{z, c, y}}{\sqrt{2^{k_0}}} S_s \left[|\gamma_1[0^{k_0}, [s \oplus D_G(t \oplus y_1)]^n]\rangle - |\gamma_1[0^{k_0}, \perp]\rangle \right] \right\|^2 \\
&\stackrel{(a)}{\leq} 2 \left\| \sum_{\substack{n(D_G) \leq q_G, n(D_H) \leq q_H - 2 \\ f_{\text{sk}}^{-1}(c) = (s, t) \neq \perp \\ D_H(s) = \perp}} \frac{\beta_{D_G, D_H}^{z, c, y}}{\sqrt{2^{k_0}}} S_s |\gamma_1[0^{k_0}, [s \oplus D_G(t \oplus y_1)]^n]\rangle \right\|^2 \\
&\quad + 2 \left\| \sum_{\substack{n(D_G) \leq q_G, n(D_H) \leq q_H - 2 \\ f_{\text{sk}}^{-1}(c) = (s, t) \neq \perp \\ D_H(s) = \perp}} \frac{\beta_{D_G, D_H}^{z, c, y}}{\sqrt{2^{k_0}}} S_s |\gamma_1[0^{k_0}, \perp]\rangle \right\|^2 \\
&\stackrel{(b)}{=} 2 \sum_{\substack{z, c, y, D_G, D_H \\ n(D_G) \leq q_G \\ n(D_H) \leq q_H - 2 \\ f_{\text{sk}}^{-1}(c) = (s, t) \neq \perp \\ D_H(s) = \perp, y_1 \in L_{D_G}^t \\ [D_G(t \oplus y_1)]_{k_1} = [s]_{k_1}}} \left\| \frac{\beta_{D_G, D_H}^{z, c, y}}{\sqrt{2^{k_0}}} |z, c, y \oplus [s \oplus D_G(t \oplus y_1)]^n, D_G, D_H \cup (s, y_1)\rangle \right\|^2 \\
&\quad + 2 \sum_{\substack{n(D_G) \leq q_G, n(D_H) \leq q_H - 2 \\ f_{\text{sk}}^{-1}(c) = (s, t) \neq \perp \\ D_H(s) = \perp, y_1 \in L_{D_G}^t \\ [D_G(t \oplus y_1)]_{k_1} = [s]_{k_1}}} \left\| \frac{\beta_{D_G, D_H}^{z, c, y}}{\sqrt{2^{k_0}}} |z, c, y \oplus \perp, D_G, D_H \cup (s, y_1)\rangle \right\|^2 \\
&= 4 \sum_{\substack{n(D_G) \leq q_G, n(D_H) \leq q_H - 2 \\ f_{\text{sk}}^{-1}(c) = (s, t) \neq \perp \\ D_H(s) = \perp}} \frac{|\{y_1 : y_1 \in L_{D_G}^t \wedge [D_G(t \oplus y_1)]_{k_1} = [s]_{k_1}\}| \cdot |\beta_{D_G, D_H}^{z, c, y}|^2}{2^{k_0}} \\
&\stackrel{(c)}{\leq} \frac{4q_G}{2^{k_0}} \sum_{\substack{n(D_G) \leq q_G, n(D_H) \leq q_H - 2 \\ f_{\text{sk}}^{-1}(c) = (s, t) \neq \perp \\ D_H(s) = \perp}} |\beta_{D_G, D_H}^{z, c, y}|^2 = 4q_G \cdot 2^{-k_0} \|\Phi_2\|^2.
\end{aligned} \tag{36}$$

Here (a) uses Lemma 10, (b) follows from the definition of $|\gamma_1[w, \nu]\rangle_{D_G, D_H}^{z, c, y}$ as shown in Eq. (35) and the fact that S_r is a unitary operation. (c) follows the fact that $|L_{D_G}^t| \leq q_G$. Additionally, similar to the computation process of the above

formula we can get

$$\|(\mathbf{U}_{\text{qDec}}^3 - \mathbf{U}_{\text{qDec}}^4) |\Phi_4\rangle |0\rangle_{\text{ST}}\|^2 \leq 4q_{\text{G}} \cdot 2^{-k_0} \|\Phi_4\|^2. \quad (37)$$

As for $\|(\mathbf{U}_{\text{qDec}}^3 - \mathbf{U}_{\text{qDec}}^4) |\Phi_3\rangle |0\rangle_{\text{ST}}\|$, we have

$$\begin{aligned} & \|(\mathbf{U}_{\text{qDec}}^3 - \mathbf{U}_{\text{qDec}}^4) |\Phi_3\rangle |0\rangle_{\text{ST}}\|^2 \\ &= \left\| \sum_{\substack{n(D_{\text{G}}) \leq q_{\text{G}}, n(D_{\text{H}}) \leq q_{\text{H}}-2 \\ f_{\text{sk}}^{-1}(c)=(s,t) \neq \perp \\ D_{\text{H}}(s)=\perp \\ u \in \{0,1\}^{k_0}, u \neq 0^{k_0}}} \frac{\beta_{D_{\text{G}}, D_{\text{H}}}^{z,c,y,u}}{2^{k_0}} \begin{bmatrix} |\gamma_2[u, [s \oplus D_{\text{G}}(t \oplus y_1)]^n]\rangle - |\gamma_2[u, \perp]\rangle \\ -|\gamma_3[u, [s \oplus D_{\text{G}}(t \oplus y_1)]^n]\rangle + |\gamma_3[u, \perp]\rangle \end{bmatrix} \right\|^2 \\ &\stackrel{(d)}{\leq} \frac{4q_{\text{G}}}{2^{k_0}} \sum_{\substack{n(D_{\text{G}}) \leq q_{\text{G}}, n(D_{\text{H}}) \leq q_{\text{H}}-2 \\ f_{\text{sk}}^{-1}(c)=(s,t) \neq \perp \\ D_{\text{H}}(s)=\perp \\ y_1 \in \mathbf{L}_{D_{\text{G}}}^t, [D_{\text{G}}(t \oplus y_1)]_{k_1}=[s]_{k_1}}} \left\| \sum_{\substack{u \in \{0,1\}^{k_0} \\ u \neq 0^{k_0}}} \frac{(-1)^{y_1 \cdot u} \beta_{D_{\text{G}}, D_{\text{H}}}^{z,c,y,u}}{\sqrt{2^{k_0}}} \left| y \oplus [s \oplus D_{\text{G}}(t \oplus y_1)]^n, \right. \right. \\ &\quad \left. \left. \begin{matrix} z, c, \\ D_{\text{G}}, D_{\text{H}} \end{matrix} \right\rangle \right\|^2 \\ &\quad + \frac{4q_{\text{G}}}{2^{k_0}} \sum_{\substack{n(D_{\text{G}}) \leq q_{\text{G}}, n(D_{\text{H}}) \leq q_{\text{H}}-2 \\ f_{\text{sk}}^{-1}(c)=(s,t) \neq \perp \\ D_{\text{H}}(s)=\perp \\ y_1 \in \mathbf{L}_{D_{\text{G}}}^t, [D_{\text{G}}(t \oplus y_1)]_{k_1}=[s]_{k_1}}} \left\| \sum_{\substack{u \in \{0,1\}^{k_0} \\ u \neq 0^{k_0}}} \frac{(-1)^{y_1 \cdot u} \beta_{D_{\text{G}}, D_{\text{H}}}^{z,c,y,u}}{\sqrt{2^{k_0}}} |z, c, y \oplus \perp, D_{\text{G}}, D_{\text{H}}\rangle \right\|^2 \\ &\quad + \frac{4q_{\text{G}}}{2^{k_0}} \sum_{\substack{n(D_{\text{G}}) \leq q_{\text{G}}, n(D_{\text{H}}) \leq q_{\text{H}}-2 \\ f_{\text{sk}}^{-1}(c)=(s,t) \neq \perp \\ D_{\text{H}}(s)=\perp, y_2 \in \{0,1\}^{k_0} \\ y_1 \in \mathbf{L}_{D_{\text{G}}}^t, [D_{\text{G}}(t \oplus y_1)]_{k_1}=[s]_{k_1}}} \left\| \sum_{\substack{u \in \{0,1\}^{k_0} \\ u \neq 0^{k_0}}} \frac{(-1)^{y_1 \cdot u} \beta_{D_{\text{G}}, D_{\text{H}}}^{z,c,y,u}}{2^{k_0}} \left| y \oplus [s \oplus D_{\text{G}}(t \oplus y_1)]^n \right. \right. \\ &\quad \left. \left. \begin{matrix} z, c, \\ D_{\text{G}}, D_{\text{H}} \cup (s, y_2) \end{matrix} \right\rangle \right\|^2 \\ &\quad + \frac{4q_{\text{G}}}{2^{k_0}} \sum_{\substack{n(D_{\text{G}}) \leq q_{\text{G}}, n(D_{\text{H}}) \leq q_{\text{H}}-2 \\ f_{\text{sk}}^{-1}(c)=(s,t) \neq \perp \\ D_{\text{H}}(s)=\perp, y_2 \in \{0,1\}^{k_0} \\ y_1 \in \mathbf{L}_{D_{\text{G}}}^t, [D_{\text{G}}(t \oplus y_1)]_{k_1}=[s]_{k_1}}} \left\| \sum_{\substack{u \in \{0,1\}^{k_0} \\ u \neq 0^{k_0}}} \frac{(-1)^{y_1 \cdot u} \beta_{D_{\text{G}}, D_{\text{H}}}^{z,c,y,u}}{2^{k_0}} \left| \begin{matrix} z, c, y \oplus \perp \\ D_{\text{G}}, D_{\text{H}} \cup (s, y_2) \end{matrix} \right\rangle \right\|^2 \\ &= \frac{16q_{\text{G}}}{2^{k_0}} \sum_{\substack{n(D_{\text{G}}) \leq q_{\text{G}}, n(D_{\text{H}}) \leq q_{\text{H}}-2 \\ f_{\text{sk}}^{-1}(c)=(s,t) \neq \perp \\ D_{\text{H}}(s)=\perp}} \sum_{y_1 \in \mathbf{L}_{D_{\text{G}}}^t, [D_{\text{G}}(t \oplus y_1)]_{k_1}=[s]_{k_1}} \left| \sum_{\substack{u \in \{0,1\}^{k_0} \\ u \neq 0^{k_0}}} \frac{\beta_{D_{\text{G}}, D_{\text{H}}}^{z,c,y,u}}{\sqrt{2^{k_0}}} \right|^2 \\ &\stackrel{(e)}{\leq} 16q_{\text{G}} \cdot 2^{-k_0} \|\Phi_3\|^2. \end{aligned} \quad (38)$$

Here (d) firstly uses Lemma 10 and the property $n(D_{\text{G}}) \leq q_{\text{G}}$, and then follows the definition of $|\gamma_2[w, \nu]\rangle_{y, D_{\text{G}}}^{z,c}$ and $|\gamma_3[w, \nu]\rangle_{y, D_{\text{G}}}^{z,c}$ as shown in Eq. (35), while (e) follows the definition of $|\Phi_3\rangle$.

Thus, by combining Eq. (36), Eq. (37) and Eq. (38), we finally obtain

$$\begin{aligned}
\|(\mathbf{U}_{\text{qDec}}^3 - \mathbf{U}_{\text{qDec}}^4) |\Phi\rangle |0\rangle_{\text{ST}}\| &\stackrel{(f)}{\leq} \sum_{i=1}^4 \|(\mathbf{U}_{\text{qDec}}^3 - \mathbf{U}_{\text{qDec}}^4) |\Phi_i\rangle |0\rangle_{\text{ST}}\| \\
&\leq 2\sqrt{q_G} \cdot \sqrt{2^{-k_0}} (\|\Phi_2\rangle\| + 2\|\Phi_3\rangle\| + \|\Phi_4\rangle\|) \\
&\stackrel{(g)}{\leq} 8\sqrt{q_G} \cdot \sqrt{2^{-k_0}} \|\Phi\rangle\| = 8\sqrt{q_G} \cdot \sqrt{2^{-k_0}}.
\end{aligned} \tag{39}$$

Here (f) uses the triangle inequality, and (g) comes from the fact that $\|\Phi_i\rangle\| \leq \|\Phi\rangle\|$ for $i \in [4]$. Recall that $\mathbf{U}_{\text{qDec}}^{3,*} = \mathbf{U}_\perp \circ \mathbf{P}_{c^*} + \mathbf{U}_{\text{qDec}}^3 \circ (\mathbf{I} - \mathbf{P}_{c^*})$ and $\mathbf{U}_{\text{qDec}}^{4,*} = \mathbf{U}_\perp \circ \mathbf{P}_{c^*} + \mathbf{U}_{\text{qDec}}^4 \circ (\mathbf{I} - \mathbf{P}_{c^*})$, thus it is easy to check that

$$\begin{aligned}
\|(\mathbf{U}_{\text{qDec}}^{3,*} - \mathbf{U}_{\text{qDec}}^{4,*}) |\Phi\rangle |0\rangle_{\text{ST}}\| &= \|(\mathbf{U}_{\text{qDec}}^3 - \mathbf{U}_{\text{qDec}}^4) \circ (\mathbf{I} - \mathbf{P}_{c^*}) |\Phi\rangle |0\rangle_{\text{ST}}\| \\
&\leq 8\sqrt{q_G} \cdot \sqrt{2^{-k_0}}.
\end{aligned}$$

Now for any unit joint state $|\Phi\rangle$ on the registers Z, l, O, D_{q_G} and D_{q_H} just before one decryption query, we have

$$\max \left\{ \|(\mathbf{U}_{\text{qDec}}^3 - \mathbf{U}_{\text{qDec}}^4) |\Phi\rangle |0\rangle_{\text{ST}}\|, \|(\mathbf{U}_{\text{qDec}}^{3,*} - \mathbf{U}_{\text{qDec}}^{4,*}) |\Phi\rangle |0\rangle_{\text{ST}}\| \right\} \leq 8\sqrt{q_G} \cdot \sqrt{2^{-k_0}}.$$

Then, similar to the proof given in Supplementary Material D.1, we can prove that $|\Pr[1 \leftarrow \mathbf{G}_3] - \Pr[1 \leftarrow \mathbf{G}_4]| \leq 8q_D\sqrt{q_G} \cdot \sqrt{2^{-k_0}}$.

D.4 Proof of Eq. (12): $\mathbf{U}_{\text{qDec}}^4/\mathbf{U}_{\text{qDec}}^{4,*}$ is Identical to $\mathbf{U}_{\text{Search}}/\mathbf{U}_{\text{Search}}^*$

For the joint state $|\Phi\rangle$ on registers Z, l, O, D_{q_G} and D_{q_H} just before one decryption query, we can denote it as a superposition of the computational basis state $|z, c, y, D_G, D_H\rangle$. Here we introduce Z to denote the ancillary quantum registers that may be used by the game. In the following, we divide $|z, c, y, D_G, D_H\rangle$ into five complete cases according to the tuple (z, c, y, D_G, D_H) and show that performing $\mathbf{U}_{\text{qDec}}^4$ and $\mathbf{U}_{\text{Search}}$ on each case results in the same state.

1. $\mathbf{f}_{\text{sk}}^{-1}(\mathbf{c}) = \perp$. It is easy to obtain that $\mathbf{U}_{\text{qDec}}^4$ maps $|z, c, y, D_G, D_H\rangle |0\rangle_{\text{ST}}$ to $|z, c, y \oplus \perp, D_G, D_H\rangle |0\rangle_{\text{ST}}$. Moreover, it must hold that $\text{Search}(\text{pk}, c, D_G, D_H) = \perp$ otherwise it will contradict with the condition $\mathbf{f}_{\text{sk}}^{-1}(c) = \perp$, meaning that $\mathbf{U}_{\text{Search}} |z, c, y, D_G, D_H\rangle |0\rangle_{\text{ST}} = |z, c, y \oplus \perp, D_G, D_H\rangle |0\rangle_{\text{ST}}$.
2. $\mathbf{f}_{\text{sk}}^{-1}(\mathbf{c}) = (\mathbf{s}, \mathbf{t}), \mathbf{D}_H(\mathbf{s}) = \perp$. By a direct computation we know that $\mathbf{U}_{\text{qDec}}^4$ maps $|z, c, y, D_G, D_H\rangle |0\rangle_{\text{ST}}$ to $|z, c, y \oplus \perp, D_G, D_H\rangle |0\rangle_{\text{ST}}$ as well. As for the $\mathbf{U}_{\text{Search}}$, suppose $\text{Search}(\text{pk}, c, D_G, D_H) \neq \perp$, then there must exist $(\tilde{s}, D_H(\tilde{s}))$ and $(\tilde{r}, D_G(\tilde{r}))$ s.t. $f_{\text{pk}}(\tilde{s}, \tilde{r} \oplus D_H(\tilde{s})) = c$. Set $\tilde{t}$ equal to $\tilde{r} \oplus D_H(\tilde{s})$, then we have that $f_{\text{pk}}(\mathbf{s}, \mathbf{t}) = f_{\text{pk}}(\tilde{s}, \tilde{t}) = c$ where $\tilde{s} \neq \mathbf{s}$, which contradicts with the injective property of f_{pk} . So in this case we also have $\mathbf{U}_{\text{Search}} |z, c, y, D_G, D_H\rangle |0\rangle_{\text{ST}} = |z, c, y \oplus \perp, D_G, D_H\rangle |0\rangle_{\text{ST}}$.

3. $f_{\mathbf{sk}}^{-1}(c) = (s, t), D_{\mathbf{H}}(s) \neq \perp, r := t \oplus D_{\mathbf{H}}(s), D_{\mathbf{G}}(r) = \perp$. Same as case 2, U_{qDec}^4 maps $|z, c, y, D_{\mathbf{G}}, D_{\mathbf{H}}\rangle|0\rangle_{\text{ST}}$ to $|z, c, y \oplus \perp, D_{\mathbf{G}}, D_{\mathbf{H}}\rangle|0\rangle_{\text{ST}}$. As for the U_{Search} , suppose that $\text{Search}(\text{pk}, c, D_{\mathbf{G}}, D_{\mathbf{H}}) \neq \perp$, then there must exist $(\tilde{s}, D_{\mathbf{H}}(\tilde{s}))$ and $(\tilde{r}, D_{\mathbf{G}}(\tilde{r}))$ s.t. $f_{\text{pk}}(\tilde{s}, \tilde{r} \oplus D_{\mathbf{H}}(\tilde{s})) = c$. Set $\tilde{t}$ equals to $\tilde{r} \oplus D_{\mathbf{H}}(\tilde{s})$, then we have $f_{\text{pk}}(s, t) = f_{\text{pk}}(\tilde{s}, \tilde{t}) = c$ where $\tilde{t} \neq t$, which contradicts with the injective property of f_{pk} . Hence in this case we also have $U_{\text{Search}}|z, c, y, D_{\mathbf{G}}, D_{\mathbf{H}}\rangle|0\rangle_{\text{ST}} = |z, c, y \oplus \perp, D_{\mathbf{G}}, D_{\mathbf{H}}\rangle|0\rangle_{\text{ST}}$.
4. $f_{\mathbf{sk}}^{-1}(c) = (s, t), D_{\mathbf{H}}(s) \neq \perp, r := t \oplus D_{\mathbf{H}}(s), D_{\mathbf{G}}(r) \neq \perp, [s]_{k_1} \neq [D_{\mathbf{G}}(r)]_{k_1}$. It is obvious that $U_{\text{qDec}}^4|z, c, y, D_{\mathbf{G}}, D_{\mathbf{H}}\rangle|0\rangle_{\text{ST}} = |z, c, y \oplus \perp, D_{\mathbf{G}}, D_{\mathbf{H}}\rangle|0\rangle_{\text{ST}}$. As for U_{Search} , suppose $\text{Search}(\text{pk}, c, D_{\mathbf{G}}, D_{\mathbf{H}}) \neq \perp$, then there must exist $(\tilde{s}, D_{\mathbf{H}}(\tilde{s}))$ and $(\tilde{r}, D_{\mathbf{G}}(\tilde{r}))$ s.t. $f_{\text{pk}}(\tilde{s}, \tilde{r} \oplus D_{\mathbf{H}}(\tilde{s})) = c$ and $[\tilde{s}]_{k_1} = [D_{\mathbf{G}}(\tilde{r})]_{k_1}$. Set $\tilde{t}$ equals to $\tilde{r} \oplus D_{\mathbf{H}}(\tilde{s})$, then we have $f_{\text{pk}}(s, t) = f_{\text{pk}}(\tilde{s}, \tilde{t}) = c$. By the injective property of f_{pk} we know that $(s, t) = (\tilde{s}, \tilde{t})$, thus $D_{\mathbf{H}}(s) = D_{\mathbf{H}}(\tilde{s}), D_{\mathbf{G}}(r) = D_{\mathbf{G}}(\tilde{r})$, and $[s]_{k_1} = [\tilde{s}]_{k_1} = [D_{\mathbf{G}}(\tilde{r})]_{k_1} = [D_{\mathbf{G}}(r)]_{k_1}$, which contradicts the condition $[s]_{k_1} \neq [D_{\mathbf{G}}(r)]_{k_1}$. Thus we have $U_{\text{Search}}|z, c, y, D_{\mathbf{G}}, D_{\mathbf{H}}\rangle|0\rangle_{\text{ST}} = |z, c, y \oplus \perp, D_{\mathbf{G}}, D_{\mathbf{H}}\rangle|0\rangle_{\text{ST}}$.
5. $f_{\mathbf{sk}}^{-1}(c) = (s, t), D_{\mathbf{H}}(s) \neq \perp, r := t \oplus D_{\mathbf{H}}(s), D_{\mathbf{G}}(r) \neq \perp, [s]_{k_1} = [D_{\mathbf{G}}(r)]_{k_1}$. In this case both U_{qDec}^4 and U_{Search} map $|z, c, y, D_{\mathbf{G}}, D_{\mathbf{H}}\rangle|0\rangle_{\text{ST}}$ to $|z, c, y \oplus [s \oplus D_{\mathbf{G}}(r)]^n, D_{\mathbf{G}}, D_{\mathbf{H}}\rangle|0\rangle_{\text{ST}}$.

According to the above analysis, we have $U_{\text{qDec}}^4|\Phi\rangle|0\rangle_{\text{ST}} = U_{\text{Search}}|\Phi\rangle|0\rangle_{\text{ST}}$. Recall that $U_{\text{qDec}}^{4,*} = U_{\perp} \circ P_{c^*} + U_{\text{qDec}}^4 \circ (I - P_{c^*})$ and $U_{\text{Search}}^* = U_{\perp} \circ P_{c^*} + U_{\text{Search}} \circ (I - P_{c^*})$, thus we also have $U_{\text{qDec}}^{4,*}|\Phi\rangle|0\rangle_{\text{ST}} = U_{\text{Search}}^*|\Phi\rangle|0\rangle_{\text{ST}}$.

D.5 Proof of Lemma 3

Here we first compute the upper bound of $\Pr[\text{Find}_{r^*} : \mathbf{G}_6]$. In fact, based on the r^* uniformly chosen from $\{0, 1\}^{k_0}$, we can directly construct a quantum oracle algorithm $\mathcal{B}^{\mathcal{O}_{\{r^*\}}^{SC}}(\text{pk})$ that runs $\mathcal{A}^{\text{G}, \text{H}, \text{qDec}}(\text{pk})$ in game $\mathbf{G}_6$ to generate (m_0^*, m_1^*) . During this running, $\mathcal{B}$ will reply to $\mathcal{A}$'s query to qDec and H by performing U_{Search} and CStO_{H} , respectively; meanwhile, every time $\mathcal{A}$ queries to G , $\mathcal{B}$ will first query its oracle $\mathcal{O}_{\{r^*\}}^{SC}$ and then perform CStO_{G} . Now it is evident that

$$\Pr[\text{Find}_{r^*} : \mathbf{G}_6] = \Pr[\text{Find}_{r^*} : \mathcal{B}^{\mathcal{O}_{\{r^*\}}^{SC}}(\text{pk})].$$

Here $\mathcal{B}$ makes at most q_{G} queries to its oracle $\mathcal{O}_{\{r^*\}}^{SC}$, and its input pk is independent of the set $\{r^*\}$. Hence by using Corollary 1 and the fact that r^* is uniformly chosen from $\{0, 1\}^{k_0}$, we have

$$\Pr[\text{Find}_{r^*} : \mathcal{B}^{\mathcal{O}_{\{r^*\}}^{SC}}(\text{pk})] \leq 4q_{\text{G}} \cdot \max_{x \in \{0, 1\}^{k_0}} \Pr[x \in \{r^*\}] = 4q_{\text{G}} \cdot 2^{-k_0}.$$

That is to say, $\Pr[\text{Find}_{r^*} : \mathbf{G}_6] \leq 4q_{\text{G}} \cdot 2^{-k_0}$. Then, by a similar analysis we also have $\Pr[\text{Find}_{s^*} : \mathbf{G}_9] \leq 4q_{\text{H}} \cdot 2^{-n-k_1}$. This proves Lemma 3.

D.6 Proof of Lemma 4

We first consider the upper bound of $\| [U_{\text{Search}}^*, \text{StdDecomp}_{r^*}] \|$. Recall that U_{Search}^* acts on registers I, O, D_{q_G} and D_{q_H} , and StdDecomp_{r^*} acts on register D_{q_G} . Here I/O represents the input/output register of the decryption oracle. D_{q_G} and D_{q_H} are the database registers used to simulate G and H , respectively.

For a unit joint state on the registers I, O, D_{q_G} and D_{q_H} , we can denote it as:

$$|\Phi\rangle := \sum_{c,y,D_G,D_H} \alpha_{D_G,D_H}^{c,y} |c,y,D_G,D_H\rangle.$$

Here similar to Supplementary Material D.2, we omit the detailed set notations for c,y,D_G and D_H , and abbreviate $|c,y\rangle_{IO} |D_G\rangle_{D_{q_G}} |D_H\rangle_{D_{q_H}}$ into $|c,y,D_G,D_H\rangle$. Next, we divide $|\Phi\rangle$ into the following four complete and mutually orthogonal parts:

$$\begin{aligned} |\Phi_1\rangle &:= \sum_{\substack{c,y,D_G,D_H \\ c=c^* \vee (c \neq c^* \wedge \nexists s \text{ s.t. } f_{pk}(s,r^* \oplus D_H(s))=c)}} \beta_{D_G,D_H}^{c,y} |c,y,D_G,D_H\rangle, \\ |\Phi_2\rangle &:= \sum_{\substack{c,y,D_G,D_H,c \neq c^* \\ \exists s \text{ s.t. } f_{pk}(s,r^* \oplus D_H(s))=c \\ D_G(r^*)=\perp}} \beta_{D_G,D_H}^{c,y} |c,y,D_G,D_H\rangle, \\ |\Phi_3\rangle &:= \sum_{\substack{c,y,D_G,D_H,c \neq c^* \\ \exists s \text{ s.t. } f_{pk}(s,r^* \oplus D_H(s))=c \\ D_G(r^*)=\perp, n(D_G) \leq q_G-1 \\ u \in \{0,1\}^{n+k_1}, u \neq 0^{n+k_1}}} \frac{\beta_{D_G,D_H}^{c,y,u}}{\sqrt{2^{n+k_1}}} \sum_{y_1 \in \{0,1\}^{n+k_1}} (-1)^{y_1 \cdot u} |c,y,D_G \cup (r^*, y_1), D_H\rangle, \\ |\Phi_4\rangle &:= \sum_{\substack{c,y,D_G,D_H,c \neq c^* \\ \exists s \text{ s.t. } f_{pk}(s,r^* \oplus D_H(s))=c \\ D_G(r^*)=\perp, n(D_G) \leq q_G-1, u=0^{n+k_1}}} \frac{\beta_{D_G,D_H}^{c,y,u}}{\sqrt{2^{n+k_1}}} \sum_{y_1 \in \{0,1\}^{n+k_1}} |c,y,D_G \cup (r^*, y_1), D_H\rangle. \end{aligned}$$

We emphasize that the notation " $f_{pk}(s, r^* \oplus D_H(s))$ " assumes that $D_H(s) \neq \perp$. In addition, since f_{pk} is an injective function, there exists at most one $(s, D_H(s))$ pair s.t. $f_{pk}(s, r^* \oplus D_H(s)) = c$ for a fixed c . This property will be used by our following proof, and we will not specifically point it out for simplicity.

For the fixed tuple (c, y, D_G, D_H) where $c \neq c^*$, $\exists s$ s.t. $f_{pk}(s, r^* \oplus D_H(s)) = c$ and $D_G(r^*) = \perp$, we define the following states that will be used later. Here $w \in \{0,1\}^{n+k_1}$ and $\nu \in \{0,1\}^n \cup \{\perp\}$.

$$\begin{aligned} |\gamma_1[w, \nu]\rangle_{D_G,D_H}^{c,y} &:= \sum_{[y_1]_{k_1}=[s]_{k_1}} (-1)^{y_1 \cdot w} |c, y \oplus \nu, D_G \cup (r^*, y_1), D_H\rangle, \\ |\gamma_2[w, \nu]\rangle_{D_G,D_H}^{c,y} &:= \sum_{[y_1]_{k_1}=[s]_{k_1}} (-1)^{y_1 \cdot w} |c, y \oplus \nu, D_G, D_H\rangle, \\ |\gamma_3[w, \nu]\rangle_{D_G,D_H}^{c,y} &:= \sum_{\substack{y_2 \in \{0,1\}^{n+k_1} \\ [y_1]_{k_1}=[s]_{k_1}}} \frac{(-1)^{y_1 \cdot w}}{\sqrt{2^{n+k_1}}} |c, y \oplus \nu, D_G \cup (r^*, y_2), D_H\rangle. \end{aligned} \tag{40}$$

Let S_{r^*} be an abbreviation of the local decompression procedure StdDecomp_{r^*} . Then by the definition of U_{Search}^* introduced in game $\mathbf{G}_5$ and the definition of S_{r^*} given in Section 2.4, we have $\|(U_{\text{Search}}^* \circ S_{r^*} - S_{r^*} \circ U_{\text{Search}}^*)|\Phi_1\rangle\| = 0$ and

$$\begin{aligned}
& \|(U_{\text{Search}}^* \circ S_{r^*} - S_{r^*} \circ U_{\text{Search}}^*)|\Phi_2\rangle\|^2 \\
&= \left\| \sum_{\substack{c,y,D_G,D_H,c \neq c^* \\ \exists s \text{ s.t. } f_{\text{pk}}(s,r^* \oplus D_H(s))=c \\ D_G(r^*)=\perp}} \frac{\beta_{D_G,D_H}^{c,y}}{\sqrt{2^{n+k_1}}} \left[|\gamma_1[0^{n+k_1}, [s \oplus y_1]^n]\rangle - |\gamma_1[0^{n+k_1}, \perp]\rangle \right] \right\|^2 \\
&\stackrel{(a)}{\leq} 2 \left\| \sum_{\substack{c,y,D_G,D_H,c \neq c^* \\ \exists s \text{ s.t. } f_{\text{pk}}(s,r^* \oplus D_H(s))=c \\ D_G(r^*)=\perp}} \frac{\beta_{D_G,D_H}^{c,y}}{\sqrt{2^{n+k_1}}} |\gamma_1[0^{n+k_1}, [s \oplus y_1]^n]\rangle \right\|^2 \\
&\quad + 2 \left\| \sum_{\substack{c,y,D_G,D_H,c \neq c^* \\ \exists s \text{ s.t. } f_{\text{pk}}(s,r^* \oplus D_H(s))=c \\ D_G(r^*)=\perp}} \frac{\beta_{D_G,D_H}^{c,y}}{\sqrt{2^{n+k_1}}} |\gamma_1[0^{n+k_1}, \perp]\rangle \right\|^2 \\
&\stackrel{(b)}{=} 2 \sum_{\substack{c,D_G,D_H,c \neq c^* \\ \exists s \text{ s.t. } f_{\text{pk}}(s,r^* \oplus D_H(s))=c \\ D_G(r^*)=\perp}} \left\| \sum_y \frac{\beta_{D_G,D_H}^{c,y}}{\sqrt{2^{n+k_1}}} |\gamma_1[0^{n+k_1}, [s \oplus y_1]^n]\rangle \right\|^2 \\
&\quad + 2 \sum_{\substack{c,D_G,D_H,c \neq c^* \\ \exists s \text{ s.t. } f_{\text{pk}}(s,r^* \oplus D_H(s))=c \\ D_G(r^*)=\perp}} \left\| \sum_y \frac{\beta_{D_G,D_H}^{c,y}}{\sqrt{2^{n+k_1}}} |\gamma_1[0^{n+k_1}, \perp]\rangle \right\|^2 \\
&\stackrel{(c)}{=} 2 \sum_{\substack{c,y,D_G,D_H,c \neq c^* \\ \exists s \text{ s.t. } f_{\text{pk}}(s,r^* \oplus D_H(s))=c \\ D_G(r^*)=\perp}} \sum_{[y_1]_{k_1}=[s]_{k_1}} \left\| \frac{\beta_{D_G,D_H}^{c,y}}{\sqrt{2^{n+k_1}}} |c, y \oplus [s \oplus y_1]^n, D_G \cup (r^*, y_1), D_H\rangle \right\|^2 \\
&\quad + 2 \sum_{\substack{c,y,D_G,D_H,c \neq c^* \\ \exists s \text{ s.t. } f_{\text{pk}}(s,r^* \oplus D_H(s))=c \\ D_G(r^*)=\perp}} \sum_{[y_1]_{k_1}=[s]_{k_1}} \left\| \frac{\beta_{D_G,D_H}^{c,y}}{\sqrt{2^{n+k_1}}} |c, y \oplus \perp, D_G \cup (r^*, y_1), D_H\rangle \right\|^2 \\
&= 4 \sum_{\substack{c,y,D_G,D_H,c \neq c^* \\ \exists s \text{ s.t. } f_{\text{pk}}(s,r^* \oplus D_H(s))=c \\ D_G(r^*)=\perp}} \frac{2^n |\beta_{D_G,D_H}^{c,y}|^2}{2^{n+k_1}} = 4 \cdot 2^{-k_1} \|\Phi_2\|^2.
\end{aligned} \tag{41}$$

Here (a) uses Lemma 10, (b) and (c) both follow the definition of $|\gamma_1[w, \nu]\rangle_{D_G,D_H}^{c,y}$ as shown in Eq. (40). Additionally, similar to the above computation, we can compute

$$\|(U_{\text{Search}}^* \circ S_{r^*} - S_{r^*} \circ U_{\text{Search}}^*)|\Phi_4\rangle\|^2 \leq 4 \cdot 2^{-k_1} \|\Phi_4\|^2. \tag{42}$$

As for $\|(\mathbf{U}_{\text{Search}}^* \circ \mathbf{S}_{r^*} - \mathbf{S}_{r^*} \circ \mathbf{U}_{\text{Search}}^*)|\Phi_3\rangle\|$, with the help of the useful property of the local decompression procedure as shown in Eq. (1), we can compute

$$\begin{aligned}
& \|(\mathbf{U}_{\text{Search}}^* \circ \mathbf{S}_{r^*} - \mathbf{S}_{r^*} \circ \mathbf{U}_{\text{Search}}^*)|\Phi_3\rangle\|^2 \\
&= \left\| \sum_{\substack{c,y,D_G,D_H,c \neq c^* \\ \exists s \text{ s.t. } f_{\text{pk}}(s,r^* \oplus D_H(s))=c \\ D_G(r^*)=\perp, n(D_G) \leq q_G-1 \\ u \in \{0,1\}^{n+k_1}, u \neq 0^{n+k_1}}} \frac{\beta_{D_G,D_H}^{c,y,u}}{2^{n+k_1}} \begin{bmatrix} |\gamma_3[u, [s \oplus y_1]^n]\rangle + |\gamma_3[u, \perp]\rangle \\ -|\gamma_2[u, [s \oplus y_1]^n]\rangle - |\gamma_2[u, \perp]\rangle \end{bmatrix} \right\|^2 \\
&\stackrel{(d)}{\leq} \frac{4}{2^{k_1}} \sum_{\substack{c,y,D_G,D_H \\ c \neq c^*, \exists s \text{ s.t.} \\ f_{\text{pk}}(s,r^* \oplus D_H(s))=c \\ D_G(r^*)=\perp, n(D_G) \leq q_G-1 \\ [y_1]_{k_1}=[s]_{k_1}}} \left\| \sum_{u \in \{0,1\}^{n+k_1}, u \neq 0^{n+k_1}} \frac{(-1)^{y_1 \cdot u} \beta_{D_G,D_H}^{c,y,u}}{\sqrt{2^{n+k_1}}} |c, y \oplus [s \oplus y_1]^n\rangle \right\|^2 \\
&\quad + \frac{4}{2^{k_1}} \sum_{\substack{c,y,D_G,D_H \\ c \neq c^*, \exists s \text{ s.t.} \\ f_{\text{pk}}(s,r^* \oplus D_H(s))=c \\ D_G(r^*)=\perp, n(D_G) \leq q_G-1 \\ [y_1]_{k_1}=[s]_{k_1}}} \left\| \sum_{u \in \{0,1\}^{n+k_1}, u \neq 0^{n+k_1}} \frac{(-1)^{y_1 \cdot u} \beta_{D_G,D_H}^{c,y,u}}{\sqrt{2^{n+k_1}}} |c, y \oplus \perp, D_G, D_H\rangle \right\|^2 \\
&\quad + \frac{4}{2^{k_1}} \sum_{\substack{c,y,D_G,D_H \\ c \neq c^*, \exists s \text{ s.t.} \\ f_{\text{pk}}(s,r^* \oplus D_H(s))=c \\ D_G(r^*)=\perp, n(D_G) \leq q_G-1 \\ [y_1]_{k_1}=[s]_{k_1} \\ y_2 \in \{0,1\}^{n+k_1}}} \left\| \sum_{u \in \{0,1\}^{n+k_1}, u \neq 0^{n+k_1}} \frac{(-1)^{y_1 \cdot u} \beta_{D_G,D_H}^{c,y,u}}{2^{n+k_1}} |c, y \oplus [s \oplus y_1]^n, D_G \cup (r^*, y_2), D_H\rangle \right\|^2 \\
&\quad + \frac{4}{2^{k_1}} \sum_{\substack{c,y,D_G,D_H \\ c \neq c^*, \exists s \text{ s.t.} \\ f_{\text{pk}}(s,r^* \oplus D_H(s))=c \\ D_G(r^*)=\perp, n(D_G) \leq q_G-1 \\ [y_1]_{k_1}=[s]_{k_1} \\ y_2 \in \{0,1\}^{n+k_1}}} \left\| \sum_{u \in \{0,1\}^{n+k_1}, u \neq 0^{n+k_1}} \frac{(-1)^{y_1 \cdot u} \beta_{D_G,D_H}^{c,y,u}}{2^{n+k_1}} |c, y \oplus \perp, D_G \cup (r^*, y_2), D_H\rangle \right\|^2 \\
&= \frac{16}{2^{k_1}} \sum_{\substack{c,y,D_G,D_H \\ c \neq c^*, \exists s \text{ s.t.} \\ f_{\text{pk}}(s,r^* \oplus D_H(s))=c \\ D_G(r^*)=\perp, n(D_G) \leq q_G-1 \\ [y_1]_{k_1}=[s]_{k_1}}} \left\| \sum_{u \in \{0,1\}^{n+k_1}, u \neq 0^{n+k_1}} \frac{(-1)^{y_1 \cdot u} \beta_{D_G,D_H}^{c,y,u}}{\sqrt{2^{n+k_1}}} \right\|^2 \\
&\stackrel{(e)}{\leq} 16 \cdot 2^{-k_1} \|\Phi_3\|^2.
\end{aligned} \tag{43}$$

Here (d) firstly uses Lemma 10 and then follows the definition of $|\gamma_2[w, \nu]\rangle_{D_G, D_H}^{c,y}$ and $|\gamma_3[w, \nu]\rangle_{D_G, D_H}^{c,y}$ as shown in Eq. (40). (e) follows from the definition of $|\Phi_3\rangle$.

Thus, by combining Eq. (41), Eq. (42) and Eq. (43), for any unit joint state $|\Phi\rangle$, we finally obtain

$$\begin{aligned}
\|(\mathbf{U}_{\text{Search}}^* \circ \mathbf{S}_{r^*} - \mathbf{S}_{r^*} \circ \mathbf{U}_{\text{Search}}^*)|\Phi\rangle\| &\stackrel{(f)}{\leq} \sum_{i=1}^4 \|(\mathbf{U}_{\text{Search}}^* \circ \mathbf{S}_{r^*} - \mathbf{S}_{r^*} \circ \mathbf{U}_{\text{Search}}^*)|\Phi_i\rangle\| \\
&\leq 2 \cdot \sqrt{2^{-k_1}} (\|\Phi_2\rangle\| + \|\Phi_4\rangle\| + 2\|\Phi_3\rangle\|) \\
&\stackrel{(g)}{\leq} 8 \cdot \sqrt{2^{-k_1}} \|\Phi\rangle\| = 8 \cdot \sqrt{2^{-k_1}}.
\end{aligned}$$

Here (f) uses the triangle inequality, and (g) comes from the fact that $\|\Phi_i\rangle\| \leq \|\Phi\rangle\|$ for $i \in [4]$. Now we prove that $\|[\mathbf{U}_{\text{Search}}^*, \text{StdDecomp}_{r^*}]\| \leq 8 \cdot \sqrt{2^{-k_1}}$.

The upper bound of $\|[\mathbf{U}_{\text{Search}}^*, \text{StdDecomp}_{s^*}]\|$ can be deduced using an analogous method. First, we give the unit joint state $|\Phi\rangle$ on the registers $\mathbf{l}, \mathbf{O}, \mathbf{D}_{q_G}$ and $\mathbf{D}_{q_H}$ a new decompose as:

$$\begin{aligned}
|\Phi'_1\rangle &:= \sum_{\substack{c,y,D_G,D_H,c=c^* \\ \vee (c \neq c^* \wedge \nexists t \text{ s.t. } f_{\text{pk}}(s^*,t)=c) \\ \vee (c \neq c^* \wedge \exists t \text{ s.t. } f_{\text{pk}}(s^*,t)=c \\ \wedge \nexists r \text{ s.t. } [D_G(r)]_{k_1}=[s^*]_{k_1})}} \eta_{D_G,D_H}^{c,y} |c,y,D_G,D_H\rangle, \\
|\Phi'_2\rangle &:= \sum_{\substack{c,y,D_G,D_H,c \neq c^* \\ \exists t \text{ s.t. } f_{\text{pk}}(s^*,t)=c, D_H(s^*)=\perp \\ \exists r \text{ s.t. } [D_G(r)]_{k_1}=[s^*]_{k_1}}} \eta_{D_G,D_H}^{c,y} |c,y,D_G,D_H\rangle, \\
|\Phi'_3\rangle &:= \sum_{\substack{c,y,D_G,D_H,c \neq c^* \\ \exists t \text{ s.t. } f_{\text{pk}}(s^*,t)=c \\ D_H(s^*)=\perp, n(D_H) \leq q_H-1 \\ \exists r \text{ s.t. } [D_G(r)]_{k_1}=[s^*]_{k_1} \\ u \in \{0,1\}^{k_0}, u \neq 0^{k_0}}} \frac{\eta_{D_G,D_H}^{c,y,u}}{\sqrt{2^{k_0}}} \sum_{y_1 \in \{0,1\}^{k_0}} (-1)^{y_1 \cdot u} |c,y,D_G,D_H \cup (s^*, y_1)\rangle, \\
|\Phi'_4\rangle &:= \sum_{\substack{c,y,D_G,D_H,c \neq c^* \\ \exists t \text{ s.t. } f_{\text{pk}}(s^*,t)=c \\ D_H(s^*)=\perp, n(D_H) \leq q_H-1 \\ \exists r \text{ s.t. } [D_G(r)]_{k_1}=[s^*]_{k_1}; u=0^{k_0}}} \frac{\eta_{D_G,D_H}^{c,y,u}}{\sqrt{2^{k_0}}} \sum_{y_1 \in \{0,1\}^{k_0}} |c,y,D_G,D_H \cup (s^*, y_1)\rangle.
\end{aligned}$$

In the above formulas and during the following proof, we emphasize that the notation $"[D_G(r)]_{k_1}"$ implicitly assumes $D_G(r) \neq \perp$.

For the fixed tuple (c,y,D_G,D_H) where $c \neq c^*, \exists t \text{ s.t. } f_{\text{pk}}(s^*,t)=c, D_H(s^*)=\perp$, and $\exists r \text{ s.t. } [D_G(r)]_{k_1}=[s^*]_{k_1}$, we define the following states that will be used later.

$$\begin{aligned}
|\gamma'_1[w, \nu]\rangle_{D_G, D_H}^{c, y} &:= \sum_{y_1 \in L_{D_G}^c} (-1)^{y_1 \cdot w} |c, y \oplus \nu, D_G, D_H \cup (s^*, y_1)\rangle, \\
|\gamma'_2[w, \nu]\rangle_{D_G, D_H}^{c, y} &:= \sum_{y_1 \in L_{D_G}^c} (-1)^{y_1 \cdot w} |c, y \oplus \nu, D_G, D_H\rangle, \\
|\gamma'_3[w, \nu]\rangle_{D_G, D_H}^{c, y} &:= \sum_{y_1 \in L_{D_G}^c, y_2 \in \{0, 1\}^{k_0}} \frac{(-1)^{y_1 \cdot w}}{\sqrt{2^{k_0}}} |c, y \oplus \nu, D_G, D_H \cup (s^*, y_2)\rangle.
\end{aligned} \tag{44}$$

Here $w \in \{0, 1\}^{k_0}$, $\nu \in \{0, 1\}^n \cup \{\perp\}$, and $L_{D_G}^c := \{y \in \{0, 1\}^{k_0} : \exists r \text{ s.t. } D_G(r) \neq \perp \wedge f_{pk}(s^*, r \oplus y) = c \wedge [s^*]_{k_1} = [D_G(r)]_{k_1}\}$. Notice that for any c , by the injective property of f_{pk} we can conclude that there exists at most one t s.t. $f_{pk}(s^*, t) = c$. Further, based on this t , it is evident that $L_{D_G}^c \subseteq \{r \oplus t : r \text{ s.t. } D_G(r) \neq \perp\}$, which means that $|L_{D_G}^c| \leq q_G$. These two facts will be used by our following proof, and we will not specifically point them out for simplicity.

Let S_{s^*} be an abbreviation of the local decompression procedure StdDecomp_{s^*} . Then by the definition of U_{Search}^* introduced in game $\mathbf{G}_5$ and the definition of S_{s^*} given in Section 2.4, we have $\|(U_{\text{Search}}^* \circ S_{s^*} - S_{s^*} \circ U_{\text{Search}}^*)|\Phi'_1\rangle\| = 0$ and

$$\begin{aligned}
&\|(U_{\text{Search}}^* \circ S_{s^*} - S_{s^*} \circ U_{\text{Search}}^*)|\Phi'_2\rangle\|^2 \\
&= \left\| \sum_{\substack{c, y, D_G, D_H, c \neq c^* \\ \exists t \text{ s.t. } f_{pk}(s^*, t) = c, D_H(s^*) = \perp \\ \exists r \text{ s.t. } [D_G(r)]_{k_1} = [s^*]_{k_1}}} \frac{\eta_{D_G, D_H}^{c, y}}{\sqrt{2^{k_0}}} \left[|\gamma'_1[0^{k_0}, [s^* \oplus D_G(t \oplus y_1)]^n]\rangle - |\gamma'_1[0^{k_0}, \perp]\rangle \right] \right\|^2 \\
&\stackrel{(h)}{\leq} 2 \left\| \sum_{\substack{c, y, D_G, D_H, c \neq c^* \\ \exists t \text{ s.t. } f_{pk}(s^*, t) = c, D_H(s^*) = \perp \\ \exists r \text{ s.t. } [D_G(r)]_{k_1} = [s^*]_{k_1}}} \frac{\eta_{D_G, D_H}^{c, y}}{\sqrt{2^{k_0}}} |\gamma_1[0^{k_0}, [s \oplus D_G(t \oplus y_1)]^n]\rangle \right\|^2 \\
&\quad + 2 \left\| \sum_{\substack{c, y, D_G, D_H, c \neq c^* \\ \exists t \text{ s.t. } f_{pk}(s^*, t) = c, D_H(s^*) = \perp \\ \exists r \text{ s.t. } [D_G(r)]_{k_1} = [s^*]_{k_1}}} \frac{\eta_{D_G, D_H}^{c, y}}{\sqrt{2^{k_0}}} |\gamma_1[0^{k_0}, \perp]\rangle \right\|^2 \\
&\stackrel{(i)}{=} 2 \sum_{\substack{c, y, D_G, D_H, c \neq c^* \\ \exists t \text{ s.t. } f_{pk}(s^*, t) = c \\ \exists r \text{ s.t. } [D_G(r)]_{k_1} = [s^*]_{k_1} \\ D_H(s^*) = \perp, y_1 \in L_{D_G}^c}} \left\| \frac{\eta_{D_G, D_H}^{c, y}}{\sqrt{2^{k_0}}} |c, y \oplus [s^* \oplus D_G(t \oplus y_1)]^n, D_G, D_H \cup (s^*, y_1)\rangle \right\|^2 \\
&\quad + 2 \sum_{\substack{c, y, D_G, D_H, c \neq c^* \\ \exists t \text{ s.t. } f_{pk}(s^*, t) = c \\ \exists r \text{ s.t. } [D_G(r)]_{k_1} = [s^*]_{k_1} \\ D_H(s^*) = \perp, y_1 \in L_{D_G}^c}} \left\| \frac{\eta_{D_G, D_H}^{c, y}}{\sqrt{2^{k_0}}} |c, y \oplus \perp, D_G, D_H \cup (s^*, y_1)\rangle \right\|^2
\end{aligned}$$

$$\begin{aligned}
&= 4 \sum_{\substack{c, y, D_G, D_H, c \neq c^* \\ \exists t \text{ s.t. } f_{pk}(s^*, t) = c, D_H(s^*) = \perp \\ \exists r \text{ s.t. } [D_G(r)]_{k_1} = [s^*]_{k_1}}} \frac{|L_{D_G}^c| \cdot |\eta_{D_G, D_H}^{c, y}|^2}{2^{k_0}} \\
&\leq \frac{4q_G}{2^{k_0}} \sum_{\substack{c, y, D_G, D_H, c \neq c^* \\ \exists t \text{ s.t. } f_{pk}(s^*, t) = c, D_H(s^*) = \perp \\ \exists r \text{ s.t. } [D_G(r)]_{k_1} = [s^*]_{k_1}}} |\eta_{D_G, D_H}^{c, y}|^2 = 4q_G \cdot 2^{-k_0} \|\Phi'_2\|^2.
\end{aligned} \tag{45}$$

Here (h) uses Lemma 10, (i) follows from the definition of $|\gamma'_1[w, \nu]\rangle_{D_G, D_H}^{c, y}$ shown in Eq. (44). Additionally, similar to the above computation, we can compute

$$\|(\mathbf{U}_{\text{Search}}^* \circ \mathbf{S}_{s^*} - \mathbf{S}_{s^*} \circ \mathbf{U}_{\text{Search}}^*)|\Phi'_4\rangle\|^2 \leq 4q_G \cdot 2^{-k_0} \|\Phi'_4\|^2. \tag{46}$$

As for $\|(\mathbf{U}_{\text{Search}}^* \circ \mathbf{S}_{s^*} - \mathbf{S}_{s^*} \circ \mathbf{U}_{\text{Search}}^*)|\Phi'_3\rangle\|$, with the help of the useful property of local decompression procedure as shown in Eq. (1), we can compute

$$\begin{aligned}
&\|(\mathbf{U}_{\text{Search}}^* \circ \mathbf{S}_{s^*} - \mathbf{S}_{s^*} \circ \mathbf{U}_{\text{Search}}^*)|\Phi'_3\rangle\|^2 \\
&= \left\| \sum_{\substack{c, y, D_G, D_H, c \neq c^* \\ u \in \{0, 1\}^{k_0}, u \neq 0^{k_0} \\ \exists t \text{ s.t. } f_{pk}(s^*, t) = c \\ \exists r \text{ s.t. } [D_G(r)]_{k_1} = [s^*]_{k_1} \\ D_H(s^*) = \perp, n(D_H) \leq q_H - 1}} \frac{\eta_{D_G, D_H}^{c, y, u}}{2^{k_0}} \begin{bmatrix} |\gamma'_2[u, \perp]\rangle - |\gamma'_2[u, [s^* \oplus D_G(t \oplus y_1)]^n]\rangle \\ -|\gamma'_3[u, \perp]\rangle + |\gamma'_3[u, [s^* \oplus D_G(t \oplus y_1)]^n]\rangle \end{bmatrix} \right\|^2 \\
&\stackrel{(j)}{\leq} \frac{4q_G}{2^{k_0}} \sum_{\substack{c, y, D_G, D_H, c \neq c^* \\ \exists t \text{ s.t. } f_{pk}(s^*, t) = c, y_1 \in L_{D_G}^c \\ \exists r \text{ s.t. } [D_G(r)]_{k_1} = [s^*]_{k_1} \\ D_H(s^*) = \perp, n(D_H) \leq q_H - 1}} \left\| \sum_{u \in \{0, 1\}^{k_0}} \frac{(-1)^{y_1 \cdot u} \eta_{D_G, D_H}^{c, y, u}}{\sqrt{2^{k_0}}} \left| c, y \oplus [s^* \oplus D_G(t \oplus y_1)]^n \right\rangle_{D_G, D_H} \right\|^2 \\
&+ \frac{4q_G}{2^{k_0}} \sum_{\substack{c, y, D_G, D_H, c \neq c^* \\ \exists t \text{ s.t. } f_{pk}(s^*, t) = c, y_1 \in L_{D_G}^c \\ \exists r \text{ s.t. } [D_G(r)]_{k_1} = [s^*]_{k_1} \\ D_H(s^*) = \perp, n(D_H) \leq q_H - 1}} \left\| \sum_{u \in \{0, 1\}^{k_0}} \frac{(-1)^{y_1 \cdot u} \eta_{D_G, D_H}^{c, y, u}}{\sqrt{2^{k_0}}} \left| c, y \oplus \perp, D_G, D_H \right\rangle \right\|^2 \\
&+ \frac{4q_G}{2^{k_0}} \sum_{\substack{c, y, D_G, D_H, c \neq c^* \\ \exists t \text{ s.t. } f_{pk}(s^*, t) = c, y_1 \in L_{D_G}^c \\ \exists r \text{ s.t. } [D_G(r)]_{k_1} = [s^*]_{k_1} \\ D_H(s^*) = \perp, n(D_H) \leq q_H - 1}} \left\| \sum_{u \in \{0, 1\}^{k_0}} \frac{(-1)^{y_1 \cdot u} \eta_{D_G, D_H}^{c, y, u}}{2^{k_0}} \left| c, y \oplus [s \oplus D_G(t \oplus y_1)]^n \right\rangle_{D_G, D_H \cup (s^*, y_2)} \right\|^2 \\
&+ \frac{4q_G}{2^{k_0}} \sum_{\substack{c, y, D_G, D_H, c \neq c^* \\ u \in \{0, 1\}^{k_0}, u \neq 0^{k_0} \\ \exists t \text{ s.t. } f_{pk}(s^*, t) = c, y_1 \in L_{D_G}^c \\ \exists r \text{ s.t. } [D_G(r)]_{k_1} = [s^*]_{k_1} \\ D_H(s^*) = \perp, n(D_H) \leq q_H - 1}} \left\| \sum_{u \in \{0, 1\}^{k_0}} \frac{(-1)^{y_1 \cdot u} \eta_{D_G, D_H}^{c, y, u}}{2^{k_0}} \left| c, y \oplus \perp \right\rangle_{D_G, D_H \cup (s^*, y_2)} \right\|^2 \\
&= \frac{16q_G}{2^{k_0}} \sum_{\substack{c, y, D_G, D_H, c \neq c^* \\ \exists t \text{ s.t. } f_{pk}(s^*, t) = c \\ \exists r \text{ s.t. } [D_G(r)]_{k_1} = [s^*]_{k_1} \\ D_H(s^*) = \perp, n(D_H) \leq q_H - 1}} \sum_{y_1 \in L_{D_G}^c} \left\| \sum_{u \in \{0, 1\}^{k_0}} \frac{\eta_{D_G, D_H}^{c, y, u}}{\sqrt{2^{k_0}}} \right\|^2 \stackrel{(k)}{\leq} 16q_G \cdot 2^{-k_0} \|\Phi'_3\|^2.
\end{aligned} \tag{47}$$

Here (j) first uses Lemma 10 and then follows the definition of $|\gamma'_2[w, \nu]\rangle_{D_G, D_H}^{c,y}$ and $|\gamma'_3[w, \nu]\rangle_{D_G, D_H}^{c,y}$ as shown in Eq. (44), while (k) follows from the definition of $|\Phi'_3\rangle$. Thus, by combining Eq. (45), (46) and (47), for any unit joint state $|\Phi\rangle$, we finally obtain

$$\begin{aligned} \|(\mathbf{U}_{\text{Search}}^* \circ \mathbf{S}_{s^*} - \mathbf{S}_{s^*} \circ \mathbf{U}_{\text{Search}}^*)|\Phi\rangle\| &\stackrel{(l)}{\leq} \sum_{i=1}^4 \|(\mathbf{U}_{\text{Search}}^* \circ \mathbf{S}_{s^*} - \mathbf{S}_{s^*} \circ \mathbf{U}_{\text{Search}}^*)|\Phi'_i\rangle\| \\ &\leq 2 \cdot \sqrt{q_G} \cdot \sqrt{2^{-k_0}} (\|\Phi'_2\rangle\| + \|\Phi'_4\rangle\| + 2\|\Phi'_3\rangle\|) \\ &\stackrel{(m)}{\leq} 8 \cdot \sqrt{q_G} \cdot \sqrt{2^{-k_0}} \|\Phi\rangle\| = 8\sqrt{q_G} \cdot \sqrt{2^{-k_0}}. \end{aligned}$$

Here (l) uses the triangle inequality, and (m) comes from the fact that $\|\Phi'_i\rangle\| \leq \|\Phi\rangle\|$ for $i \in [4]$. Now we prove that $\|[\mathbf{U}_{\text{Search}}^*, \text{StdDecomp}_{s^*}]\| \leq 8\sqrt{q_G} \cdot \sqrt{2^{-k_0}}$.

D.7 Proof of Lemma 5

As shown in Fig. 8, game $\mathbf{G}_{13}$ is the same as game $\mathbf{G}_{12}$, except that the performing of StdDecomp_{r^*} (resp. StdDecomp_{s^*}) just after the performing of $\mathbf{U}_{g^*}^*$ (resp. $\mathbf{U}_{h^*}^*$) is canceled.

In game $\mathbf{G}_{12}$, we have the following four observations about StdDecomp_{r^*} and StdDecomp_{s^*} :

1. StdDecomp_{r^*} must commute with StdDecomp_{s^*} and $\mathbf{U}_{h^*}^*$ since they act on different database registers, meanwhile StdDecomp_{r^*} and StdDecomp_{s^*} both commute with the adversary's operations since the adversary cannot perform operations on the database registers.
2. StdDecomp_{r^*} must commute with the oracle queries to $\mathbf{G}$ performed during the generation of b' , since $\text{CStO}_G^{r^*, g^*}$ used to simulate $\mathbf{G}$ does not depend on the database state at point r^* .
3. StdDecomp_{s^*} must commute with the oracle queries to $\mathbf{H}$ performed during the generation of b' , since $\text{CStO}_H^{s^*, h^*}$ used to simulate $\mathbf{H}$ does not depend on the database state at point s^* .
4. StdDecomp_{r^*} and StdDecomp_{s^*} must commute with the oracle queries to $\mathbf{H}$ and $\mathbf{G}$ performed during the generation of b' , respectively, since they act on different registers.

In addition, as shown in Lemma 4, StdDecomp_{r^*} and StdDecomp_{s^*} both almost commute with the queries to $\mathbf{qDec}$ performed during the generation of b' in game $\mathbf{G}_{12}$. Note that $\mathbf{qDec}$ is queried at most q_D times, hence with an error of $8q_D \cdot \sqrt{2^{-k_1}} + 8q_D \sqrt{q_G} \cdot \sqrt{2^{-k_0}}$, we can move StdDecomp_{r^*} and StdDecomp_{s^*} to the end of the game and then remove them without affecting the adversary's output. That is to say, our game $\mathbf{G}_{13}$ given above actually satisfies that

$$|\Pr[1 \leftarrow \mathbf{G}_{12}] - \Pr[1 \leftarrow \mathbf{G}_{13}]| \leq 8q_D \cdot \sqrt{2^{-k_1}} + 8q_D \sqrt{q_G} \cdot \sqrt{2^{-k_0}}.$$

D.8 Proof of Lemma 6

We first introduce the Fixed-Permutation One-Way to Hiding theorem which will be used in this proof. In fact, the O2H theorems introduced in Section 2.3 take the quantum accessible random function H as a unitary operation that maps $|x, y\rangle$ into $|x, y \oplus H(x)\rangle$. In [22], the quantum accessible permutation P , which is implemented by a more concise unitary operation $|x\rangle \mapsto |P(x)\rangle$ ²³, is considered. Based on this unitary operation tailored to the permutation, [22] proved the following new O2H theorem:

Theorem 6 (Fixed-Permutation O2H [22, Theorem 1]). *Let z be a random bitstring. Let $P_0, P_1 : \mathcal{X} \rightarrow \mathcal{X}$ be two fixed permutations. Let $\mathcal{A}$ be a quantum oracle algorithm making at most q queries. Let*

$$P_{\text{left}} := \Pr[b = 1 : b \leftarrow \mathcal{A}^{P_0}(z)], \quad P_{\text{right}} := \Pr[b = 1 : b \leftarrow \mathcal{A}^{P_1}(z)].$$

Here oracle P_0 (resp. P_1) is implemented as $|x\rangle \mapsto |P_0(x)\rangle$ (resp. $|x\rangle \mapsto |P_1(x)\rangle$).

Let $\mathcal{B}^{P_1}$ be an algorithm that on input z , chooses $i \leftarrow_{\$} [q]$, runs $\mathcal{A}^{P_1}$ until (just before) the i -th query, measures the query register and outputs the measurement outcome w . Let $P_{\text{guess}} := \Pr[P_0(w) \neq P_1(w) : w \leftarrow \mathcal{B}^{P_1}(z)]$. Then

$$|P_{\text{left}} - P_{\text{right}}| \leq 2q \cdot \sqrt{P_{\text{guess}}} \quad \text{and} \quad \left| \sqrt{P_{\text{left}}} - \sqrt{P_{\text{right}}} \right| \leq 2q \cdot \sqrt{P_{\text{guess}}}.$$

Next, we present our proof. Recall that $\mathcal{C}$ denotes the ciphertext space and $\mathcal{M} = \{0, 1\}^n$ denotes the message space. Define $\mathcal{M}_{\perp} := \mathcal{M} \cup \{\perp\} \cup \{y \oplus \perp : y \in \mathcal{M}\}$ ²⁴. Let S_{pk} be the set of all possible pk generated by Gen . Now we define three fixed permutations on the set

$$\mathcal{C} \times \mathcal{M}_{\perp} \times \mathbf{D}_{q_G} \times \mathbf{D}_{q_H} \times S_{\text{pk}} \times \{0, 1\}^{k_0} \times \{0, 1\}^{n+k_1} \times \{0, 1\}^{n+k_1} \times \{0, 1\}^{k_0} \times \mathcal{C}$$

as follows:

- Permutation P_{Search} : Transform the tuple $(c, y, D_G, D_H, \text{pk}, r, g, s, h, c_1)$ into $(c, y \oplus \text{Search}(\text{pk}, c, D_G, D_H), D_G, D_H, \text{pk}, r, g, s, h, c_1)$, where Search is the classical function defined in Eq. (11).
- Permutation P_G : Transform the tuple $(c, y, D_G, D_H, \text{pk}, r, g, s, h, c_1)$ into $(c, y, D'_G, D_H, \text{pk}, r, g, s, h, c_1)$, where $D'_G = D_G \cup (r, g)$ if $D_G(r) = \perp$, $D'_G = D_G \setminus (r, g)$ if $D_G(r) = g$, and $D'_G = D_G$ otherwise.
- Permutation P_H : Transform the tuple $(c, y, D_G, D_H, \text{pk}, r, g, s, h, c_1)$ into $(c, y, D_G, D'_H, \text{pk}, r, g, s, h, c_1)$, where $D'_H = D_H \cup (s, h)$ if $D_H(r) = \perp$, $D'_H = D_H \setminus (s, h)$ if $D_H(s) = h$, and $D'_H = D_H$ otherwise.

Then we define two fixed permutations P_0 and P_1 as follows:

²³ We say that this unitary operation is more concise, because it does not need an additional register to store the output value.

²⁴ In fact, as shown in Supplementary Material B, this set is used when we quantumly encode the set $\mathcal{M} \cup \{\perp\}$.

- Permutation P_0 : Transform the tuple $(c, y, D_G, D_H, \text{pk}, r, g, s, h, c_1)$ into the tuple $(c, y \oplus \perp, D_G, D_H, \text{pk}, r, g, s, h, c_1)$ if $c = c_1$. Otherwise, perform P_{Search} .
- Permutation P_1 : Transform the tuple $(c, y, D_G, D_H, \text{pk}, r, g, s, h, c_1)$ into the tuple $(c, y \oplus \perp, D_G, D_H, \text{pk}, r, g, s, h, c_1)$ if $c = c_1$. Otherwise, sequentially perform $P_G, P_H, P_{\text{search}}, P_G, P_H$.

At this point, based on the above permutations P_0 and P_1 , and parameters $(\text{pk}, r^*, g^*, s^*, h^*, c^*)$ introduced in game $\mathbf{G}_{14}$ and game $\mathbf{G}_{15}$, we can conclude that the U_{Search}^* utilized in game $\mathbf{G}_{14}$ must satisfy that $U_{\text{Search}}^*|c, y, D_G, D_H\rangle = |c', y', D'_G, D'_H\rangle$ if and only if

$$P_0(c, y, D_G, D_H, \text{pk}, r^*, g^*, s^*, h^*, c^*) = (c', y', D'_G, D'_H, \text{pk}, r^*, g^*, s^*, h^*, c^*).$$

Moreover we can conclude that the $U_{h^*}^{s^*} \circ U_{g^*}^{r^*} \circ U_{\text{Search}}^* \circ U_{h^*}^{s^*} \circ U_{g^*}^{r^*}$ utilized in game $\mathbf{G}_{15}$ must satisfy $U_{h^*}^{s^*} \circ U_{g^*}^{r^*} \circ U_{\text{Search}}^* \circ U_{h^*}^{s^*} \circ U_{g^*}^{r^*}|c, y, D_G, D_H\rangle = |c', y', D'_G, D'_H\rangle$ if and only if

$$P_1(c, y, D_G, D_H, \text{pk}, r^*, g^*, s^*, h^*, c^*) = (c', y', D'_G, D'_H, \text{pk}, r^*, g^*, s^*, h^*, c^*).$$

Here we design a new quantum oracle algorithm $\mathcal{A}_1$, which has oracle access to $P_0 : |x\rangle \mapsto |P_0(x)\rangle$ and runs game $\mathbf{G}_{14}$. Specifically, when $\mathcal{A}$ queries qDec by $\sum |c, y, D_G, D_H\rangle^{25}$ on the registers $\text{IOD}_{q_G} D_{q_H}$ during the generation of b' , $\mathcal{A}_1$ first expands this state into $\sum |c, y, D_G, D_H\rangle \otimes |\text{pk}, r^*, g^*, s^*, h^*, c^*\rangle$ and then forwards it to its oracle P_0 . Note that such expansion is feasible because all of the elements in tuple $(\text{pk}, r^*, g^*, s^*, h^*, c^*)$ have been generated after the computation of c^* . In fact, by the two conclusions mentioned above, $\mathcal{A}_1$ perfectly simulates the procedure of $\mathcal{A}$ querying qDec during the generation of b' , thus we have

$$\Pr[1 \leftarrow \mathbf{G}_{14}] = \Pr[1 \leftarrow \mathcal{A}_1^{P_0}].$$

Similarly, if we change the oracle $P_0 : |x\rangle \mapsto |P_0(x)\rangle$ to $P_1 : |x\rangle \mapsto |P_1(x)\rangle$, we can obtain

$$\Pr[1 \leftarrow \mathbf{G}_{15}] = \Pr[1 \leftarrow \mathcal{A}_1^{P_1}].$$

Assume that during the generation of b' , $\mathcal{A}$ makes q'_D queries to qDec . Then it is obvious that $\mathcal{A}_1$ makes q'_D queries to oracle P_0 or P_1 .

Now we construct game $\mathbf{G}_{15}^1$ as shown in Fig. 13, which chooses $i \leftarrow_{\$} [q'_D]$, runs $\mathcal{A}$ in game $\mathbf{G}_{15}$ until (just before) its i -th query to qDec during the generation of b' , measures the registers $\text{IOD}_{q_G} D_{q_H}$ to get the outcome (c, y, D_G, D_H) , and then extends it to $w := (c, y, D_G, D_H, \text{pk}, r^*, g^*, s^*, h^*, c^*)$. If $P_0(w) \neq P_1(w)$, game $\mathbf{G}_{15}^1$ outputs 1; otherwise outputs 0. Then by applying Theorem 6 and the fact that $q'_D \leq q_D$, we have

$$|\Pr[1 \leftarrow \mathbf{G}_{14}] - \Pr[1 \leftarrow \mathbf{G}_{15}]| \leq 2q_D \sqrt{\Pr[1 \leftarrow \mathbf{G}_{15}^1]}. \quad (48)$$

Next, we construct games $\mathbf{G}_{15}^2$ to $\mathbf{G}_{15}^9$, which aim at providing an upper bound for the probability $\Pr[1 \leftarrow \mathbf{G}_{15}^1]$. All of these games are shown in Fig. 13.

²⁵ Note that $\mathcal{A}$ can only control registers IO and the coefficient of this state is omit, so this is actually an informal description. Without loss of generality, such informal (but

GAMES $\mathbf{G}_{15}^1$ to $\mathbf{G}_{15}^9$	
1. $(pk, sk) \leftarrow \text{Gen}, r^* \leftarrow_{\$} \{0, 1\}^{k_0}, s^* \leftarrow_{\$} \{0, 1\}^{n+k_1}, t^* \leftarrow_{\$} \{0, 1\}^{k_0}, c^* = f_{pk}(s^*, t^*)$	// $\mathbf{G}_{15}^1$ - $\mathbf{G}_{15}^9$
2. $(m_0^*, m_1^*) \leftarrow \mathcal{A}^{G,H,qDec}(pk)$	// $\mathbf{G}_{15}^1$ - $\mathbf{G}_{15}^9$
3. $b \leftarrow_{\$} \{0, 1\}$	// $\mathbf{G}_{15}^1$ - $\mathbf{G}_{15}^9$
$g^* = (m_b^* 0^{k_1}) \oplus s^*, h^* = r^* \oplus t^*$, perform $U_{h^*}^{s^*} \circ U_{g^*}^{r^*}$	// $\mathbf{G}_{15}^1$
4. $i \leftarrow_{\$} [qD]$, run $b' \leftarrow \mathcal{A}^{qDec,G,H}(pk, c^*)$ until i -th qDec query	// $\mathbf{G}_{15}^1$ - $\mathbf{G}_{15}^9$
measure registers $\text{IOD}_{qG} D_{qH}$ to get (c, y, D_G, D_H)	// $\mathbf{G}_{15}^1$ - $\mathbf{G}_{15}^9$
compute $ c, y, D'_G, D'_H\rangle := U_{h^*}^{s^*} \circ U_{g^*}^{r^*} c, y, D_G, D_H\rangle$	// $\mathbf{G}_{15}^2$ - $\mathbf{G}_{15}^4$
extend (c, y, D_G, D_H) to $w := (c, y, D_G, D_H, pk, r^*, g^*, s^*, h^*, c^*)$	// $\mathbf{G}_{15}^1$
extend (c, y, D'_G, D'_H) to $w := (c, y, D'_G, D'_H, pk, r^*, g^*, s^*, h^*, c^*)$	// $\mathbf{G}_{15}^2$ - $\mathbf{G}_{15}^4$
5. return $\llbracket P_0(w) \neq P_1(w) \rrbracket$	// $\mathbf{G}_{15}^1$ - $\mathbf{G}_{15}^4$
return $\llbracket (c, y, D_G, D_H) \in R \rrbracket$	// $\mathbf{G}_{15}^5$ - $\mathbf{G}_{15}^9$
$G(x_G, y_G\rangle)$	$H(x_H, y_H\rangle)$
if (m_0^*, m_1^*) has not been defined	if (m_0^*, m_1^*) has not been defined
apply CStO_G	apply CStO_H
// $\mathbf{G}_{15}^1$ - $\mathbf{G}_{15}^2$	// $\mathbf{G}_{15}^1$ - $\mathbf{G}_{15}^3$
apply $\text{CStO}_G \circ \mathcal{O}_{\{r^*\}}^{SC}$	apply $\text{CStO}_H \circ \mathcal{O}_{\{s^*\}}^{SC}$
// $\mathbf{G}_{15}^3$ - $\mathbf{G}_{15}^5$	// $\mathbf{G}_{15}^4$ - $\mathbf{G}_{15}^5$
apply CStO_G	apply CStO_H
// $\mathbf{G}_{15}^6$ - $\mathbf{G}_{15}^9$	// $\mathbf{G}_{15}^6$ - $\mathbf{G}_{15}^9$
if Find_{r^*} occurs, return $\perp$	if Find_{s^*} occurs, return $\perp$
else	else
apply $\text{CStO}_G^{r^*, g^*}$	apply $\text{CStO}_H^{s^*, h^*}$
// $\mathbf{G}_{15}^1$ - $\mathbf{G}_{15}^6$	// $\mathbf{G}_{15}^1$ - $\mathbf{G}_{15}^6$
apply $\text{CStO}_G^{r^*, g^*} \circ \mathcal{O}_{\{r^*, s^*\}}^{SC}$	apply $\text{CStO}_H^{s^*, h^*} \circ \mathcal{O}_{\{r^*, s^*\}}^{SC}$
// $\mathbf{G}_{15}^7$	// $\mathbf{G}_{15}^7$
apply $\text{CStO}_G \circ \mathcal{O}_{\{r^*, s^*\}}^{SC}$	apply $\text{CStO}_H \circ \mathcal{O}_{\{r^*, s^*\}}^{SC}$
// $\mathbf{G}_{15}^8$	// $\mathbf{G}_{15}^8$
if Find_{r^*, s^*} occurs, return $\perp$	if Find_{r^*, s^*} occurs, return $\perp$
apply CStO_G	apply CStO_H
// $\mathbf{G}_{15}^9$	// $\mathbf{G}_{15}^9$
$qDec(c, y\rangle)$	$qDec(c, y\rangle)$
if c^* has not been defined, apply U_{Search}	// $\mathbf{G}_{15}^1$
else apply U_{Search}^{**}	// $\mathbf{G}_{15}^1$
	// $\mathbf{G}_{15}^2$ - $\mathbf{G}_{15}^9$
	if c^* has not been defined, apply U_{Search}
	else apply U_{Search}^*

Fig. 13. Games $\mathbf{G}_{15}^1$ to $\mathbf{G}_{15}^9$ in the proof of Lemma 6. Here different from game $\mathbf{G}_{14}$ and game $\mathbf{G}_{15}$, the games shown in this figure modifies the computation of $c^* = f_{pk}(s^*, t^*)$ to be placed just after the generation of (pk, sk) , which is acceptable since s^* and t^* are both uniformly sampled independent of the adversary.

Game $\mathbf{G}_{15}^2$: Compared with game $\mathbf{G}_{15}^1$, this game has the following three changes:

1. The computing of g^* and h^* and the performing of $U_{h^*}^{s^*} \circ U_{g^*}^{r^*}$ just after the sampling of b are canceled.
2. The U_{Search}^{**} used to simulate $\mathbf{qDec}$ is changed back to U_{Search}^* .
3. After getting the final measurement outcome (c, y, D_G, D_H) , game $\mathbf{G}_{15}^2$ additionally computes $U_{h^*}^{s^*} \circ U_{g^*}^{r^*} |c, y, D_G, D_H\rangle = |c, y, D'_G, D'_H\rangle$ and then outputs $\llbracket P_0(c, y, D'_G, D'_H, \text{pk}, r^*, g^*, s^*, h^*, c^*) \neq P_1(c, y, D'_G, D'_H, \text{pk}, r^*, g^*, s^*, h^*, c^*) \rrbracket$.

Follow a similar analysis used to deduce Eq. (7) in the game hop from $\mathbf{G}_{15}$ to $\mathbf{G}_{16}$, the $U_{g^*}^{r^*}$ and $U_{h^*}^{s^*}$ additionally introduced in U_{Search}^{**} can be canceled out, but the last performing of $U_{g^*}^{r^*}$ and $U_{h^*}^{s^*}$ cannot directly remove²⁶ and hence for consistency, the above third change additionally computes $U_{h^*}^{s^*} \circ U_{g^*}^{r^*} |c, y, D_G, D_H\rangle$. That is to say, we have

$$\Pr[1 \leftarrow \mathbf{G}_{15}^1] = \Pr[1 \leftarrow \mathbf{G}_{15}^2]. \quad (49)$$

Game $\mathbf{G}_{15}^3$: Compared with game $\mathbf{G}_{15}^2$, this game has the following two changes:

1. Before $\mathcal{A}$ outputs (m_0^*, m_1^*) , $\mathbf{G}$ is simulated by $\text{CStO}_G \circ \mathcal{O}_{\{r^*\}}^{SC}$, which is an operation that first queries the semi-classical oracle $\mathcal{O}_{\{r^*\}}^{SC}$ (Definition 1) on the input register of $\mathbf{G}$ and then performs CStO_G .
2. If Find_{r^*} occurs before $\mathcal{A}$ outputs (m_0^*, m_1^*) , the game finally returns $\perp$. Here Find_{r^*} is the event that the measurement of $\mathcal{O}_{\{r^*\}}^{SC}$ ever has result 1.

Following a similar analysis used to deduce Eq. (14) in the game hop from $\mathbf{G}_5$ to $\mathbf{G}_6$ and use Theorem 1 with the form $|\sqrt{P_{\text{left}}} - \sqrt{P_{\text{right}}}|$, we have

$$\begin{aligned} \left| \sqrt{\Pr[1 \leftarrow \mathbf{G}_{15}^2]} - \sqrt{\Pr[1 \leftarrow \mathbf{G}_{15}^3]} \right| &\leq \sqrt{(q_G + 1) \cdot \Pr[\text{Find}_{r^*} : \mathbf{G}_{15}^3]} \\ &\stackrel{(a)}{\leq} 2(q_G + 1) \cdot \sqrt{2^{-k_0}}. \end{aligned} \quad (50)$$

Here (a) uses Lemma 3 and $\Pr[\text{Find}_{r^*} : \mathbf{G}_{15}^3] = \Pr[\text{Find}_{r^*} : \mathbf{G}_6]$. In fact, this equality is obvious since the first two steps of games $\mathbf{G}_{15}^3$ and $\mathbf{G}_6$ are identical.

Game $\mathbf{G}_{15}^4$: Compared with game $\mathbf{G}_{15}^3$, this game has the following two changes:

1. Before $\mathcal{A}$ outputs (m_0^*, m_1^*) , $\mathbf{H}$ is simulated by $\text{CStO}_H \circ \mathcal{O}_{\{s^*\}}^{SC}$, which is an operation first queries the semi-classical oracle $\mathcal{O}_{\{s^*\}}^{SC}$ (Definition 1) on the input register of $\mathbf{H}$ and then performs CStO_H .

convenient) description used here is acceptable and will not cause any ambiguity in the subsequent proof.

²⁶ In the game hop from $\mathbf{G}_{15}$ to $\mathbf{G}_{16}$, the last $U_{g^*}^{r^*}$ and $U_{h^*}^{s^*}$ can be directly remove without affecting $\mathcal{A}$'s output because $\mathcal{A}$ does not measure the database registers. However, the final measurement of game $\mathbf{G}_{15}^1$ needs to measure the the database registers, which makes that we cannot directly remove the last performing of $U_{g^*}^{r^*}$ and $U_{h^*}^{s^*}$.

2. If Find_{s^*} occurs before $\mathcal{A}$ outputs (m_0^*, m_1^*) , the game finally returns $\perp$. Here Find_{s^*} is the event that the measurement of $\mathcal{O}_{\{s^*\}}^{SC}$ ever has result 1.

Following a similar analysis used to deduce Eq. (18) in the game hop from $\mathbf{G}_8$ to $\mathbf{G}_9$ and use Theorem 1 with the form $|\sqrt{P_{\text{left}}} - \sqrt{P_{\text{right}}}|$, we have

$$\begin{aligned} \left| \sqrt{\Pr[1 \leftarrow \mathbf{G}_{15}^3]} - \sqrt{\Pr[1 \leftarrow \mathbf{G}_{15}^4]} \right| &\leq \sqrt{(q_H + 1) \cdot \Pr[\text{Find}_{s^*} : \mathbf{G}_{15}^4]} \\ &\stackrel{(b)}{\leq} 2(q_H + 1) \cdot \sqrt{2^{-n-k_1}}. \end{aligned} \quad (51)$$

Here (b) uses Lemma 3 and $\Pr[\text{Find}_{s^*} : \mathbf{G}_{15}^4] = \Pr[\text{Find}_{s^*} : \mathbf{G}_9]$. In fact, this equality is obvious since the first two steps of games $\mathbf{G}_{15}^4$ and $\mathbf{G}_9$ are identical.

In game $\mathbf{G}_{15}^4$, under the condition that Find_{r^*} and Find_{s^*} both do not occur, there will be no record of $(r^*, D_G(r^*))$ or $(s^*, D_H(s^*))$ in the database state before $\mathcal{A}$ outputs (m_0^*, m_1^*) . Furthermore, during the generation of b' , we note that the unitary operations $\text{CStO}_G^{r^*, g^*} / \text{CStO}_H^{s^*, h^*}$ used to simulate G/H will not update the database pair $(r^*, D_G(r^*))$ or $(s^*, D_H(s^*))$, so until the final measurement of game $\mathbf{G}_{15}^4$ there will be no record of $(r^*, D_G(r^*) \neq \perp)$ or $(s^*, D_H(s^*) \neq \perp)$ in the database state. That is to say, for the outcome (c, y, D_G, D_H) of the measurement on qDec 's query state, we have $D_G(r^*) = \perp$ and $D_H(s^*) = \perp$. Then after performing $U_{h^*}^{s^*} \circ U_{g^*}^{r^*} |c, y, D_G, D_H\rangle$ to get $|c, y, D'_G, D'_H\rangle$, one can check that

$$(c, y, D'_G, D'_H) = (c, y, D_G \cup (r^*, s^* \oplus (m_b^* || 0^{k_1})), D_H \cup (s^*, r^* \oplus t^*)).$$

As for the final judgment criteria $P_0(w) \neq P_1(w)$ of game $\mathbf{G}_{15}^4$, by the above observation, we can derive an implication relationship as:

$$P_0(w) \neq P_1(w) \implies \begin{pmatrix} (c, y, D_G, D_H) \text{ satisfies} \\ \exists (s, D_H(s) \neq \perp) \text{ s.t. } c = f_{\text{pk}}(s, r^* \oplus D_H(s)) \\ \text{or } \exists (r, D_G(r) \neq \perp) \text{ s.t. } c = f_{\text{pk}}(s^*, r \oplus r^* \oplus t^*) \end{pmatrix}$$

Game $\mathbf{G}_{15}^5$: Compared with game $\mathbf{G}_{15}^4$, this game has the following two changes:

1. After the final measurement on the query state of qDec , cancel the performing of $U_{h^*}^{s^*} \circ U_{g^*}^{r^*}$.
2. Alter the judgment criteria for game outputting 1 from $P_0(w) \neq P_1(w)$ to $(c, y, D_G, D_H) \in R$, where

$$R := \left\{ (c, y, D_G, D_H) : \begin{array}{l} \exists (s, D_H(s) \neq \perp) \text{ s.t. } c = f_{\text{pk}}(s, r^* \oplus D_H(s)) \\ \text{or } \exists (r, D_G(r) \neq \perp) \text{ s.t. } c = f_{\text{pk}}(s^*, r \oplus r^* \oplus t^*) \end{array} \right\}. \quad (52)$$

According to the analysis given above, we have

$$\Pr[1 \leftarrow \mathbf{G}_{15}^4] \leq \Pr[1 \leftarrow \mathbf{G}_{15}^5]. \quad (53)$$

Game $\mathbf{G}_{15}^6$: Before $\mathcal{A}$ outputs (m_0^*, m_1^*) , the queries to the semi-classical oracles $\mathcal{O}_{\{r^*\}}^{SC}$ and $\mathcal{O}_{\{s^*\}}^{SC}$ performed in game $\mathbf{G}_{15}^5$ are deleted, meanwhile the two flags

"Find $_{r^*}$ occurs, game finally returns $\perp$ " and "Find $_{s^*}$ occurs, game finally returns $\perp$ " are both removed.

In fact, the changes introduced in game $\mathbf{G}_{15}^6$ is a combination of those in games $\mathbf{G}_{15}^3$ and $\mathbf{G}_{15}^4$, but are applied in the opposite direction. Hence similar to the analysis given in game hop from $\mathbf{G}_{15}^2$ to $\mathbf{G}_{15}^3$ and $\mathbf{G}_{15}^3$ to $\mathbf{G}_{15}^4$, we have

$$\left| \sqrt{\Pr[1 \leftarrow \mathbf{G}_{15}^5]} - \sqrt{\Pr[1 \leftarrow \mathbf{G}_{15}^6]} \right| \leq 2(q_G+1) \cdot \sqrt{2^{-k_0}} + 2(q_H+1) \cdot \sqrt{2^{-n-k_1}}. \quad (54)$$

Game $\mathbf{G}_{15}^7$: Compared with game $\mathbf{G}_{15}^6$, this game has the following two changes:

1. during the generation of b' , G is simulated by $\text{CStO}_G^{r^*, g^*} \circ \mathcal{O}_{\{r^*, s^*\}}^{SC}$, which is an operation first queries the semi-classical oracle $\mathcal{O}_{\{r^*, s^*\}}^{SC}$ (Definition 1) on the input register of G , and then performs $\text{CStO}_G^{r^*, g^*}$. Similarly, H is simulated by $\text{CStO}_H^{s^*, h^*} \circ \mathcal{O}_{\{r^*, s^*\}}^{SC}$.
2. If Find $_{r^*, s^*}$ occurs during the generation of b' , the game finally returns $\perp$. Here Find $_{r^*, s^*}$ is the event the measurement of $\mathcal{O}_{\{r^*, s^*\}}^{SC}$ ever has result 1.

We first construct a quantum oracle algorithm $\mathcal{A}_2$ as follows, which has oracle access to a constant zero function F .

- $\mathcal{A}_2$ runs game $\mathbf{G}_{15}^6$. Specifically, when $\mathcal{A}$ queries oracle G (resp. H) by a (informal) query state $\sum |x, y\rangle_{X_G Y_G}$ (resp. $\sum |x, y\rangle_{X_H Y_H}$) after it outputs (m_0^*, m_1^*) , $\mathcal{A}_2$ will perform an additional operation: first expand this query state into $\sum |x, y\rangle \otimes_{X_G Y_G} |0\rangle_{O_F}$ (resp. $\sum |x, y\rangle \otimes_{X_H Y_H} |0\rangle_{O_F}$) and then forward the input/output register X_G/O_F (resp. X_H/O_F) to its function oracle F .

Obviously $\mathcal{A}_2$ makes at most $q_G + q_H$ queries to its oracle, and such additional queries do not disturb $\mathcal{A}$'s computation since F is a constant zero function. Hence,

$$\Pr[1 \leftarrow \mathbf{G}_{15}^6] = \Pr \left[b = 1 : \begin{array}{l} b \leftarrow \mathcal{A}_2^F(r^*, s^*, z), z \leftarrow D_{\mathcal{A}_2} \\ r^* \leftarrow_{\$} \{0, 1\}^{k_0}, s^* \leftarrow_{\$} \{0, 1\}^{n+k_1} \end{array} \right].$$

Here we particularly write r^*, s^* and assume (r^*, s^*, z) as the classical input of $\mathcal{A}_2$ without loss of generality, where $r^* \leftarrow_{\$} \{0, 1\}^{k_0}$, $s^* \leftarrow_{\$} \{0, 1\}^{n+k_1}$ and z is sampled according to a distribution $D_{\mathcal{A}_2}$. Then, if we change F into $F \setminus \{r^*, s^*\}$, i.e., the oracle that first queries $\mathcal{O}_{\{r^*, s^*\}}^{SC}$ on the input register of F and then queries F , it is easy to check that

$$\Pr[1 \leftarrow \mathbf{G}_{15}^7] = \Pr \left[(b = 1) \wedge \neg \text{Find}_{r^*, s^*} : \begin{array}{l} b \leftarrow \mathcal{A}_2^{F \setminus \{r^*, s^*\}}(r^*, s^*, z), z \leftarrow D_{\mathcal{A}_2} \\ r^* \leftarrow_{\$} \{0, 1\}^{k_0}, s^* \leftarrow_{\$} \{0, 1\}^{n+k_1} \end{array} \right].$$

Let D be a joint distribution of $(E, F, \{r^*, s^*\}, z)$, where E, F are both constant zero functions, $r^* \leftarrow_{\$} \{0, 1\}^{k_0}$, $s^* \leftarrow_{\$} \{0, 1\}^{n+k_1}$ and $z \leftarrow D_{\mathcal{A}_2}$. Now by applying Theorem 1 with the form of $|\sqrt{P_{\text{left}}} - \sqrt{P_{\text{right}}}|$ under this joint distribution D ,

we have

$$\begin{aligned}
& \left| \sqrt{\Pr[1 \leftarrow \mathbf{G}_{15}^6]} - \sqrt{\Pr[1 \leftarrow \mathbf{G}_{15}^7]} \right| \\
& \leq \sqrt{(q_G + q_H + 1) \cdot \Pr[\text{Find}_{r^*, s^*} : \mathcal{A}_2^{\mathbf{F} \setminus \{r^*, s^*\}}(z), (\mathbf{E}, \mathbf{F}, \{r^*, s^*\}, z) \leftarrow D]} \quad (55) \\
& = \sqrt{(q_G + q_H + 1) \cdot \Pr[\text{Find}_{r^*, s^*} : \mathbf{G}_{15}^7]},
\end{aligned}$$

The above final equation follows from the definition of $\mathcal{A}_2$ and game $\mathbf{G}_{15}^7$.

Game $\mathbf{G}_{15}^8$: This game is the same as game $\mathbf{G}_{15}^7$, except that the simulation of $\mathbf{G}$ and $\mathbf{H}$ after the generation of (m_0^*, m_1^*) is changed. In fact, the $\text{CStO}_G^{r^*, s^*}$ is replaced by CStO_G , and the $\text{CStO}_H^{s^*, h^*}$ is replaced by CStO_H .

Note that the situation where game $\mathbf{G}_{15}^7$ returns 1 and game $\mathbf{G}_{15}^8$ returns 1 both imply that the event Find_{r^*, s^*} does not occur, which means that $\mathcal{A}$ only queries $\mathbf{G}$ (resp. $\mathbf{H}$) by a (informal) query state in the form $\sum_{x \neq r^*} |x, y\rangle_{\mathbf{X}_G \mathbf{Y}_G}$ (resp. $\sum_{x \neq s^*} |x, y\rangle_{\mathbf{X}_H \mathbf{Y}_H}$). So if Find_{r^*, s^*} does not occur, the change from game $\mathbf{G}_{15}^7$ to game $\mathbf{G}_{15}^8$ cannot be detected by the adversary. Hence, we have

$$\Pr[\text{Find}_{r^*, s^*} : \mathbf{G}_{15}^7] = \Pr[\text{Find}_{r^*, s^*} : \mathbf{G}_{15}^8], \quad (56)$$

and

$$\Pr[1 \leftarrow \mathbf{G}_{15}^7] = \Pr[1 \leftarrow \mathbf{G}_{15}^8]. \quad (57)$$

Game $\mathbf{G}_{15}^9$: During the generation of b' , the query to the semi-classical oracle $\mathcal{O}_{\{r^*, s^*\}}^{SC}$ performed in game $\mathbf{G}_{15}^8$ is deleted, meanwhile the flag " Find_{r^*, s^*} occurs, game finally returns $\perp$ " is removed.

Using a similar analysis of the game hop from $\mathbf{G}_{15}^6$ to $\mathbf{G}_{15}^7$, we have

$$\left| \sqrt{\Pr[1 \leftarrow \mathbf{G}_{15}^8]} - \sqrt{\Pr[1 \leftarrow \mathbf{G}_{15}^9]} \right| \leq \sqrt{(q_G + q_H + 1) \cdot \Pr[\text{Find}_{r^*, s^*} : \mathbf{G}_{15}^8]}. \quad (58)$$

Next we compute the upper bound of $\Pr[\text{Find}_{r^*, s^*} : \mathbf{G}_{15}^8]$ and $\Pr[1 \leftarrow \mathbf{G}_{15}^9]$.

For the $\Pr[\text{Find}_{r^*, s^*} : \mathbf{G}_{15}^8]$, we assume without loss of generality that, the query number of $\mathcal{O}_{\{r^*, s^*\}}^{SC}$ in game $\mathbf{G}_{15}^8$ is exactly $q_G + q_H$ (if it does not, we can modify game $\mathbf{G}_{15}^8$ by adding a number of useless queries to $\mathcal{O}_{\{r^*, s^*\}}^{SC}$ in the end). Hence we can directly construct a quantum oracle algorithm $\mathcal{A}_3(\text{pk}, c^*)$, which queries $\mathcal{O}_{\{r^*, s^*\}}^{SC}$ with times $q_G + q_H$ and uses its input (pk, c^*) to run game $\mathbf{G}_{15}^8$. Now we have

$$\Pr[\text{Find}_{r^*, s^*} : \mathbf{G}_{15}^8] = \Pr \left[\begin{array}{l} \mathcal{A}_3^{\mathcal{O}_{\{r^*, s^*\}}^{SC}}(\text{pk}, c^*), (\text{pk}, \text{sk}) \leftarrow \text{Gen} \\ \text{Find}_{r^*, s^*} : \begin{array}{l} s^* \leftarrow_{\$} \{0, 1\}^{n+k_1}, t^* \leftarrow_{\$} \{0, 1\}^{k_0} \\ r^* \leftarrow_{\$} \{0, 1\}^{k_0}, c^* = f_{\text{pk}}(s^*, t^*) \end{array} \end{array} \right]. \quad (59)$$

Let D be a joint distribution of $(\{r^*, s^*\}, \text{pk}, c^*)$, where $r^*, t^* \leftarrow_{\$} \{0, 1\}^{k_0}$, $s^* \leftarrow_{\$} \{0, 1\}^{n+k_1}$, pk is sampled by $(\text{pk}, \text{sk}) \leftarrow \text{Gen}$ and $c^* = f_{\text{pk}}(s^*, t^*)$. By applying

Theorem 2 with this joint distribution D , we can construct a quantum algorithm $\mathcal{B}_1$ which first chooses $j \leftarrow_{\$} [q_G + q_H]$, runs $\mathcal{A}_3^{\mathcal{O}^{SC}}(\text{pk}, c^*)$ until its j -th query to $\mathcal{O}_0^{SC}$, measures the input register to get the output x , satisfying:

$$\Pr \left[\begin{array}{l} \text{Find}_{r^*, s^*} : \mathcal{A}_3^{\mathcal{O}^{SC}}(\text{pk}, c^*), (\text{pk}, \text{sk}) \leftarrow \text{Gen} \\ s^* \leftarrow_{\$} \{0, 1\}^{n+k_1}, t^* \leftarrow_{\$} \{0, 1\}^{k_0} \\ r^* \leftarrow_{\$} \{0, 1\}^{k_0}, c^* = f_{\text{pk}}(s^*, t^*) \end{array} \right] \leq 4(q_G + q_H) \cdot \Pr[x \in \{r^*, s^*\} : x \leftarrow \mathcal{B}_1(\text{pk}, c^*), (\{r^*, s^*\}, \text{pk}, c^*) \leftarrow D]. \quad (60)$$

Further we can compute

$$\begin{aligned} & \Pr[x \in \{r^*, s^*\} : x \leftarrow \mathcal{B}_1(\text{pk}, c^*), (\{r^*, s^*\}, \text{pk}, c^*) \leftarrow D] \\ & \leq \Pr[x = r^* : x \leftarrow \mathcal{B}_1(\text{pk}, c^*), (\{r^*, s^*\}, \text{pk}, c^*) \leftarrow D] \\ & \quad + \Pr[x = s^* : x \leftarrow \mathcal{B}_1(\text{pk}, c^*), (\{r^*, s^*\}, \text{pk}, c^*) \leftarrow D] \\ & \stackrel{(a)}{=} 2^{-k_0} + \text{Adv}_{\text{TDL}, \mathcal{B}_1}^{\text{PD-OW}}. \end{aligned} \quad (61)$$

Here (a) follows from two facts: first, under the joint distribution D , (pk, c^*) is independent of r^* , and the other is that, under the joint distribution D , $\mathcal{B}_1$ can be viewed as a PD-OW adversary against TDL if it aims at searching s^* . Since $\mathcal{B}_1$ needs to use the compressed standard oracle technique to simulate G and H , and uses U_{Search} , $\text{U}_{\text{Search}}^*$ to simulate qDec , the running time of $\mathcal{B}_1$ can be upper bounded by $\mathcal{T}_{\mathcal{A}} + \mathcal{O}(q_G^2 + q_H^2) + \mathcal{O}(q_D q_G q_H) \cdot \mathcal{T}_f$. Now by combining Eq. (59) to Eq. (61), we obtain

$$\Pr[\text{Find}_{r^*, s^*} : \mathbf{G}_{15}^8] \leq 4(q_G + q_H) \cdot 2^{-k_0} + 4(q_G + q_H) \cdot \text{Adv}_{\text{TDL}, \mathcal{B}_1}^{\text{PD-OW}}. \quad (62)$$

For the $\Pr[1 \leftarrow \mathbf{G}_{15}^9]$, we first rewrite game $\mathbf{G}_{15}^9$ as a search algorithm $\mathcal{A}_4(\text{pk}, c^*, s^*, t^*)$, where the sampling of $(\text{pk}, c^*, s^*, t^*)$ is the same as that in game $\mathbf{G}_{15}^9$. As for the r^* , which is uniformly sampled from $\{0, 1\}^{k_0}$, is independent of $\mathcal{A}_4$'s view since the running of game $\mathbf{G}_{15}^9$ does not depend on it. Then we have

$$\Pr[1 \leftarrow \mathbf{G}_{15}^9] = \Pr[(c, y, D_G, D_H) \in R : (c, y, D_G, D_H) \leftarrow \mathcal{A}_4(\text{pk}, c^*, s^*, t^*)]. \quad (63)$$

Here set R is defined in Eq. (52), and $(c, y, D_G, D_H) \in R$ actually means that

$$\begin{aligned} & - \exists (s, D_H(s) \neq \perp) \text{ s.t. } c = f_{\text{pk}}(s, r^* \oplus D_H(s)), \\ & - \text{ or } \exists (r, D_G(r) \neq \perp) \text{ s.t. } c = f_{\text{pk}}(s^*, r \oplus r^* \oplus t^*). \end{aligned}$$

For the first case, we can directly recover r^* by first computing $f_{\text{sk}}^{-1}(c)$ to get $(s, r^* \oplus D_H(s))$ and then extracting r^* from $r^* \oplus D_H(s)$. For the second case, we can also recover r^* with a noticeable probability. That is, first compute $f_{\text{sk}}^{-1}(c)$ to get $(s^*, r \oplus r^* \oplus t^*)$, and then use t^* with a uniformly-chosen r from $\{r : D_G(r) \neq \perp\}$ to extract r^* from $r \oplus r^* \oplus t^*$. That is to say, if the (c, y, D_G, D_H) generated by $\mathcal{A}_4$ belongs to set R , we can recover r^* from (c, y, D_G, D_H) even

though $\mathcal{A}_4$'s view is independent of r^* . Recall that r^* is uniformly sampled from $\{0, 1\}^{k_0}$, so through a simple probability analysis we have

$$\Pr[(c, y, D_G, D_H) \in R : (c, y, D_G, D_H) \leftarrow \mathcal{A}_4(\mathbf{pk}, c^*, s^*, t^*)] \leq (q_G + 1) \cdot 2^{-k_0}. \quad (64)$$

Combine the above equation with Eq. (63), we get $\Pr[1 \leftarrow \mathbf{G}_{15}^9] \leq (q_G + 1) \cdot 2^{-k_0}$.

Finally, by combining Eq.(48) to Eq.(64), we obtain

$$\begin{aligned} |\Pr[1 \leftarrow \mathbf{G}_{14}] - \Pr[1 \leftarrow \mathbf{G}_{15}]| &\leq 18q_D(q_G + q_H + 1) \cdot \sqrt{2^{-k_0}} \\ &\quad + 8q_D(q_H + 1) \cdot \sqrt{2^{-n-k_1}} \\ &\quad + 8q_D(q_G + q_H + 1) \cdot \sqrt{\text{Adv}_{\text{TDI}, \mathcal{B}_1}^{\text{PD-OW}}}, \end{aligned}$$

and we complete the proof.

D.9 Proof of Lemma 7

During the generation of b' in game $\mathbf{G}_{15}$ (Fig. 9), G is simulated by the $\text{CStO}_G^{r^*, g^*}$ defined in Eq. (22), which will directly XOR g^* on the output register by CNOT_{g^*} . This means that $\text{CStO}_G^{r^*, g^*}$ will not update the record of database D_G on the pair $(r^*, D_G(r^*))$. Based on this fact, we can conclude that $U_{g^*}^{r^*}$ commutes with $\text{CStO}_G^{r^*, g^*}$ since $U_{g^*}^{r^*}$ defined in Eq. (15) only updates the record of D_G on the pair $(r^*, D_G(r^*))$. Moreover, $U_{g^*}^{r^*}$ also commutes with $\text{CStO}_H^{s^*, h^*}$ and the unitary operation which simulates H during the generation of b' in game $\mathbf{G}_{15}$, since they act on different database registers. Therefore, we have that $U_{g^*}^{r^*}$ commutes with both $\text{CStO}_G^{r^*, g^*}$ and $\text{CStO}_H^{s^*, h^*}$, and by a similar analysis we have that $U_{h^*}^{s^*}$ commutes with both $\text{CStO}_G^{r^*, g^*}$ and $\text{CStO}_H^{s^*, h^*}$. In addition, the adversary's operations performed before outputting b' also commute with $U_{g^*}^{r^*}$ and $U_{h^*}^{s^*}$, since the adversary cannot perform operations on the database registers.

Note that $U_{g^*}^{r^*}$ must commute with $U_{h^*}^{s^*}$ since they act on different database registers, hence we find that the $U_{g^*}^{r^*}$ and $U_{h^*}^{s^*}$ added in U_{Search}^{**} actually commute with all the other operations performed during the generation of b' in game $\mathbf{G}_{15}$. Now we can cancel out these two operations as follows, since they are both involutions:

- $U_{g^*}^{r^*}$ and $U_{h^*}^{s^*}$ performed before the generation of c^* can cancel out with the first two $U_{g^*}^{r^*}$ and $U_{h^*}^{s^*}$ performed in the first implementation of U_{Search}^{**} .
- The second $U_{g^*}^{r^*}$ and $U_{h^*}^{s^*}$ performed in the implementation of U_{Search}^{**} can cancel out with the $U_{g^*}^{r^*}$ and $U_{h^*}^{s^*}$ performed firstly in the implementation of the next U_{Search}^{**} .
- As for the second $U_{g^*}^{r^*}$ and $U_{h^*}^{s^*}$ performed in the final implementation of U_{Search}^{**} , by the commutativity analysis given in the previous paragraph, we can move these two operations to the end of the game and then delete them without affecting the adversary's output.

So after the above cancelation of $U_{g^*}^{r^*}$ and $U_{h^*}^{s^*}$, the performing of $U_{g^*}^{r^*}$ and $U_{h^*}^{s^*}$ just before the generation of c^* is canceled and U_{Search}^{**} will degrade into U_{Search}^* . Meanwhile, g^* and h^* are no longer used by the game and hence can be removed. That is, game $\mathbf{G}_{16}$ given above is actually a rewrite of game $\mathbf{G}_{15}$, hence

$$\Pr[1 \leftarrow \mathbf{G}_{15}] = \Pr[1 \leftarrow \mathbf{G}_{16}].$$

D.10 Proof of Lemma 8

By mimicking the analysis of the game hop from $\mathbf{G}_{15}^6$ to $\mathbf{G}_{15}^7$, $\mathbf{G}_{15}^7$ to $\mathbf{G}_{15}^8$ and $\mathbf{G}_{15}^8$ to $\mathbf{G}_{15}^9$ in the proof of Lemma 6, we can transform game $\mathbf{G}_{16}$ to game $\mathbf{G}_{17}$, and the corresponding bound is

$$|\Pr[1 \leftarrow \mathbf{G}_{16}] - \Pr[1 \leftarrow \mathbf{G}_{17}]| \leq 4(q_G + q_H + 1) \cdot \left(\sqrt{2^{-k_0}} + \sqrt{\text{Adv}_{\text{TDL}, \mathcal{B}_2}^{\text{PD-OW}}} \right),$$

where the PD-OW adversary $\mathcal{B}_2$ first chooses $j \leftarrow_{\$} [q_G + q_H]$ ²⁷, runs the adversary $\mathcal{A}$ in game $\mathbf{G}_{17}$ until its j -th query to $\mathbf{G}$ or $\mathbf{H}$ during the generation of b' , and then measures the query register to get its output x . The running time of $\mathcal{B}_2$ can be upper bounded by $\mathcal{T}_{\mathcal{A}} + \mathcal{O}(q_G^2 + q_H^2) + \mathcal{O}(q_D q_G q_H) \cdot \mathcal{T}_f$.

²⁷ Here we have assumed without loss of generality that, the total query number of $\mathbf{G}$ and $\mathbf{H}$ during the generation of b' in game $\mathbf{G}_{17}$ is exactly $q_G + q_H$ (if it does not, we can modify game $\mathbf{G}_{17}$ by adding a number of useless queries to $\mathbf{G}$ and $\mathbf{H}$ in the end).